Universidade Federal do Piauí
Centro de Educação Aberta e a Distância

ALGORITMOS E PROGRAMAÇÃO II

Magno Santos



Ministério da Educação - MEC
Universidade Aberta do Brasil - UAB
Universidade Federal do Piauí - UFPI
Universidade Aberta do Piauí - UAPI
Centro de Educação Aberta a Distância - CEAD

ALGORITMOS E PROGRAMAÇÃO II

Magno Santos

2010

PRESIDENTE DA REPÚBLICA	*Luiz Inácio Lula da Silva*
MINISTÉRIO DA EDUCAÇÃO	*Fernando Haddad*
GOVERNADOR DO ESTADO	*Wilson Nunes Martins*
REITOR DA UNIVERSIDADE FEDERAL DO PIAUÍ	*Luiz de Sousa Santos Júnior*
SECRETÁRIO DE EDUCAÇÃO A DISTÂNCIA DO MEC	*Carlos Eduardo Bielshowsky*
PRESIDENTE DA CAPES	*Jorge Almeida Guimarães*
COORDENADORIA GERAL DA UNIVERSIDADE ABERTA DO BRASIL	*Celso Costa*
DIRETOR DO CENTRO DE EDUCAÇÃO ABERTA A DISTÂNCIA DA UFPI	*Gildásio Guedes Fernandes*

CONSELHO EDITORIAL	*Prof. Dr. Ricardo Alaggio Ribeiro (Presidente)* *Des. Tomaz Gomes Campelo* *Prof. Dr. José Renato de Araújo Sousa* *Profª. Drª. Teresinha de Jesus Mesquita Queiroz* *Profª. Francisca Maria Soares Mendes* *Profª. Iracildes Maria de Moura Fé Lima* *Prof. Dr. João Renór Ferreira de Carvalho*

COORDENAÇÃO DE MATERIAL DIDÁTICO	*Cleidinalva Maria Barbosa Oliveira*
COORDENAÇÃO DE REVISÃO	*Naziozênio Antonio Lacerd*
PROJETO GRÁFICO	*Samuel Falcão Silva*
DIAGRAMAÇÃO	*Giselle da Silva Castro*
REVISÃO	*Francisca das Dores Oliveira Araújo*
REVISOR GRÁFICO	*Genuvina de Lima Melo Neta*

A responsabilidade pelo conteúdo e imagens desta obra é do autor. O conteúdo desta obra foi licenciado temporária e gratuitamente para utilização no âmbito do Sistema Universidade Aberta do Brasil, através da UFPI. O leitor se compromete a utilizar o conteúdo desta obra para aprendizado pessoal, sendo que a reprodução e distribuição ficarão limitadas ao âmbito interno dos cursos. A citação desta obra em trabalhos acadêmicos e/ou profissionais poderá ser feita com indicação da fonte. A cópia deste obra sem autorização expressa ou com intuito de lucro constitui crime contra a propriedade intelectual, com sansões previstas no Código Penal.

Apresentação

Este livro é destinado aos estudantes aprendizes que participam do programa de Educação a Distância da Universidade Aberta do Piauí (UAPI) vinculada ao consórcio formado pela Universidade Federal do Piauí (UFPI), Universidade Estadual do Piauí (UESPI) e Instituto Federal de Educação, Ciência e Tecnologia do Piauí (IFPI), com apoio do Governo do Estado do Piauí, através da Secretaria de Educação (SEDUC).

O texto deste compêndio é composto de quatro unidades, contendo quinze sub-unidades, estruturadas de modo sequencial, nas quais se discute sobre os fundamentos da programação de computadores orientada a objetos.

Na Unidade 1, discutiremos sobre os fundamentos de engenharia de *software* e da programação orientada a objetos, apresentando a anatomia de um programa orientado a objetos com seus elementos constituintes. Mostraremos os primeiros passos com a IDE *NetBeans* e como escrever um código para ler e escrever dados na entrada e saída padrão.

Na Unidade 2, serão apresentadas as estruturas de controle de um programa em *Java*; exemplificaremos as estruturas de repetição incremental, repetição com teste no início e repetição com teste no final, além das estruturas de desvio condicional.

Na Unidade 3, abordaremos as estruturas de dados de arranjos homogêneos e heterogêneos. Trabalharemos exemplos com vetores, matrizes e adicionalmente passagem de parâmetros como argumentos para um programa

Na Unidade 4, serão expostos conteúdos sobre o paradigma da programação orientada a objetos. Focaremos nas definições e exemplos de classe, objeto, mensagem; e ainda, apresentaremos características

Sumário

09 **UNIDADE 1**
Fundamentos de Programação Orientada a Objetos

Introdução à Engenharia de *Software* 11
Introdução à Programação de Computadores..... 25
Histórico da Linguagem *Java* 32
Instalação do Java e do *NetBeans* 37
Primeiro Programa *Java* com a IDE *NetBeans* 42
Entrada e Saída de Dados..... 54

62 **UNIDADE 2**
Estruturas de Controle

Estrutura de Controle de Decisão: *if-then-else, switch* 65
Estrutura de Controle de Repetição: *while, do-while, for*..... 75
Estrutura de Interrupção: *break*, *continue*, *return* 82

89 **UNIDADE 3**
Arranjos e Argumentos

Array em *Java*..... 91
Argumentos em *Java*..... 98

105 **UNIDADE 4**
Paradigma de Programação Orientada a Objetos

Classes, Objetos e Métodos 107
Definição de Classe em *Java*..... 128
Herança, Polimorfismo e *Interfaces* 149
Tratamento Básico de Exceções..... 163

Referências Bibliográficas

UNIDADE 01

Fundamentos de Programação Orientada a Objetos

OBJETIVOS:

- **Orientar na geração de um código simples, dentro dos padrões mínimos exigidos pela Engenharia de *Software*.**

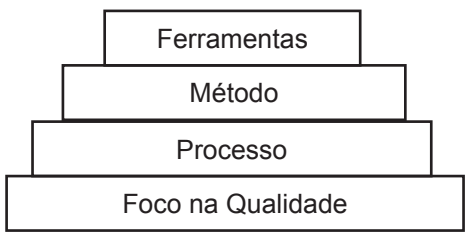
1 FUNDAMENTOS DE PROGRAMAÇÃO ORIENTADA A OBJETOS

Introdução à Engenharia de *Software*

Engenharia de *Software*: uma visão em camadas

Engenharia de Software é uma disciplina que aplica princípios da engenharia de desenvolvimento na qualidade do *software* em um determinado tempo e com um custo efetivo, usando uma abordagem sistemática e metodológica para produzir resultados que possam ser quantificados. Faz uso de medição e métricas para avaliar a qualidade, não somente do *software*, mas também do processo, e também utilizada para avaliar e gerenciar projetos de desenvolvimento de *software*.

Engenharia de *software* é vista de modo diferente pelos diversos profissionais. Pressman tem uma visão da engenharia de *software* como uma camada tecnológica 1. Essa visão consiste em quatro camadas: foco na qualidade, processo, método e ferramentas. A Figura abaixo ilustra essa visão.



Foco na qualidade

Essa camada busca um total foco na qualidade. É uma cultura em que o compromisso com a melhoria no processo de desenvolvimento do *software* é sustentado, permitindo o desenvolvimento de mais abordagens

efetivas para engenharia de *software*.

Processo

Define uma estrutura, que consiste em áreas de processos-chave, que define e permite a entrega racional a tempo de um software. Áreas de processos chave são a base para o gerenciamento de projeto de software. Estabelecem que métodos técnicos sejam aplicados, quais ferramentas são usadas, que produtos de trabalho precisam ser produzidos, e que marcos são definidos. Incluem a garantia que a qualidade será mantida, e que a mudança é devidamente controlada e gerenciada.

Método

Métodos definem procedimentos sistemáticos e ordenados de construção de *software*, proporcionando uma estrutura global interna em que as atividades do engenheiro de software são realizadas. Essas atividades incluem um conjunto amplo de tarefas, tais como: análise de requisitos, *design*, construção do programa, teste e manutenção.

Metodologia é a ciência de pensamento sistemático, que usa métodos ou procedimentos para uma disciplina em particular. Existem várias metodologias da engenharia de *software* que são usadas atualmente. Algumas delas estão enumeradas abaixo:

Metodologias estruturadas:

- Informações de engenharia
- Desenvolvimento do ciclo de vida do software/ciclo de vida do projeto
- Metodologia de Desenvolvimento de Aplicação *Rapid*
- Metodologia de Desenvolvimento de Aplicação *Joint*
- Método CASE*

Metodologias orientadas a objeto:

- Método *Booch*
- Método *Coad* e *Yourdon*
- Método *Jacobson*
- Método *Rambaugh*
- Método *Wirfs-Brock*

Ferramentas

Promovem o suporte aos processos e métodos. Ferramentas CASE (*Computer Aided Software Engineeing*) proporcionam um sistema de suporte ao projeto de desenvolvimento, em que as informações criadas por uma ferramenta podem ser usadas por outras. Podem ser automáticas ou semiautomáticas.

Muitas ferramentas são usadas para desenvolver modelos. Modelos são patterns (padrões) de algo que foi criado ou são simplificações. Existem dois modelos que geralmente são desenvolvidos por um engenheiro de *software*, especialmente, o modelo de sistema e o modelo de *software*. O modelo de sistema é uma representação acessível de um sistema complexo que precisa ser estudado, enquanto o modelo de *software* é chamado de *blueprint* do *software* que precisa ser construído. Assim como as metodologias, vários modelos de ferramentas são usados para representar sistemas e *softwares*.

Alguns estão descritos abaixo.

Abordagem de modelos de ferramentas estruturada:

- Diagrama de Entidade-Relacionamento
- Diagrama de fluxo de dados
- Pseudocódigo
- Fluxograma

Abordagem de modelo de ferramenta orientada a objeto:

- Linguagem de Modelagem Unificada (UML)

Qualidade dentro do esforço de desenvolvimento

Conforme mencionado anteriormente, a qualidade é a mente que influencia todo engenheiro de *software*. Focando na qualidade em todas as atividades de engenharia de *software*, reduz-se custo e melhora-se o tempo de desenvolvimento pela minimização de um novo trabalho de correção. Para proceder dessa forma, um engenheiro de *software* precisa definir explicitamente que qualidade de *software* é ter um conjunto de atividades assegurando que todo produto de trabalho da engenharia de software exibe alta qualidade, fazer controle de qualidade e atividades garantidas, o uso de métricas para desenvolver estratégias para melhorar o produto de *software* e o processo.

O que é qualidade?

Qualidade é a característica total de uma entidade para satisfazer necessidades declaradas e implícitas. Essas características ou atributos devem ser mensuráveis de modo que possam ser comparados por padrões conhecidos.

Como definimos qualidade?

Três perspectivas são usadas na compreensão da qualidade, especialmente, olhamos a qualidade do produto, do processo e no contexto do ambiente de negócios.

Qualidade do produto

É muito relativo analisar qualidade, pois apresenta significados diferentes para cada pessoa. Para os usuários finais, o *software* possui qualidade se fornecer-lhes o que desejam e quando desejam o tempo todo. Também é julgado baseado na facilidade de uso e na forma de aprendizagem de como usá-lo.

Normalmente, os usuários avaliam e categorizam o *software* com base em características externas, tal como número de falhas por tipo. Essas falhas podem ser categorizadas como insignificantes, ou importantes ,e catastróficas. Para que outros possam desenvolver e manter o *software*, estes devem ficar de olho nas características internas em vez das externas. Exemplos que incluem erros e falhas encontradas durante as fases de análise de requisitos, *design*, e codificação são normalmente feitos anteriormente ao carregamento dos produtos para os usuários finais.

Como engenheiros de *software*, devemos construir modelos baseados em como os requisitos dos usuários externos serão relacionados com os requisitos internos dos desenvolvedores.

Qualidade do processo

Existem várias tarefas que afetam a qualidade do *software*. Às vezes, quando uma tarefa falha, a qualidade do software falha. Como engenheiros de *softwares*, devemos validar a qualidade no processo de desenvolvimento do *software*. Regras de processo sugerem que

pela melhoria do processo de desenvolvimento do *software*, também há melhora da qualidade do produto resultante. Algumas regras de processo são demonstradas abaixo:

• *Capability Maturity Model Integration*(CMMI) - foram formulados pelo *Software Engineering Institute* (SEI). É um processo meta-modelo, que é baseado em um conjunto de sistemas e competências da engenharia de software que devem existir dentro de uma organização. Essa organização atinge diferentes níveis de capacidade e maturidade desses processos de desenvolvimento.

• ISSO 9000:2000 para *Software* - é um padrão genérico, aplicado para qualquer organização que queira melhorar a qualidade global dos produtos, sistemas ou serviços que proporciona.

• *Software Process Improvement e Capability Determination* (SPICE) - é um padrão que define um conjunto de requisitos para avaliação do processo de software. O objetivo desse padrão é auxiliar organizações a desenvolver uma análise objetiva da eficácia de qualquer processo de *software* definido.

Nessa perspectiva, qualidade é visualizada em termos de produtos e serviços, sendo proporcionada pelo negócio em que o *software* é usado. Melhorando a qualidade técnica dos processos de negócio, agrega-se valor ao negócio, por exemplo, valor técnico do *software* traduz o valor do negócio. Também é importante medir o valor do *software* em termos de terminologias de negócio, tal como “quantos pedidos de venda foram processados hoje?”, valor do dólar sobre o retorno em cima dos investimentos (ROI), etc. Se o *software* não agrega valor ao negócio, qual a necessidade de tê-lo em primeiro lugar?

Como endereçamos os pontos importantes sobre qualidade?

Podemos endereçar os pontos importantes sobre qualidade em:

1. Uso de padrões de qualidade. Padrões de qualidade são um conjunto de princípios, procedimentos, metodologias e regras para resumir, sobre qualidade no processo, tais como CMMI, ISO 9000:2000 para software e SPICE.

2. Compreender pessoas envolvidas no processo de desenvolvimento incluindo usuários finais e participantes. Sustenta um ambiente de colaboração e comunicação efetiva.

3. Compreender as tendências sistemáticas na natureza humana tal como as pessoas tendem a ser contrárias ao risco quando existe

uma perda potencial, são indevidamente otimistas em seus planos e projeções, e preferem usar julgamentos intuitivos em vez de modelos quantitativos.

4. Engajamento para a qualidade. Uma mente focada sobre qualidade é necessária para descobrir erros e defeitos assim que possam ser endereçados imediatamente.

5. Requisitos de usuários administradores porque mudarão ao longo do tempo. Requisitos são a base para definir as características da qualidade de *software*.

Técnicas e garantias de qualidade de software

Garantia de qualidade de *Software* é um subconjunto da engenharia de *software* assegurando que todos os produtos de trabalho sejam realizados, e que cumpram com as exigências e padrões estabelecidos pelos usuários. É considerada uma das atividades mais importantes, sendo aplicada durante todo o processo do desenvolvimento do *software*. O objetivo é detectar defeitos antes do *software* ser entregue como um produto acabado para o usuário final. Isto abrange uma aproximação eficaz da gerência de qualidade, tecnologia de engenharia de software (métodos e ferramentas), técnicas formais de revisão, várias estratégias de teste, controle de documentação de *software* e alterações feitas, um procedimento para assegurar a conformidade com os padrões de desenvolvimento de *software*, e um mecanismo para mensurá-los e documentá-los.

Qualidade de *software*

Um *software* possui qualidade se ele estiver ajustado para uso, isto é, se estiver trabalhando corretamente. Para que ele trabalhe corretamente, ele deve estar em conformidade com os requisitos funcionais e de performance características externas dos usuários), padrões explicitamente documentados de desenvolvimento (padrões de qualidade), e características implícitas (características internas aos desenvolvedores) que são esperadas por todo desenvolvimento profissional de *software*.

Três pontos importantes enfatizados para definir a qualidade do *software*.

1. Requisitos de *software* são a base para a qualidade do software.

É necessário explicitar, especificar e priorizar.

2. Padrões definem um de critérios de desenvolvimento que irão mostrar a maneira com a qual o *software* será desenvolvido.

3. Características implícitas deverão ser identificadas e documentadas; elas influenciam na maneira de como o *software* será desenvolvido assim como sua manutenibilidade.

Características para uma boa engenharia de software

Para definir uma boa engenharia de *software*, dê uma olhada nas características específicas que o software apresenta. Algumas delas estão enumeradas abaixo:

• Usabilidade - é a característica do *software* de apresentar facilidades entre a comunicação dos usuários com o sistema.

• Portabilidade - é a capacidade do *software* ser executado em diferentes plataformas e arquiteturas.

• Reusabilidade - é a habilidade do *software* de se transferir de um sistema para outro.

• Manutenibilidade - é a habilidade do *software* de se envolver e adaptar-se às alterações em um curto espaço de tempo. É caracterizado pela fácil atualização e manutenção.

• Dependência - é a característica do *software* ser confiável e de segurança

• Eficiência - é a capacidade do *software* utilizar os recursos com maior eficiência.

Atividades da garantia de qualidade de software

Garantia de Qualidade de *Software* é composta por uma variedade de atividades com o objetivo de construir *software* com qualidade. Isto envolve dois grupos de desenvolvedores e a equipe de SQA (*Software Quality Assurance)*. A equipe de SQA tem responsabilidade em garantir plenamente à qualidade, supervisionar, manter, analisar e reportar defeitos. As atividades envolvidas são as seguintes:

1. A equipe de SQA prepara o plano de SQA. Isto se dá durante a fase de planejamento de projeto. Identificam-na:

• Avaliação a ser executada;

• Auditorias e revisões a serem executadas;

• Padrões que devem ser aplicados;

• Procedimentos de erros reportados e monitorados;
• Documentos que devem ser produzidos;
• Conjunto de respostas que se fizer necessário.

2. A equipe de SQA participa na descrição do processo de desenvolvimento de *software*. Já o time de desenvolvedores escolhe o processo de desenvolvimento, e a equipe de SQA deve verificar se ele se enquadra na política organizacional e nos padrões de qualidade

3. A equipe de SQA revisa as atividades de engenharia de *software* empregadas pelo time de desenvolvedores para checar a conformidade com o processo de desenvolvimento de *software*. Eles monitoram e seguem desvios do processo do desenvolvimento do *software*; documentam-no e asseguram-se de que as correções sejam feitas.

4. A equipe de SQA revê o trabalho para verificar se estão conforme o padrão definido. Eles monitoram e marcam defeitos e falhas encontrados em cada trabalho.

5. A equipe de SQA assegura-se que os desvios nas atividades de software e no processo de produção estejam seguramente baseados na definição de procedimentos e padrões de operação

6. A equipe de SQA envia desvios e desconformidades aos padrões para os gerentes ou a quem for de interesse.

Técnicas formais de revisão

Produtos de trabalho são as saídas esperadas como resultado da execução de tarefas no processo de desenvolvimento de *software*. Esses resultados contribuem para o desenvolvimento de *software* com qualidade. Consequentemente, devem ser mensurados e verificados novamente se vão ao encontro das exigências e dos padrões. As alterações nos produtos de trabalho são significativas; elas podem ser monitoradas e controladas. A técnica de checar a qualidade dos produtos de trabalho é a técnica formal de revisão. *Formal Technical Reviews* (FTR) são executadas em vários pontos do processo do desenvolvimento do *software*. Ela serve para descobrir erros e defeitos que podem ser eliminados antes do software ser enviado para o usuário final. Especificamente, seus objetivos são:

1. Descobrir erros em funções, na lógica ou na execução para toda a representação do *software*;

2. Verificar se o *software* sob a revisão encontra-se de acordo com os requisitos do usuário;

3. Assegurar-se que o *software* esteja de acordo com os padrões definidos;

4. Conseguir que o *software* seja desenvolvido de uma maneira uniforme; e

5. Desenvolver projetos mais gerenciáveis.

Um guia geral de condução das técnicas formais de revisão está listado abaixo.

• Revisar o produto de trabalho e NÃO o desenvolvedor do produto de trabalho. O objetivo da revisão e descobrir erros e defeitos para melhorar a qualidade do *software*, e o tom da revisão pode ser brando, porém construtivo.

• Planejar e cumprir a agenda. Revisões não devem durar mais de duas horas.

• Minimizar os debates e discussões. É inevitável que os problemas sejam levantados e isso não cause efeito nas pessoas, lembre a todos que não é hora de resolver os problemas que serão apenas documentados, uma outra reunião deve ser agendada para resolvê-los.

• Indique áreas de problema, mas não tente resolvê-las. Mencione e esclareça áreas de problema, entretanto, não é hora de resolver problemas, deverão ser resolvidos em uma outra reunião.

• Tome nota. É uma boa prática tomar nota do que foi dito e suas prioridades para que elas possam ser vistas por outros revisores; isso ajudará a esclarecer os defeitos e ações a serem tomadas.

• Mantenha o número dos participantes a um mínimo e insista em preparar-se para a revisão. Escrever comentários e observações pelos revisores é uma boa técnica.

• Forneça uma lista de verificação para o produto de trabalho que é provável ser revista. A lista de revisão provê uma estrutura que conduz a a revisão; isto também ajuda os revisores a manterem o foco na questão.

• Programe as revisões como parte do processo de desenvolvimento de software e assegure-se de que os recursos sejam fornecidos para cada revisor. Preparação prevê interpretações em uma reunião, ajudando os revisores a manterem o foco na questão.

• Sumário da revisão. Verifica a eficácia do processo da revisão.

Duas técnicas formais de revisão do produto de trabalho usadas na indústria são *Fagan's Inspection Method e Walkthroughs*.

Método de inspeção de Fagan

Introduzido por Fagan em 1976 na IBM, originalmente foi utilizado para verificar códigos de programas. Entretanto, pode ser estendido para incluir outros produtos de trabalho como técnicas de documentos, modelo de elementos, projetos de códigos, dados etc. Isso é gerenciado por um moderador que é responsável por supervisionar a revisão e requer uma equipe de inspetores designados a verificar se as regras do produto de trabalho vão de encontro à lista de interesse preparada. É mais formal que o *walkthrough*.

A seguir estão descritas regras determinadas na qual cada participante deverá aderir:

• As inspeções são realizadas em um número de pontos no processo do planejamento do projeto e do desenvolvimento dos sistemas.

• Todas as classes com defeito são documentadas e os produtos do trabalho são inspecionados não somente a nível lógico, de especificações ou de funções de erros.

• A inspeção é realizada por colegas em todos os níveis, exceto o chefe.

• As inspeções são realizadas em uma lista prescrita das atividades.

• As reuniões de inspeção são limitadas a duas horas.

• As inspeções são conduzidas por um moderador treinado.

• Inspetores são designados a especificar regras para aumentar a eficácia. As listas de verificação dos questionários a serem perguntados pelos inspetores são usadas para definir tarefas e estimular a encontrar defeitos, e os materiais são inspecionados minuciosamente para que seja encontrado o máximo número de possíveis erros.

• Estatísticas com os tipos de erros são vitais e utilizadas para obter análises de uma maneira similar à análise financeira.

Conduzir inspeções requer muitas atividades. Elas estão categorizadas a seguir:

• Planejamento. O moderador deve se preparar para a inspeção, decidindo quem serão os inspetores e as regras que estes devem obedecer, quem e quando desempenharão seus papéis e distribuir a documentação necessária.

• Uma rápida apresentação. Apenas 30 minutos de apresentação do projeto dos inspetores é o suficiente, e isso pode ser omitido se todos estiverem bem familiarizados com o projeto.

• Preparando. Cada inspetor terá de 1 a 2 horas sozinho para inspecionar o produto de trabalho e irá executar as regras passadas a ele com base na documentação provida pelo moderador, tentando descobrir defeitos no produto de trabalho. O inspetor não deverá reparar defeitos ou criticar o desenvolvedor do produto de trabalho.

• Realizando a reunião. Os participantes da reunião são inspetores, moderadores e desenvolvedores do produto de trabalho. Os desenvolvedores do produto de trabalho estão presentes para explicar o produto de trabalho, e responder às perguntas que os inspetores fizerem. Nenhuma discussão se o defeito é ou não real é permitida. Uma lista de defeitos deve ser produzida pelo moderador.

• Refazendo o produto de trabalho. A lista de defeitos deve ser atribuída a uma pessoa para repará-la. Normalmente, essa pesoa é o desenvolvedor do produto de trabalho.

• Acompanhando os reajustes. O moderador assegura-se que os defeitos nos produtos de trabalho sejam endereçados e solucionados. Mais tarde esse moderador deve ser inspecionado por outro inspetor.

• Realizando uma reunião ocasional de análise. Isso é opcional, momento onde é dada a possibilidade aos inspetores de expressarem sua visão pessoal sobre erros e melhorias. A ênfase é dada à maneira que a inspeção foi feita.

Walkthrough

O *walkthrough* é menos formal que a inspeção. Aqui, o produto de trabalho e sua documentação correspondente são entregues para um time de revisores, normalmente em torno de 3 pessoas, onde comentários de sua exatidão são apresentados. Ao contrário da inspeção onde um é o moderador, o desenvolvedor do produto de trabalho coordena o walkthrough. Um escrivão também deve estar presente para documentar a lista de ações. Uma lista de ações deve ser feita a fim de melhorar a qualidade do produto final a qual inclui ajustes dos defeitos, resoluções dos problemas etc.

Alguns passos devem ser seguidos para obter sucesso no walkthrough. Eles estão listados abaixo:

• Nenhum gerente deve estar presente;

• Enfatizar que o *walkthrough* é para detecção de erros e não para correção;

• Manter o interesse do grupo;

• Nenhuma contagem ou atribuição de nota;
• Criticar o produto; não a pessoa;
• Sempre documentar a lista de ações.

Conduzir o *walkthrough* é similar à inspeção, requer muitas atividades. Elas estão categorizadas como se segue:

• Antes do *walkthrough;*

O desenvolvedor do produto de trabalho agenda o *walkthrough*, preferivelmente, com um dia de antecedência ou dois no máximo.

Distribuir o material necessário para o produto de trabalho dos revisores.

Pede-se especificamente que cada revisor traga pelo menos dois comentários positivos do *walkthrough* e um comentário negativo sobre o produto do trabalho.

• Durante o *walkthrough;*

O desenvolvedor do produto de trabalho faz uma rápida apresentação do seu produto de trabalho. Esse passo pode ser ignorado caso os revisores conheçam bem o produto de trabalho.

Solicitar comentários aos revisores. Às vezes, problemas são levantados e apresentados, mas não devem ser solucionados durante o walkthrough. Esses problemas deverão ser incluídos em uma lista de ações.

Uma lista de ações deve ser produzida até o fim do *walkthrough*.

• Após o *walkthrough*;

O desenvolvedor do produto de trabalho recebe a lista de ações.

Pede-se para enviar os estados das ações com o objetivo de resolver erros ou discrepâncias apresentadas na lista de ações.

Possivelmente, um outro *walkthrough* deve ser agendado.

Documentação no esforço de desenvolvimento

O que é documentação?

É um conjunto de documentos ou informações do produto que descrevem o sistema. Cada documento é desenhado para executar uma função específica, como:

• REFERÊNCIA, como por exemplo, especificações técnicas ou funcionais.

• INSTRUCIONAL, como por exemplo, tutoriais, demonstrações

ou protótipos.

• MOTIVACIONAL, como por exemplo, brochuras, demonstrações ou protótipos.

Há vários tipos de documentação e informações funcionais do produto. Alguns são citados abaixo:

• Características e funções do sistema
• Sumário gerencial e do usuário
• Manual do usuário
• Manual de administração do sistema
• Vídeo
• Multimídia
• Tutoriais
• Demonstrações
• Guia de referência
• Guia de referência rápida
• Referências técnicas
• Arquivos de manutenção do sistema
• Modelos de teste do sistema
• Procedimentos de conversão
• Manual de operações/operador
• *Help on-line*
• *Wall charts*
• *Layout* de teclado ou *templates*
• Jornais

Bons documentos não geram sistemas complicados. No entanto, eles podem ajudar de outra forma. A tabela seguinte mostra como a documentação ajuda no processo de desenvolvimento de software.

Se o manual do usuário for desenvolvido durante....	**Então, o manual pode...**
A definição do produto	• Clarear procedimentos e regras. • Identificar elementos não amigáveis. • Aumentar as chances de satisfação do cliente.
O projeto e codificação	• Clarear problemas e erros. • Identificar causas de inconsistências. • Forçar o projetista a tomar decisões mais cedo.

A distribuição e uso do sistema	• Ajudar os usuários a se adaptarem ao sistema. • Alertar sobre problemas no sistema. • Negar responsabilidades.

Existem dois principais propósitos da documentação. Especificamente, eles:

• Fornecem um argumento racional e permanente para a estrutura do sistema ou comportamento através dos manuais de referência, guia do usuário e documentos de arquitetura do sistema.

• Servem como documentos transitórios que são parte de uma infraestrutura envolvida em uma execução de um projeto real como: cenários, documentação do projeto interno, relatório de reuniões e problemas.



Exercícios Propostos

1. Discuta a visão em camadas, tendo em vista no gerenciamento e desenvolvimento do software.
2. Qualidade do software é a característica que satisfaz as necessidades declaradas e implícitas do contratante. Como mensurar essa característica de modo que possa ser comparada a padrões conhecidos?

3. Pesquisar, na Internet exemplos de documentação de software como: a) manual do usuário, b) manual de administração do sistema, c) vídeo, multimídia, tutoriais, d) demonstrações, e) arquivos de aanutenção do sistema, f) manual de operações/operador, g) help on-line, wall charts.
4. Apresente um algoritmo (em fluxograma ou em passos lógicos) do fluxo de desenvolvimento do software. Apresente as fases e atividades importantes para garantir a qualidade do software.
5. Documente o código abaixo.

```
public class Documento {
public static void main( String args[] ) {
  System.out.println( "Código documentado!" ); {
  }
}
```

Desafio:

• A equipe de desenvolvimento é responsável pela construção do software que irá dar suporte ao sistema de informação solicitado. Em geral é formado por analista de sistemas, projetista, programadores e testadores. Qual o papel de cada um no processo de engneharia do software.

Introdução à Programação de Computadores

O que é uma linguagem de programação?

Linguagem de programação é uma técnica de comunicação padronizada para expressar instruções para um computador. Assim como os idiomas utilizados pelos seres humanos, cada linguagem tem sua própria sintaxe e gramática.

Linguagens de programação possibilitam ao programador especificar precisamente com quais dados o computador irá interagir, como esses dados serão gravados/transmitidos, e precisamente quais ações serão tomadas de acordo com as circunstâncias.

Existem diferentes tipos de linguagens de programação que podem ser usadas para a criação de programas, mas, independente da linguagem utilizada, essas instruções são traduzidas em linguagem de máquina, e podem ser entendidas por computadores.

Categorias das linguagens de programação

Linguagens de programação de alto nível

Uma linguagem de programação de alto nível é uma linguagem de programação que é mais amigável para o usuário, em alguns casos independente de plataforma, e que abstrai operações de baixo nível como acesso a memória. Uma instrução de programação pode ser traduzida em uma ou várias instruções de máquina por um compilador.

Exemplos são Java, C, C++, Basic, Fortran

Linguagens de montagem de baixo nível

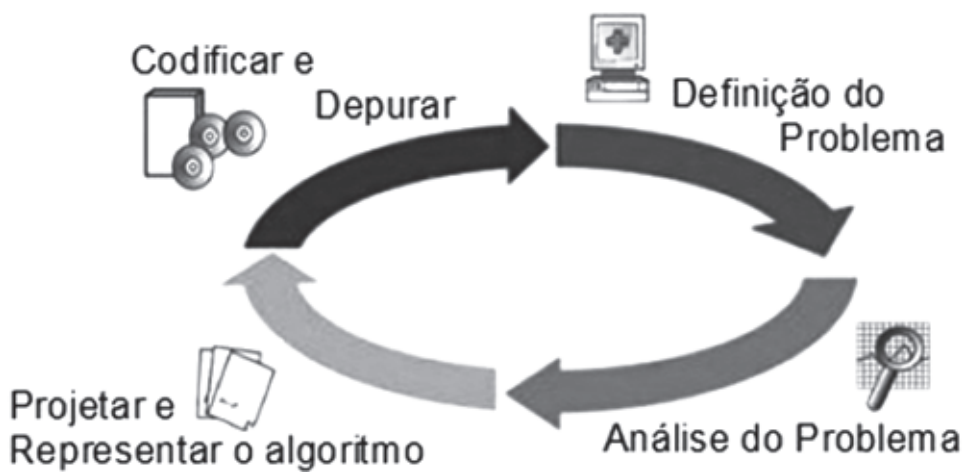
Linguagens de montagem são similares às linguagens de máquina, são mais simples de programar, pois possuem poucos comandos e permitem ao programador substituir nomes por números. Linguagens de montagem estão disponíveis em cada família de CPU, e cada instrução de montagem é traduzida em uma instrução de máquina por um programa montador.

> **Nota**: Os termos "alto nível" e "baixo nível" são relativos. Originalmente, linguagens de montagem eram consideradas de baixo nível e COBOL, C, etc. eram consideradas de alto nível. Muitos programadores, hoje em dia, podem se referir a estas últimas como linguagens de baixo nível.

O ciclo de vida do desenvolvimento de programas

Programadores não sentam e simplesmente começam a escrever código de uma vez, quando estão tentando fazer um programa de computador. Ao invés disso, eles seguem um planejamento organizado ou metodologia, que quebra o processo em uma série de tarefas.

Este é o ciclo de vida quando se tenta resolver um problema no computador:



Para entendermos o funcionamento desse ciclo na solução de problemas no computador, vamos definir um problema exemplo que iremos resolver passo a passo enquanto discutimos as metodologias para resolução de problemas em detalhe.

Definir o problema

Geralmente, um programador recebe uma tarefa na forma de um problema. Antes do programa ser projetado para resolver um problema em particular, o problema deve, em primeiro lugar, ser bem e claramente definido em termos dos seus requisitos de entrada e saída.

Um problema claramente definido já é metade da solução. Programação de computadores requer que o problema seja primeiro definido antes de se pensar em criar a solução.

Vamos definir o problema exemplo:

"Crie um programa que irá determinar o número de vezes que um nome aparece em uma lista."

Analisar o problema

Depois do problema ter sido definido adequadamente, o mais simples e também o mais eficiente e efetivo meio de se resolver será visualizá-lo através de uma representação clara e objetiva.

Geralmente, esse passo se dá com a quebra do problema em subproblemas menores e mais simples.

Problema exemplo:

Determinar o número de vezes que um nome aparece em uma lista

Entrada para o programa:

Lista de nomes, nome que se deseja procurar

Saída do programa:

O número de vezes que o nome aparece em uma lista

Projetar e representar o algoritmo

Logo que o problema estiver sido claramente definido, podemos nos concentrar em desenvolver a solução. Na programação de computadores, geralmente é requerido que expressemos a solução passo a passo.

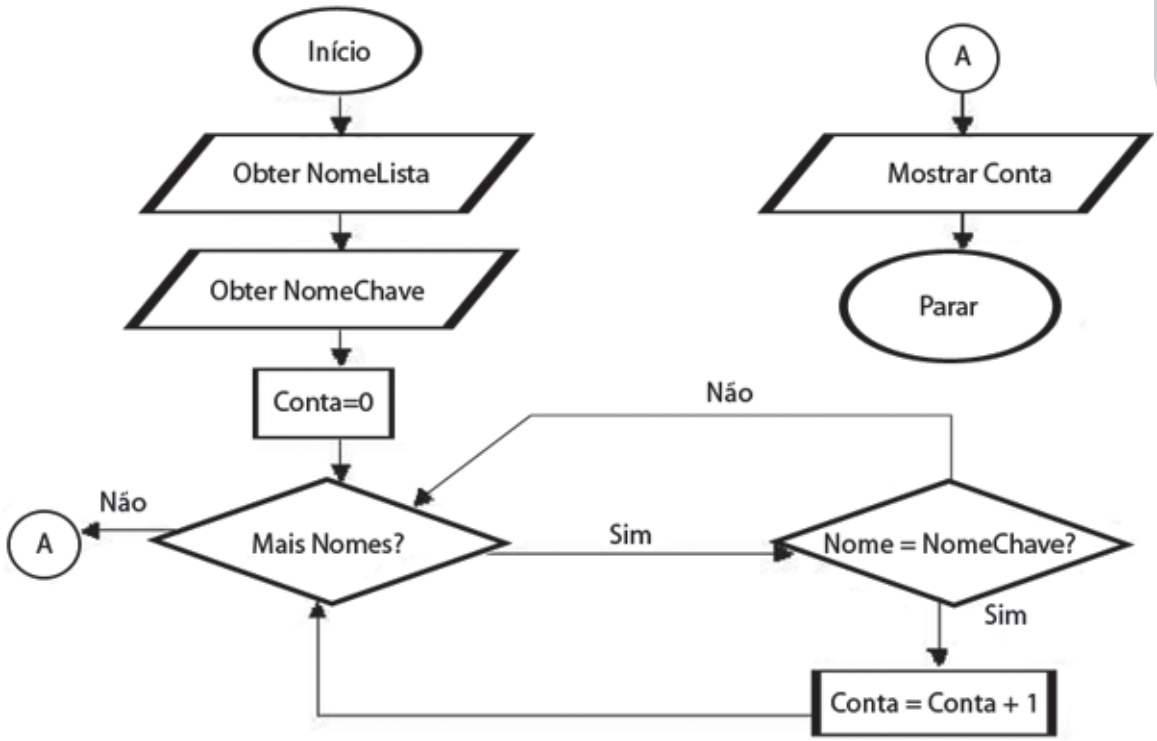
Um Algoritmo é uma especificação clara e não ambígua dos passos necessários para se resolver o problema. Ele pode ser expresso tanto em linguagem humana (Inglês, Tagalog e Português), como através de representação gráfica como fluxograma ou através de pseudocódigo, que é um meio termo entre a linguagem humana e a linguagem de programação.

Dado o problema definido na seção anterior, como podemos expressar a solução de uma maneira simples e que possa ser entendida?

Expressando a solução através da linguagem humana:

1. Obter a lista de nomes, vamos chamá-la de **NomeLista**;
2. Obter o nome a ser procurado, vamos chamá-lo de **NomeChave**;
3. Criar um contador, vamos chamá-lo de **Conta**;
4. Pegar cada nome em **NomeLista**;
5. Se NomeChave for igual ao nome selecionado em **NomeLista**;
6. Adicionar 1 a **Conta**;
7. Repetir 4 até que todos os nomes já tiverem sido comparados;
8. Exibir o valor de **Conta**.

Expressando a solução através de um fluxograma:



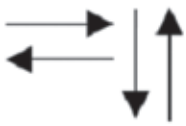
Expressando a solução através de pseudocódigo:

```
Fazer NomeLista = Lista de Nomes
Fazer NomeChave = o nome a ser procurado
Fazer conta = 0
Para cada nome em NomeLista fazer
  Se nome é igual a NomeChave
    Fazer Conta = Conta + 1
Mostrar Conta
```

Símbolos do fluxograma e o seu significado

Um fluxograma é uma ferramenta de projeto usada para representar graficamente a lógica de uma solução. Os fluxogramas, tipicamente, não mostram comandos de linguagem de programação. Ao invés disto, eles mostram o conceito em Português ou em notação matemática.

Aqui estão algumas dicas dos símbolos mais usados para a criação de fluxogramas. Pode-se utilizar quaisquer símbolos quando criar os seus fluxogramas, desde que sejam usados de maneira consistente.

Símbolo	Nome	Significado
	Símbolo de Processo	Representa o processo de se executar uma operação definida ou grupo de operações que resultam em mudança de valor, forma ou localização da informação. Também funciona como símbolo padrão quando nenhum outro símbolo estiver disponível.
	Símbolo de Entrada/Saída (E/S)	Representa a função de E/S que faz com que os dados fiquem disponíveis para processamento (entrada) ou para exibição (saída) das informações processadas.
	Símbolo de Linha	Representa a sequencia de informações disponíveis e operações executáveis. As linhas conectam outros símbolos, e as setas são obrigatórias somente em fluxos com orientação da direita para esquerda e de baixo para cima.
	Símbolo de Anotação	Representa a adição de informação descritiva, comentários, ou notas explicativas para esclarecimentos. Alinha vertical e a linha pontilhada podem ser colcoadas à esquerda, como mostrado, ou a direita.
	Símbolo de Decisão	Representa a decisão que determina qual das alternativas será seguida.
	Símbolo Terminal	Representa o começo, o final, um ponto de interrupção ou um atraso em um programa.
	Símbolo Conector	Representa qualquer entrada, ou saída, a outra parte do fluxograma. Também serve como um conector fora de página.
	Símbolo de Processo Pré-definido	Representa um processo nomeado consistindo de uma ou mais operações ou passos de programa especificados em algum outro lugar.

Codificar e depurar

Depois de construir o algoritmo, será possível criar o código fonte. Usando o algoritmo como base, o código fonte pode ser escrito usando a linguagem de programação escolhida.

Na maioria das vezes, depois do programador ter escrito o programa, este poderá não estar funcionando 100% no início, mas o programador pode corrigir o programa no caso de erros (também conhecidos como Erros de Compilação) que ocorrem no programa. Esse processo é chamado de depuração de erros (debug).

Existem dois tipos de erros que os programadores poderão encontrar. O primeiro é o erro em tempo de compilação e o outro é o erro em tempo de execução.

Erro em tempo de compilação ocorre se houver um erro de sintaxe no código. Nesse caso, o compilador irá detectar o erro e o programa nem mesmo compilará, sendo que o programador ficará inapto a criar um executável que possa ser executado pelo usuário até que o erro seja corrigido.

Esquecer um ponto-e-vírgula no final de uma instrução ou escrever um comando erroneamente, por exemplo, são erros em tempo de compilação. É algo que o compilador pode detectar como sendo um erro.

Compiladores não são perfeitos e então não podem detectar todos os erros em tempo de compilação. Isso é especialmente verdadeiro para erros de lógica como as repetições (*loops*) infinitos. Este tipo de erro é chamado de erro em tempo de execução.

Por exemplo, a sintaxe do código pode estar correta. Entretanto, ao seguir a lógica do código, o mesmo pedaço de instrução é executado várias e várias vezes, infinitamente. Assim, os compiladores não são espertos o suficiente para pegar todos estes tipos de erro em tempo de compilação, consequentemente, o programa compila corretamente em um arquivo executável.

Porém, quando o usuário final roda o programa, o programa (ou mesmo o computador inteiro) congela devido a uma repetição infinita. Outros tipos de erro em tempo de execução são: um valor errado a ser

Exercícios Propostos



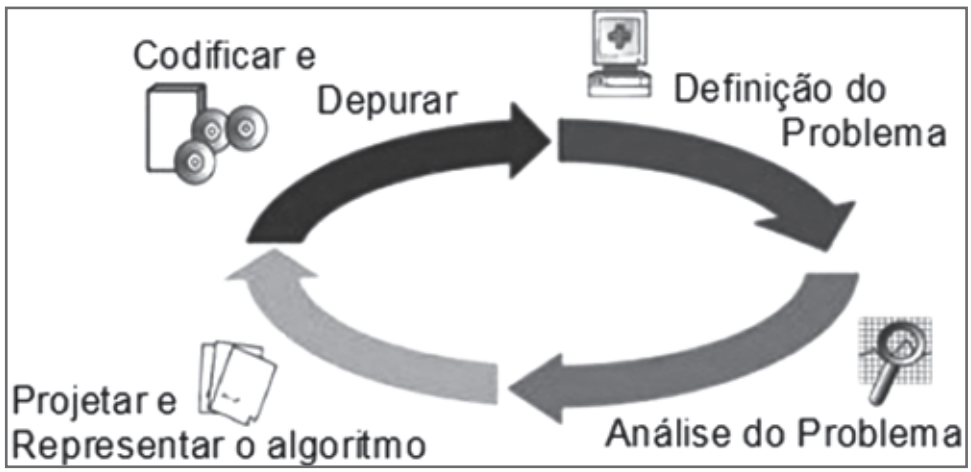
1. Escrevendo algoritmos: Dado o seguinte conjunto de tarefas, crie um algoritmo para realizar cada uma das tarefas abaixo. Escreva os algoritmos usando pseudocódigo ou fluxogramas.

a) Assar pão

b) Acessar o computador

c) Obter a média de três números

2. Com base na figura abaixo, defina um problema simples, faça a análise de como o problema será resolvido, represente o problema em fluxograma e codifique em pseudocódigo (portugol).



Desafio:

• Suponha que um certo instituto deseja fazer uma pesquisa de opinião pública. Os dados serão coletados nos bairros da várias cidades. Os dados serão armazenados e deverão ser processados para gerar os relatórios estatísticos da pesquisa. Construa um fluxograma de como será o processo de coleta, processamento e resultados do sistema.

Histórico da Linguagem Java

Um pouco da história

Java foi criado em 1991 por James Gosling da Sun *Microsystems*. Inicialmente chamada OAK (Carvalho), em homenagem à uma árvore de janela do Gosling, seu nome foi mudado para Java devido a existência de uma linguagem com o nome OAK.

A motivação original do Java era a necessidade de uma linguagem independente de plataforma que podia ser utilizada em vários produtos eletrônicos, tais como torradeiras e refrigeradores. Um dos primeiros

projetos desenvolvidos utilizando *Java* era um controle remoto pessoal chamado *7 (*Star Seven*).

Ao mesmo tempo, a *World Wide Web* e a Internet foram ganhando popularidade. Gosling achava que a linguagem Java poderia ser usada para programação da Internet.

O que é a tecnologia Java?

Uma linguagem de programação

Como linguagem de programação, Java pode ser utilizado para criar todos os tipos de aplicações existentes, de programas de Inteligência Artificial para Robôs até programas para aparelhos celulares.

Um ambiente de desenvolvimento

Como ambiente de desenvolvimento, a tecnologia Java fornece um grande conjunto de ferramentas: um compilador, um interpretador, um gerador de documentação, ferramenta de empacotamento de classes de arquivos e outros.

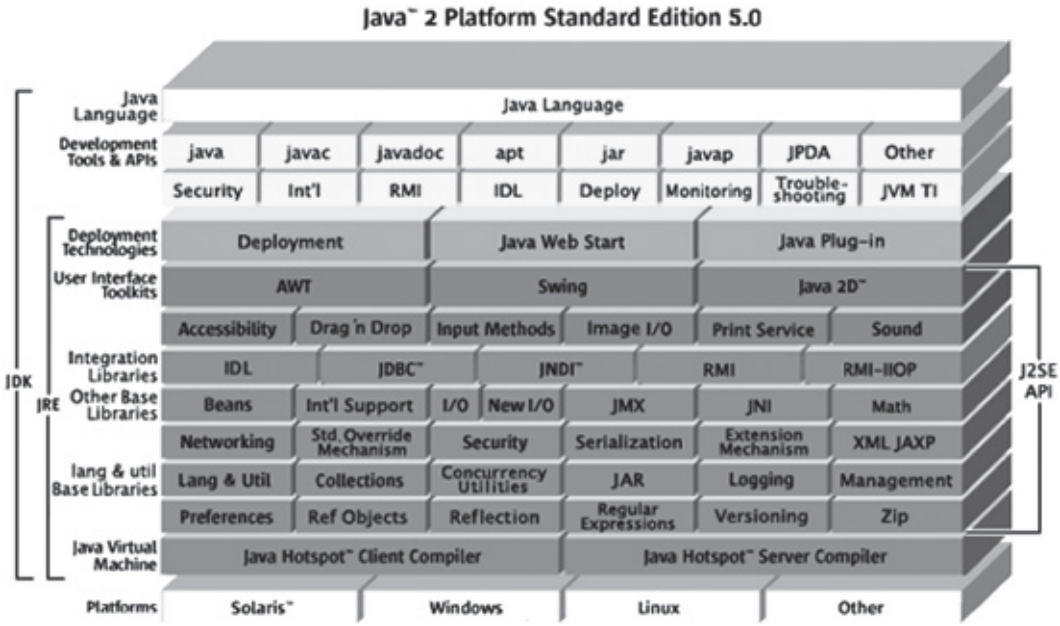
Um ambiente de aplicação

Aplicações de tecnologia Java são tipicamente programas de propósito geral que executam sobre uma máquina onde o *Java Runtime Environment* é instalado.

Um ambiente de distribuição

Há dois ambientes de distribuição principais: primeiro, o JRE, fornecido através do Java 2 *Software Development* Kit (SDK), contém um conjunto completo de arquivos de classes para todos pacotes de tecnologia *Java*.

Outro ambiente de distribuição é o navegador *web*, ou seja, o browser. Os navegadores *web* atuais fornecem interpretação à tecnologia e ambiente Java em tempo de execução.



Algumas características do Java

Máquina Virtual Java

A Máquina Virtual *Java* é uma máquina imaginária eque é implementada através de um *software* emulador em uma máquina real. A JVM provê especificações de plataforma de *hardware* na qual compila-se todo código de tecnologia *Java*. Essas especificações permitem que o *software* Java seja uma plataforma independente, pois a compilação é feita por uma máquina genérica conhecida como JVM.

O bytecode é uma linguagem de máquina especial que pode ser entendida pela Máquina Virtual *Java* (JVM). O bytecode é independente de qualquer hardware de computador particular. Assim, qualquer computador com o interpretador Java pode executar um programa Java compilado, não importando em que tipo de computador o programa foi compilado.

Garbage collection

Muitas linguagens de programação permitem ao programador alocar memória durante o tempo de execução. Entretanto, após utilizar a memória alocada, deve existir uma maneira para desalocar o bloco

de memória de forma que os demais programas a utilizem novamente. Em C, C++ e outras linguagens o programador é o responsável por isso. Isso, às vezes, pode ser difícil já que instâncias podem ser esquecidas de serem desalocadas da memória pelos programadores e resultar no que chamamos de escapes da memória.

Em Java, o programador não possui a obrigação de retirar uma variável criada das áreas de memória, isto é feito por uma parte da JVM específica que chamamos de *Garbage Collection*. O *Garbage Collection* é o grande responsável pela liberação automática do espaço em memória.

Isso acontece automaticamente durante o tempo de vida do programa *Java*.

Segurança do código

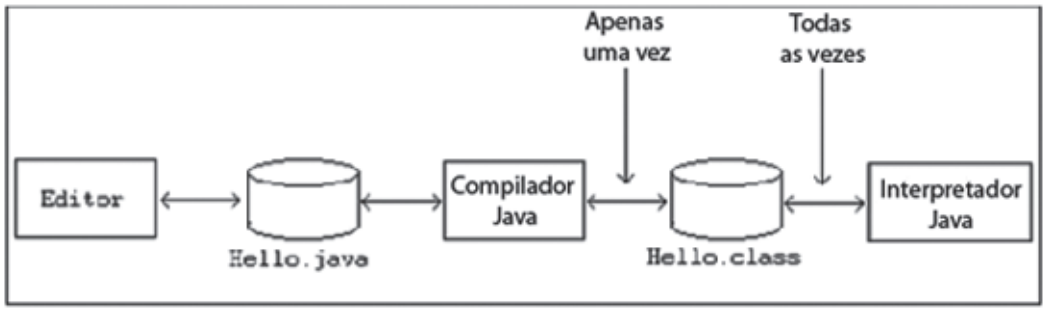
Segurança do Código é alcançada em Java através da implementação da J*ava Runtime Environment* (JRE). A JRE roda códigos compilados para a JVM e executa o carregamento de classes (através do *Class Loader*), verificação de código (através do verificador de *bytecode*) e finalmente o código executável.

O *Class Loader* é responsável por carregar todas as classes necessárias ao programa *Java*. Isso adiciona segurança através da separação do namespace entre as classes do sistema de arquivos local e aquelas que são importadas pela rede. Isso limita qualquer ação de programas que podem causar danos, pois as classes locais são carregadas primeiro. Depois de carregar todas as classes, a quantidade de memória que o executável irá ocupar é determinada. Isto acrescenta, novamente, uma proteção ao acesso não autorizado de áreas restritas ao código, pois a quantidade de memória ocupada é determinada em tempo de execução.

Após carregar as classes e definir a quantidade de memória, o verificador de *bytecode* verifica o formato dos fragmentos de código e pesquisa nesses fragmentos por códigos ilegais que possam violar o direito de acesso aos objetos. Depois que tudo isso tiver sido feito, o código é finalmente executado.

Fases do programa *Java*

A figura seguinte descreve o processo de compilação e execução de um programa *Java*.



O primeiro passo para a criação de um programa Java é escrever os programas em um editor de texto. Exemplos de editores de texto que podem ser utilizados: bloco de notas, vi, emacs, etc. Esses arquivos são armazenados no disco rígido com a extensão .java.

Após o programa *Java* ter sido criado e salvo, compile o programa utilizando o compilador *Java*. A saída desse processo é um arquivo de bytecode com extensão .*class*.

O arquivo .class é então lido pelo Interpretador *Java* que converte os *bytecodes* em linguagem de máquina do computador que se está usando.

Tarefa	Ferramenta utilizada	Saída
Escrever o programa	Qualquer editor de texto	Arquivo com extensão .java
Compilar o programa	Compilador Java	Arquivo com extensão .class (Java bytecode)
Executar o programa	Interpretador Java	Saída do programa

Exercícios Propostos

1. O que a tecnologia Java inovou em relação as tecnologias já existentes como C++, *Delphi, Visual Basic*?
2. Com base a figura abaixo, escreva um fluxograma das fases de implementação de um programa em Java. Especifique os desvios lógicos e desvios com repetições.
3. Com vista no desepenho em tempo de execução, compare a execução de uma aplição em Java e outra em C++ no ambiente Linux.

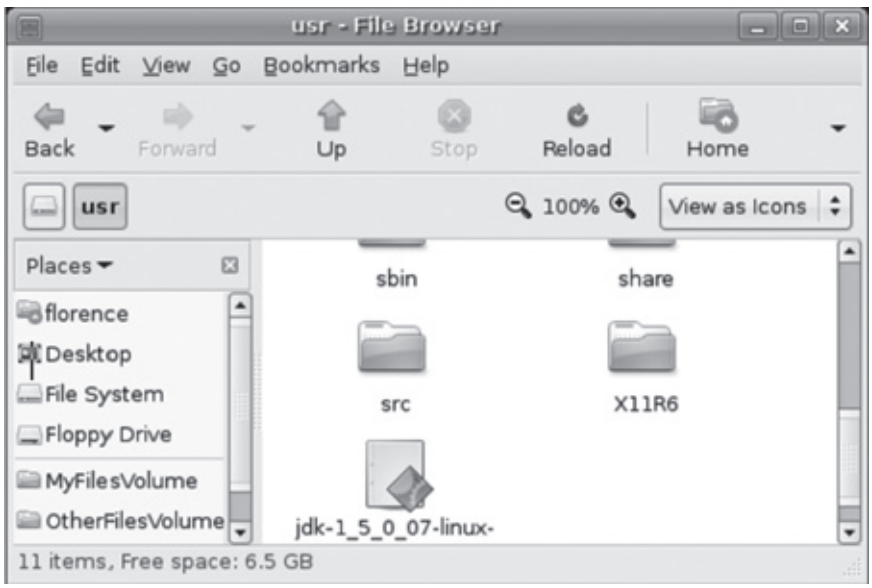
Desafio:

Pesquise e relacione os sistemas middlewares (máquinas virtuais) existentes para linguagens e aplicativos. Descreva a função de cada um.

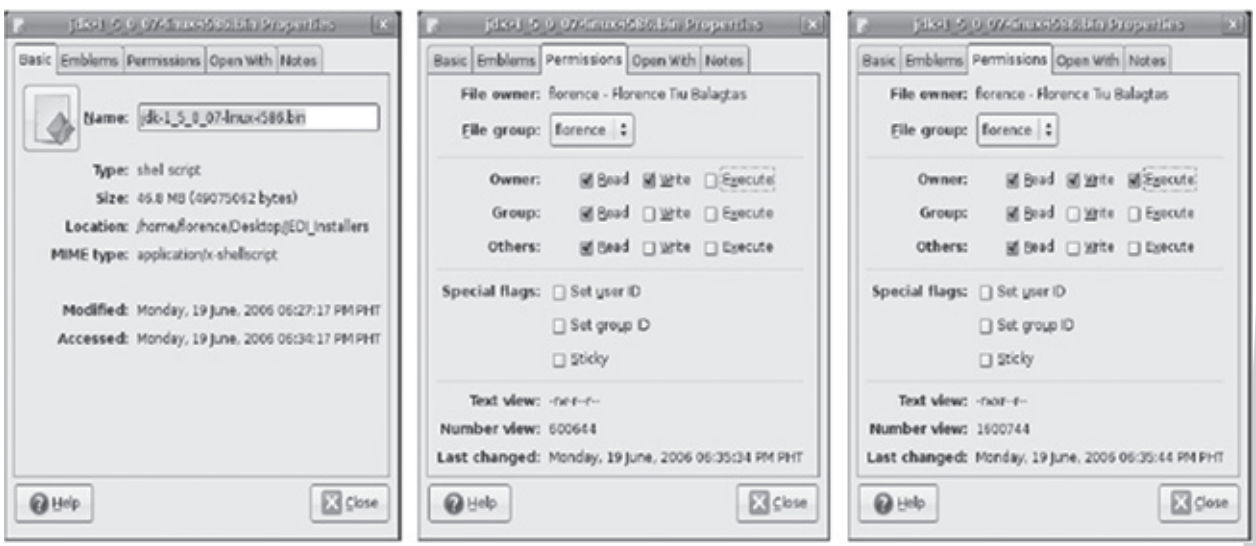
Instalação do Java e do NetBeans

Instalando Java no Ubuntu

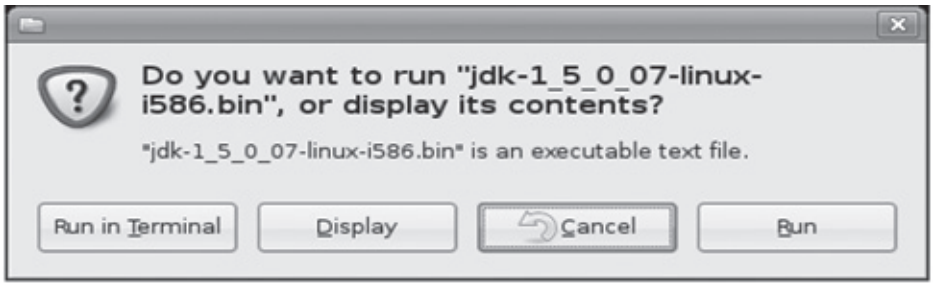
Passo 1: No diretório onde foi efetuado o download dos instaladores.



Passo 2: Antes de executar o instalador, assegure-se de que o arquivo seja um executável. Para tanto, pressione o botão direito do mouse no ícone do instalador, e em seguida selecione Properties. Selecione na aba Permissions, e então marque a opção Execute. Feche a janela.



Passo 3: Duplo-clique no arquivo jdk-1_5_0_07-linux-i586.bin. A caixa de diálogo abaixo será mostrada. Pressione o botão *Run in Terminal*.



No console será mostrado o contrato de licença do software.

Pressione *ENTER* até ser mostrada a pergunta: Do you agree to the above license terms? [yes or no]. Caso concorde com os termos apresentados digite a palavra yes e pressione a tecla *ENTER*. Aguarde que o instalador termine de descompactar e instale o Java.

Passo 4: Devemos um caminho de pesquisa a fim de permitir a execução de comandos java em qualquer local. Para isso, entraremos na pasta /usr/local/bin. Digitando:



Para criar os links simbólicos para os comandos, tecle:

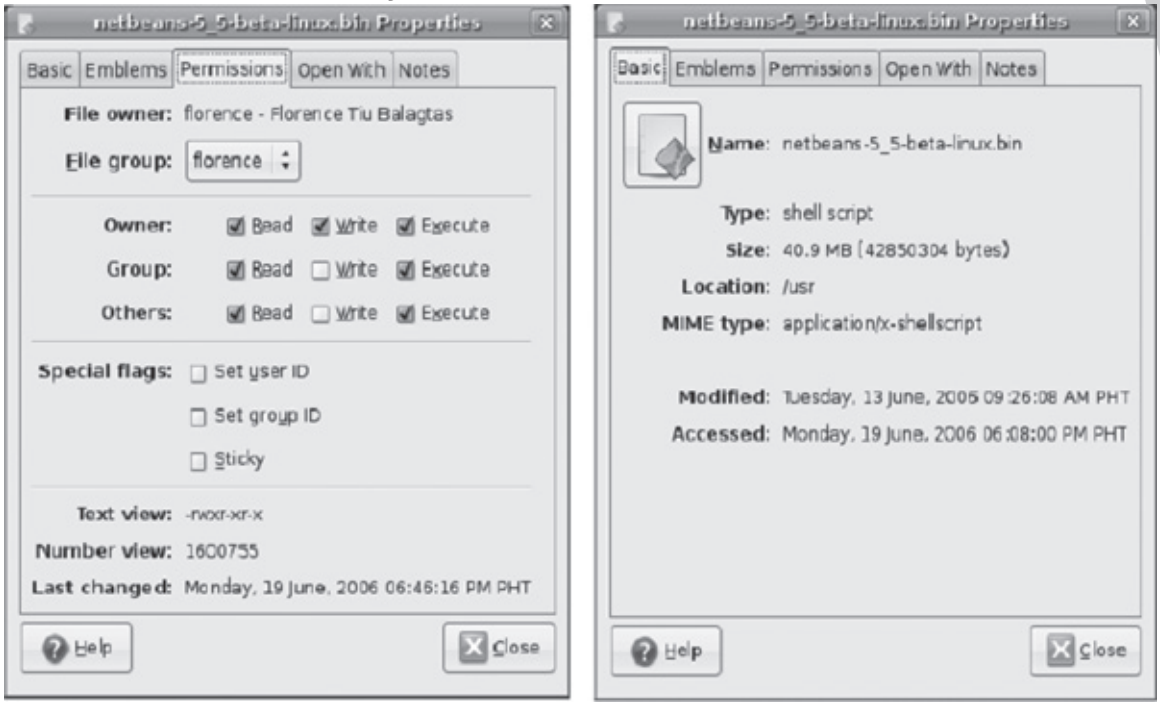
sudo ln -s /usr/java/jdk1.5.0_07/bin/ .*

Instalando NetBeans no Ubuntu

Passo 1: Vá para a pasta onde estiver o instalador do NetBeans.

Passo 2: Antes de executar o instalador, assegure-se de que o arquivo seja executável. Para tanto, utilize o botão direito do *mouse* no ícone do instalador e, em seguida selecione *Properties*. Selecione a aba

Permissions, e marque a opção Execute.



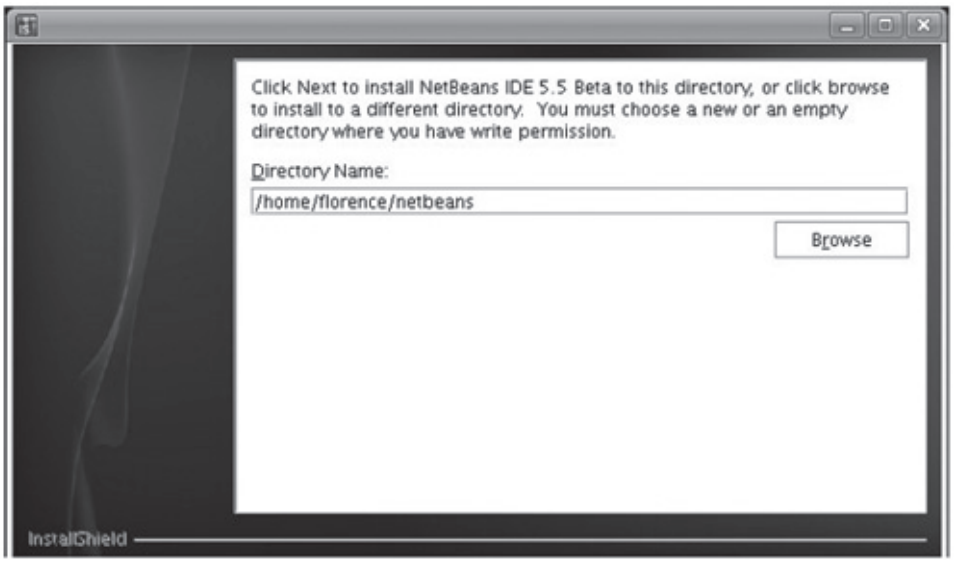
Passo 3: Duplo-clique no arquivo de instalação do NetBeans. Pressione o botão Run in Terminal. Será mostrada uma caixa de diálogo do *NetBeans* 5.5. *Pressione* o botão Next >.



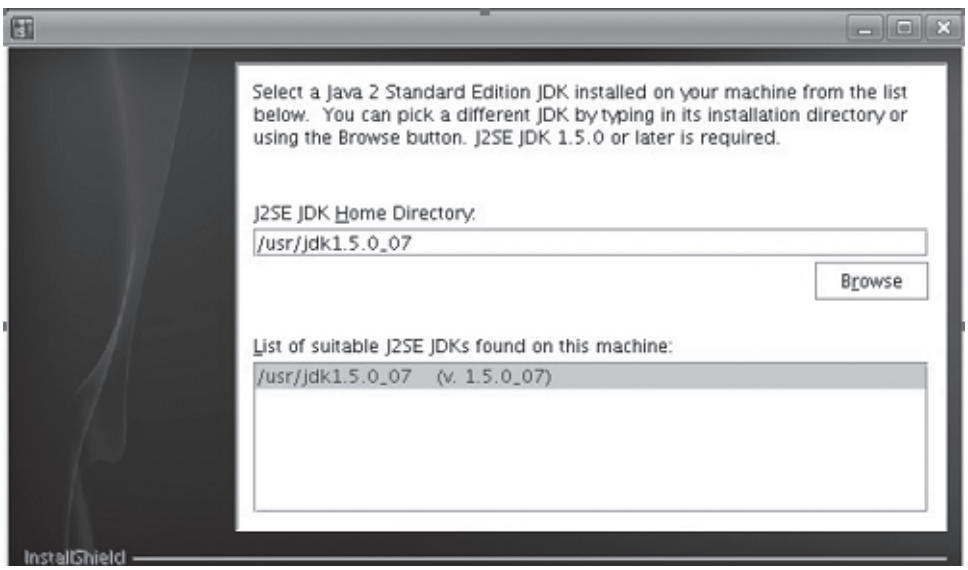
Na próxima janela, os termos da licença serão mostrados; caso

concorde, selecione a opção I accept the terms in the license agreement, e então pressione o botão Next >.

Modifique o nome do diretório para: /usr/java/netbeans-5.5, então pressione o botão Next >.



Na pasta do JDK, selecione /usr/java/jdk1.5.0_07, e então pressione o botão Next >.



A próxima caixa de diálogo mostra apenas informações sobre o *NetBeans* que você está instalando. Pressione o botão Next >. Aguarde o *NetBeans* terminar o processo de instalação.

Pressione o botão Finish para completar a instalação.

Passo 4: A fim de possibilitar a execução do NetBeans a partir de qualquer pasta no computador, precisamos criar um caminho de pesquisa. Para isso, entramos na pasta :/usr/local/bin. com o comando:

cd /usr/local/bin

Crie um caminho de pesquisa para o NetBeans, digitando:

sudo ln -s /usr/java/netbeans-5.5 .

É possível executar o NetBeans a partir de qualquer pasta, digitando:

netbeans &



Exercícios Propostos

1. Baixe da Internet e instale o sistema de desenvolvimento java (J2SDK) no ambiente Windowns. Edite (usando o editor padrão - NotePad), compile e execute (usando linha de comando – CMD) o programa “Wello World”.
2. Baixe da Internet e instale o sistema de desenvolvimento java (J2SDK) no ambiente Linux. Edite (usando o editor padrão – vi ou pico, etc), compile e execute (usando linha de comando do terminal console) o programa “Wello World”.
3. Baixe e instale no NetBeans no ambiente Windonws e Linux. Teste alguns exemplos.

Primeiro Programa Java com a IDE NetBeans

Antes de explicar o que o programa significa, vamos escrevê-lo e executá-lo.

Utilizando a console e um editor de texto

Neste exemplo, utilizaremos um simples editor de texto, que pode ser o gedit do **Linux**, para editar o código fonte. Em seguida, será necessário abrir uma janela terminal para compilar e executar os programas.

Passo 1: executar um editor de texto

Para iniciar um editor de texto no **Linux**, selecione Applications Þ Accessories Þ Text Editor.

Passo 2: Abrir a janela de console

Para abrir o terminal no **Linux**, selecione Applications Þ Accessories Þ Terminal.

Passo 3: Escrever as instruções utilizando o editor de texto

Digite as seguintes instruções no editor de textos:

```
public class Hello
{
    /**
     * Meu primeiro programa Java
     */
    public static void main(String[] args) {
      // Mostra na tela o texto "Hello world"
         System.out.println("Hello world!");
    }
}
```

Passo 4: Salvar o programa Java

Chamaremos o programa de "Hello.java" e o colocaremos em uma pasta denominada "myJavaPrograms".

Caso esta pasta não tenha sido criada, retorne à janela de terminal

aberta e insira as seguintes instruções:

Para o **Linux**:

$ md myJavaPrograms

Retorne ao editor de textos e salve o programa. Para abrir a caixa de diálogo salvar, selecione a opção "File" localizada na barra de menus e depois clique na opção "Save".

Selecione a nova pasta criada como myJavaPrograms para entrar nela. A pasta deve estar vazia porque ainda não salvamos nada dentro dela.

Na caixa de texto "Name", digite o nome do programa (Hello.java), e depois clique no botão salvar.

Após salvar o arquivo, observe que o título da janela mudou de "Untitled" para "Hello.java"; caso deseje alterar novamente o arquivo, basta editá-lo e depois salvá-lo novamente clicando em File Þ Save.

Passo 5: Entrar na pasta que contém o programa

O próximo passo deve ser o de compilar o programa. Inicialmente, precisamos entrar na pasta que o contém. Retorne à janela do terminal.

Em **Linux**:

Normalmente, quando abrimos uma janela terminal, ela vai diretamente para sua pasta home (identificada por $). Para ver o que tem dentro do diretório, digite **ls** (LS em minúscula, significando "List Sources") e pressione ENTER. Isso fará com que sejam listados os arquivos e pastas da pasta home.

Verifique a existência de uma pasta chamada "myJavaPrograms", criada a pouco, sendo esta o local em que foi salvo o programa "Hello. java". Mudaremos o contexto para essa pasta.

Para entrar nessa pasta, devemos utilizar o comando: cd [nome da pasta]. O comando "cd" significa "Change Directory". Digitaremos:

$ cd myJavaPrograms

Agora que estamos dentro da pasta onde o arquivo do programa está, poderemos então compilá-lo.

Certifique-se de que o arquivo está realmente dentro desta,

executando o comando **ls** (LS em minúscula) novamente.

Passo 6: Compilar o programa

Para compilar o programa, utilizamos o comando: **javac [Nome do Arquivo]**. Ou seja:

javac Hello.java

Durante a compilação, é criado o arquivo: **[Nome do Arquivo]. class**, neste caso, **Hello.class**, que contém o código em linguagem de máquina (chamado de **bytecode**).

Passo 7: Executar o programa

Assumindo que não ocorreu problemas na compilação (caso tenha ocorrido qualquer problema refaça os passos realizados), estamos prontos para executar o programa.

Para executar o programa, utilizamos o comando: java [nome do arquivo sem a extensão].

No caso do exemplo, digite:

java Hello

Veremos na mesma tela, em que foi executado o comando, a seguinte mensagem:

Hello world!

Erros

Vimos um pequeno programa Java, geralmente não encontraremos qualquer problema para compilar e executar esses programas, entretanto nem sempre este é o caso, como mencionamos na primeira parte deste curso, ocasionalmente encontramos erros durante esse processo.

Como mencionamos antes, há dois tipos de erros: o primeiro pode ocorrer durante a compilação, chamado de erro de sintaxe, o segundo pode ocorrer durante a execução, chamado runtime error.

Erros de sintaxe

Os erros de sintaxe normalmente são erros de digitação, ocasionados pelo programador que pode ter se equivocado e digitar uma instrução errada, ou por esquecimento de alguma parte da instrução, por exemplo, um ponto e vírgula. O compilador tenta isolar o erro exibindo a linha de instrução e mostrando o primeiro caractere incorreto naquela linha, entretanto, um erro pode não estar exatamente neste ponto.

Outros erros comuns são a troca de letras, troca de letras maiúsculas por minúsculas (a linguagem Java é completamente **case-sensitive**, ou seja, o caractere "a" é completamente diferente do caractere "A", e o uso incorreto da pontuação.

Vamos retornar ao exemplo, o programa **Hello.java.** Intencionalmente, escreveremos a palavrachave "static" de forma errada e omitiremos o ponto-e-vírgula em uma instrução e a deixaremos errada.

```
public class Hello
{
    /**
     * Meu primeiro programa Java
     */
    public statict void main(String[] args) {
      // A linha abaixo foi retirado o ;
         System.out.println("Hello world!")
    }
```

Salve o programa e execute os passos necessários para compilá-lo. Observe a mensagem de erro gerada ao se tentar compilar novamente o programa:

```
Hello.java:6: <identifier> expected
        public statict void main(String[] args)
                        ^
Hello.java:10: ';' expected
        }
        ^
1 error
```

A primeira mensagem de erro sugere que existe um erro na linha 6 do programa apontado para a palavra **void**, entretanto, essa palavra está correta. O erro é na palavra anterior **statict** que deve ser digitada como **static**.

A segunda mensagem de erro sugere que faltou um ponto-e-vírgula na linha 10, entretanto, esta contém simplesmente o comando de

fechar o bloco do método main. O erro está exatamente na linha anterior.

Como regra, ao encontrar muitas mensagens de erros, devemos corrigir o primeiro erro da lista e tentar novamente compilar o programa. Desse modo, reduziremos o número total de mensagens de erro dramaticamente, pois podem existir o que chamamos de erros derivados, ou seja, um erro que tem por causa a instrução anterior.

Erros em tempo de execução (Erros de run-time)

Os erros em tempo de execução são erros que não aparecerão até que tentemos executar o programa. Os programas são compilados com sucesso, mas apresentarão respostas erradas, que podem ter como causa se o programador não obedeceu uma lógica coerente ou no caso em erro de estruturas do programa.

Usando NetBeans

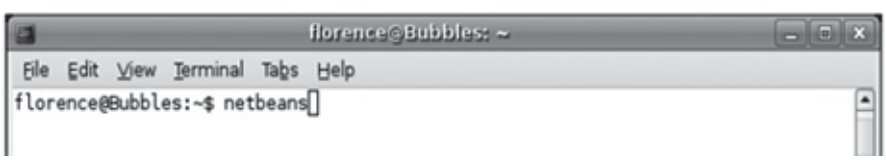
Construímos o programa sem utilizar nenhum recurso sofisticado, iremos aprender como fazer todo o processo da seção anterior utilizando uma IDE.

Nesta parte da lição, utilizaremos o **NetBeans** que é um Ambiente de Desenvolvimento Integrado (IDE - Integrated Development Environment).

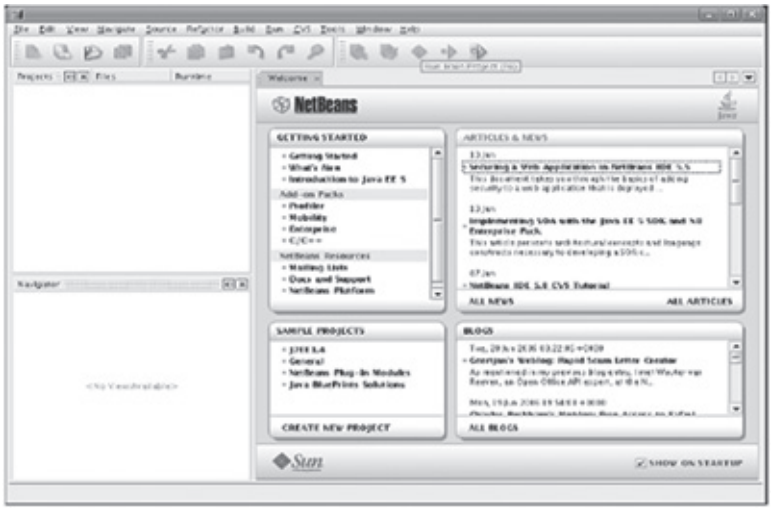
Um ambiente de desenvolvimento integrado é um software aplicativo que possui uma interface construtora, um editor de texto, um editor de código, um compilador e/ou interpretador e um depurador.

Passo 1: Executar o NetBeans

Para executar o NetBeans por intermédio da linha de comando, abra uma janela terminal (Os passos para abrir a janela terminal foram discutidos anteriormente) e digite:

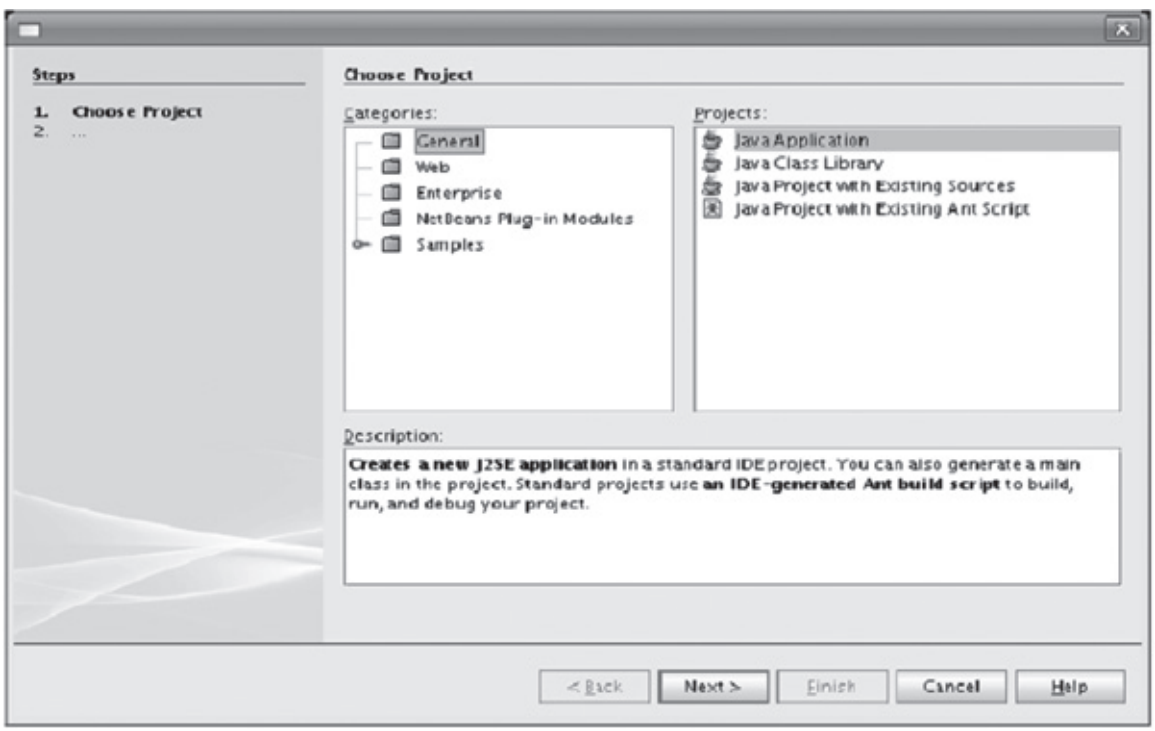


Depois de abrir a IDE NetBeans, será mostrada a interface gráfica GUI, conforme a Figura:

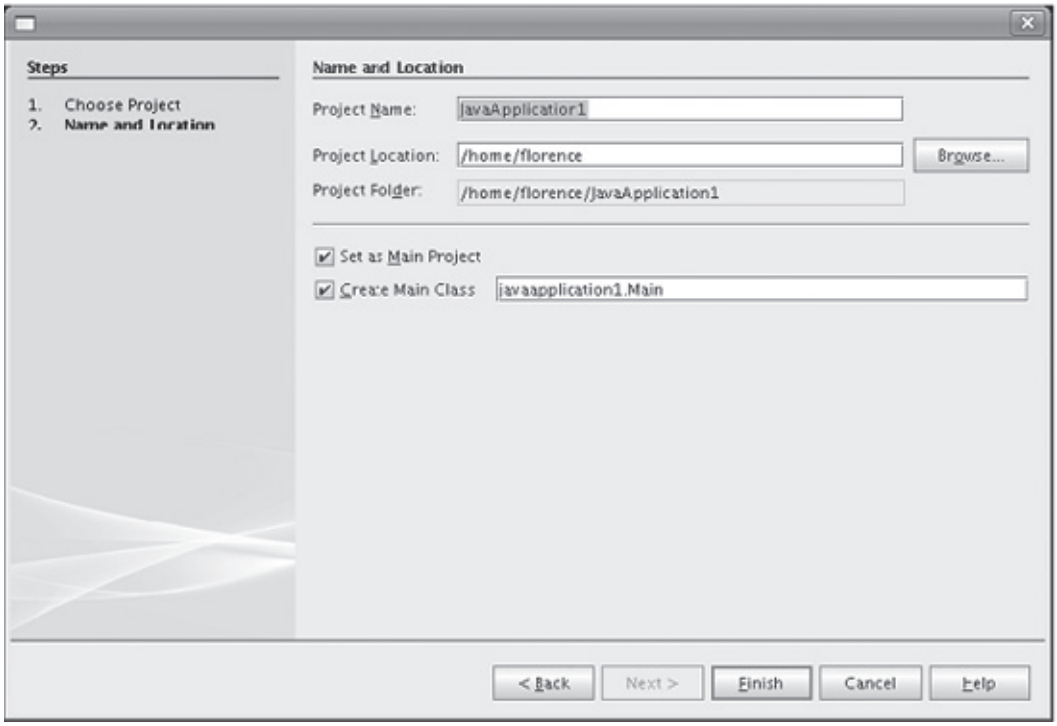


Passo 2: Construir o projeto

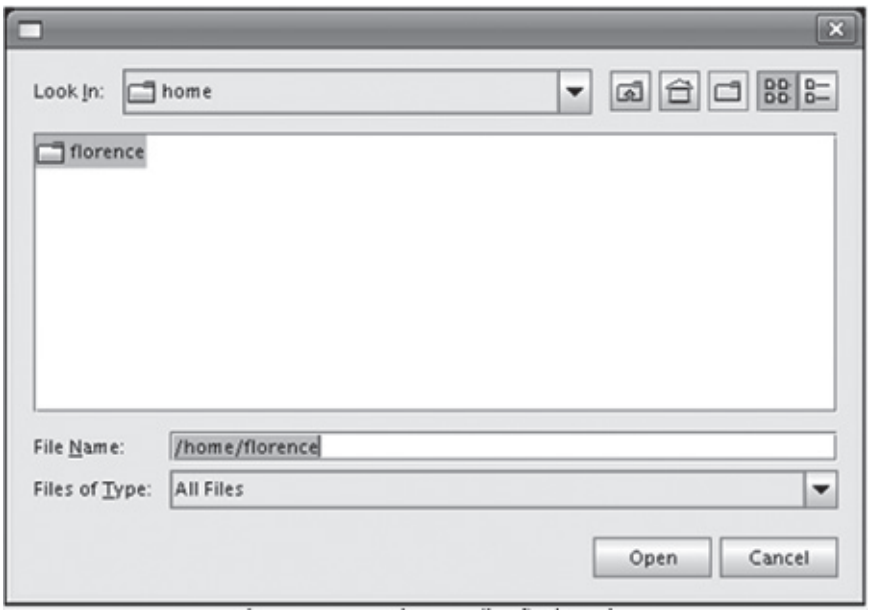
Clique em File Þ New Project, depois de fazer isso, uma janela de diálogo aparecerá. Neste momento, deve-se clicar em "Java Application" e em seguida clicar no botão "Next >".



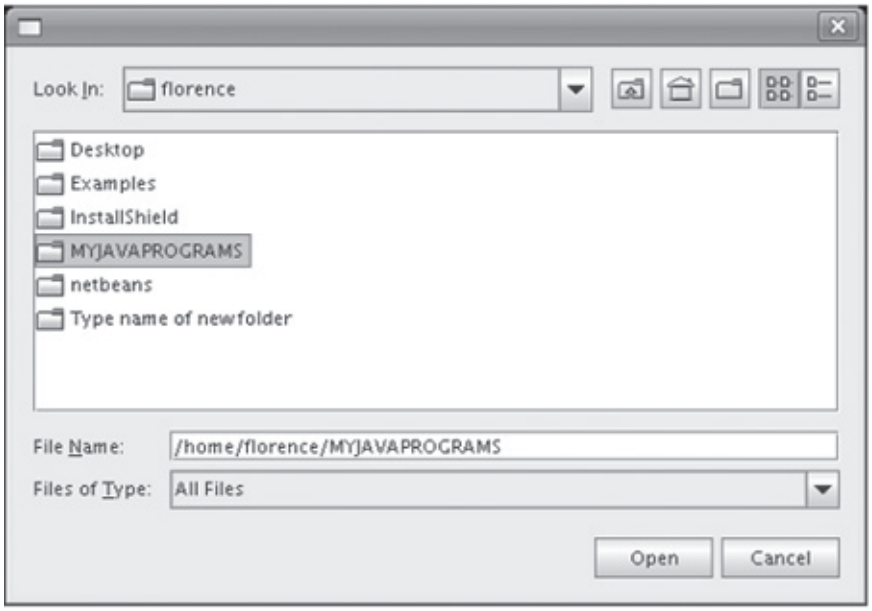
Será mostrada uma nova janela de diálogo, conforme a figura.



Troque o local da aplicação clicando no botão "Browse...". Aparecerá uma janela de diálogo para localização do diretório. Dê um clique duplo no seu diretório home.

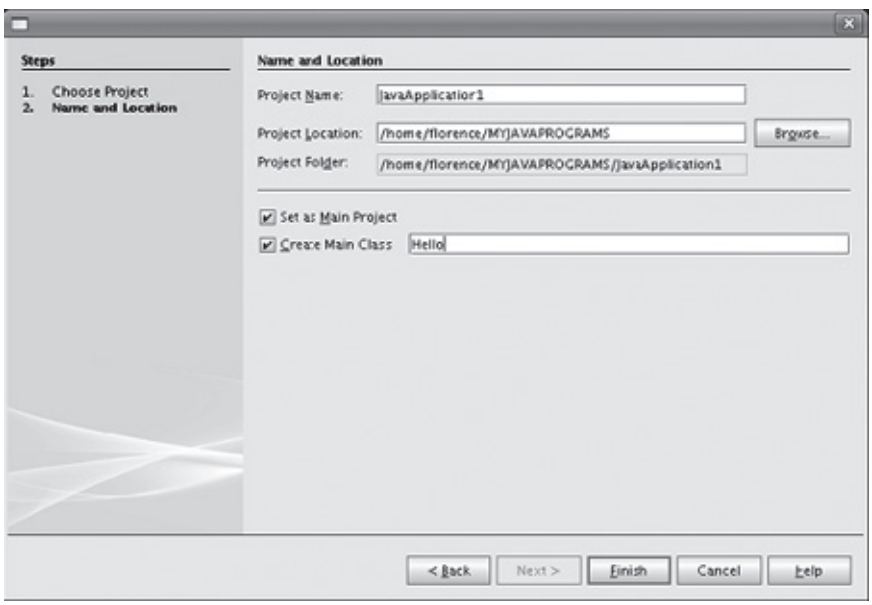


O conteúdo da raiz do diretório será apresentado. Dê um clique duplo no diretório MYJAVAPROGRAMS e depois dê um clique no botão "Open".



Veja que a localização do projeto mudou para /home/florence/MYJAVAPROGRAMS.

Finalmente, no campo "Create Main Class", digite "Hello", que será o nome da classe principal, e em seguida clique no botão "Finish".

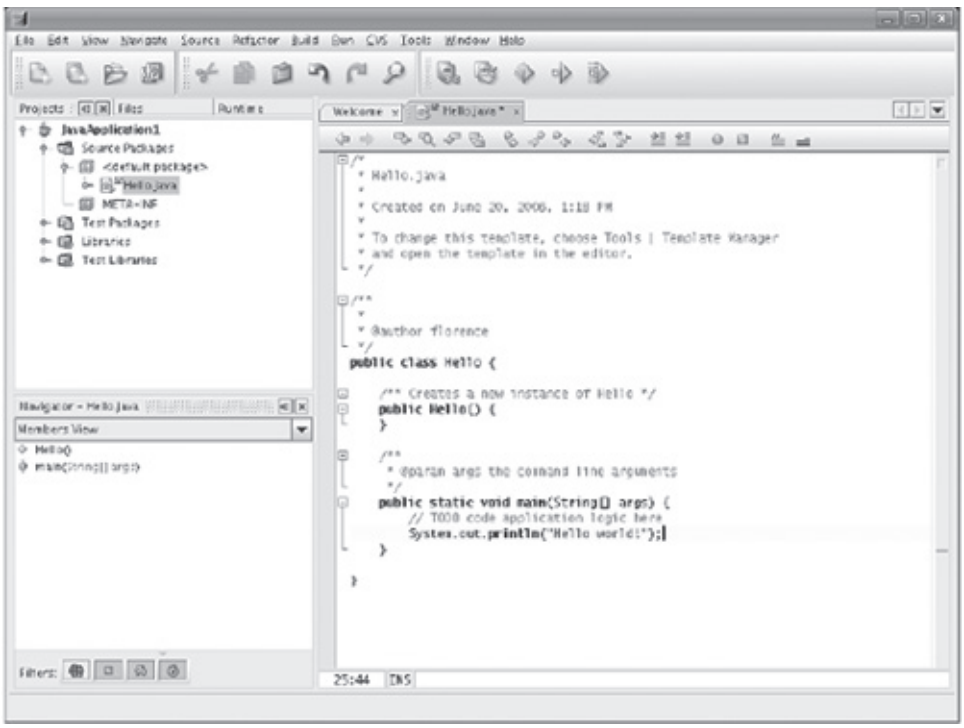


Passo 3: Escrever os detalhes do programa

Antes de escrever o programa, descreveremos a janela principal. Como mostrado na próxima figura, automaticamente o NetBeans

cria um código básico para o programa Java. Poderemos adicionar as declarações nesse código gerado. No lado esquerdo da janela visualizamos uma lista de pastas e arquivos que o NetBeans gerou antes de criar o projeto.

Tudo se encontra dentro da sua pasta MYJAVAPROGRAMS, onde foi configurado o local do projeto. No lado direito, visualizamos o código gerado.



Modifique o código gerado pelo NetBeans, por hora ignoraremos as outras partes das instruções discutindo os detalhes destas posteriormente. Insira a seguinte instrução:

System.out.println("Hello world!");

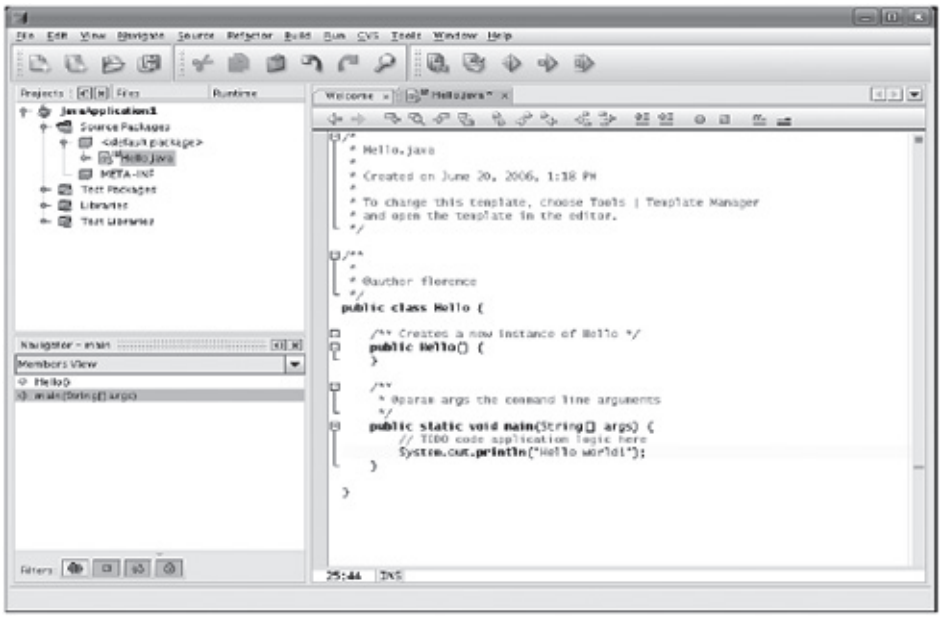
Isso significa que você deseja que seja mostrada a mensagem "Hello world!" na saída padrão do computador, e em seguida seja feito um salto de linha. Poderíamos substituir essa instrução por duas equivalentes:

System.out.print("Hello");
System.out.println(" world!");

O método print() faz com que não seja provocado o salto de linha, e

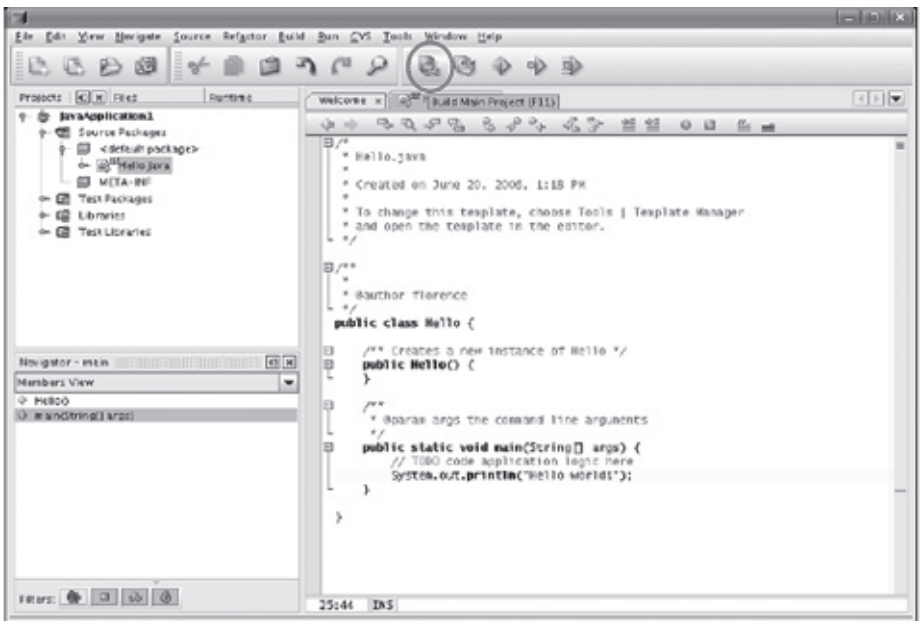
utilizaremos para este exemplo a primeira instrução. Insira esta instrução após a linha de comentário (que será desprezada pelo compilador):

//TODO code application logic here.

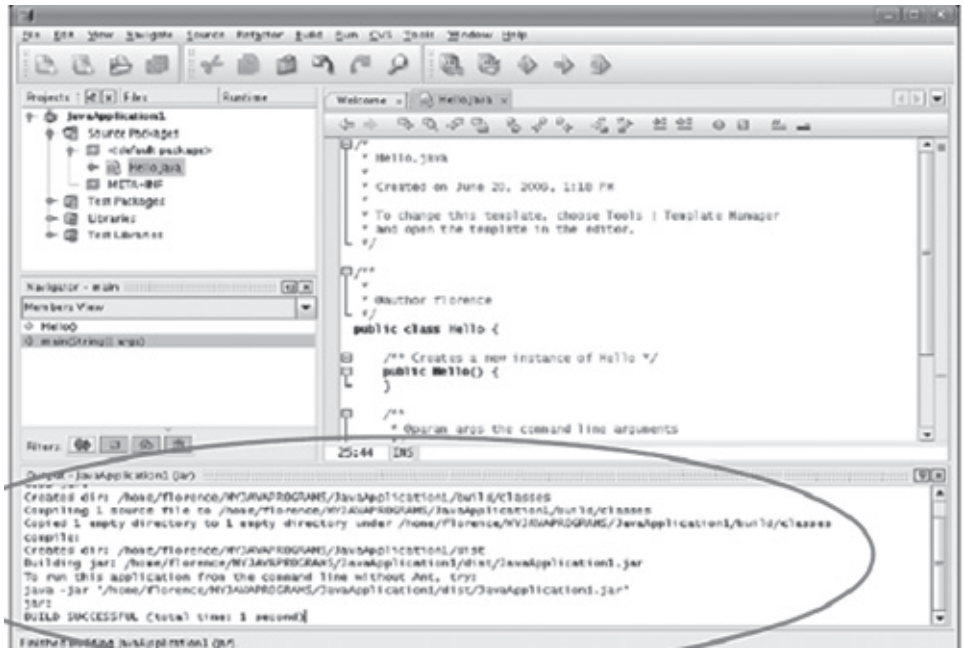


Passo 4: Compilar o projeto

Para compilar o programa, a partir do Menu Principal selecione Build Þ Build Main Project, ou utilize a tecla de atalho F11, ou utilize o botão de atalho para compilar o código.

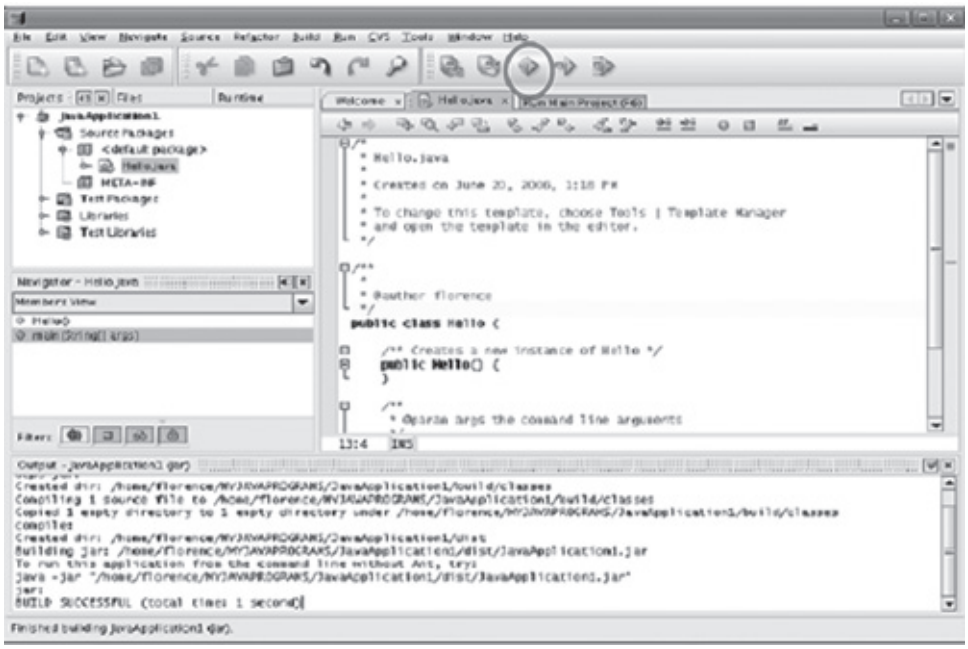


Se não existir erros no programa, veremos a mensagem de sucesso na janela de saída.

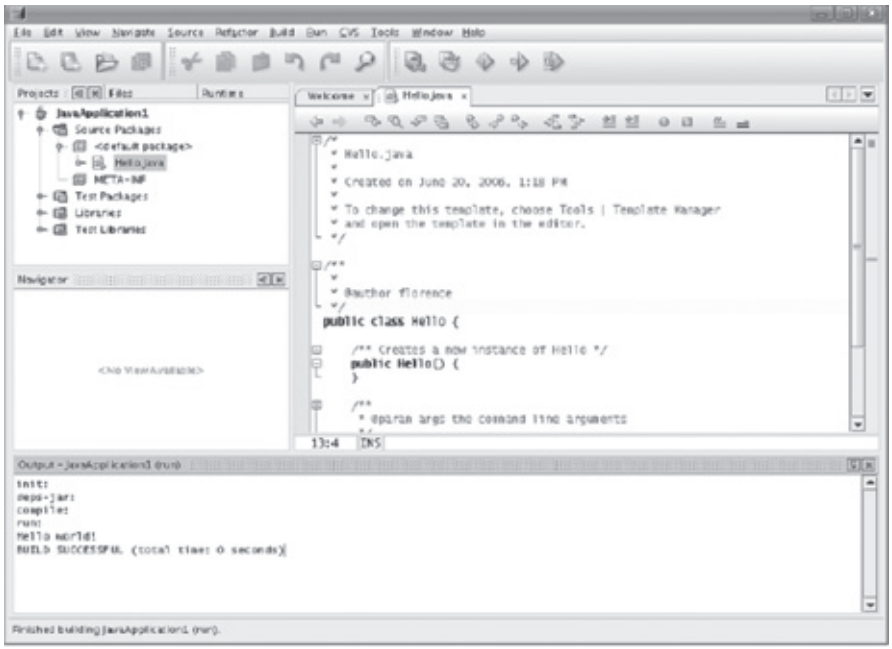


Passo 5: Executar o projeto

Para executar o programa, clique em Run Þ Run Main Project, ou utilize a tecla de atalho F6, ou utilize o botão de atalho para executar o programa.



O resultado final do programa, será mostrado na janela de saída.



Exercícios Propostos

1. Baixe e instale o NetBeans no ambiente Windonws e Linux. Teste alguns exemplos inclusive o “Hello World”
2. Melhore o Hello World. Utilizando o NetBeans, crie uma classe chamada [MeuNome], o programa deverá mostrar como resultado a mensagem: “Welcome to Java Programming [MeuNome]!!!”
3. Utilizando o NetBeans, crie uma classe chamada **TheTree**. O programa deverá mostrar as seguintes linhas na saída:

“Estou aprendendo a usar a IDE NetBeans”
“Minha produtividade vai melhorar”
“O desenvolvimento fica facilitado”

Desafio:

• Baixe da Internet e instale a IDE, bastante conhecida pelos desenvolvedores Java, chamada “Eclipse” http://www.eclipse.org/. Instale no ambiente Win ou Linux. Execute o memo aplicativo “Wello World”. Descreva as dificuldades de manipulação entre o Eclipse e o NetBeans.

BufferedReader para capturar dados

Primeiramente, utilizaremos a classe **BufferedReader** do pacote java.io para capturar dados de entrada através do teclado.

Passos para capturar os dados digitados. Tomemos por base o programa visto na lição anterior:

1. Digite a seguinte instrução no início do programa:

```
import java.io.*;
```

2. Adicione as seguintes instruções no corpo do método **main**:

```
BufferedReader dataIn = new BufferedReader(
new InputStreamReader(System.in));
```

3. Declare uma variável temporária do tipo String para gravar os dados digitados pelo usuário e chame o método readLine() que vai capturar linha por linha o que o usuário digitar. Isso deverá ser escrito dentro de um bloco **try-catch** para tratar possíveis exceções.

```
try {
String temp = dataIn.readLine();
} catch (IOException e) {
System.out.println("Error in getting input");
}
```

Abaixo, segue o programa completo:

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;
public class GetInputFromKeyboard {
public static void main(String[] args) {
BufferedReader dataIn = new BufferedReader(new
InputStreamReader(System.in));
String name = "";
```

```
System.out.print("Please Enter Your Name:");
try {
name = dataIn.readLine();
} catch (IOException e) {
System.out.println("Error!");
}
System.out.println("Hello " + name +"!");
}
}
```

Faremos uma análise deste programa linha por linha:

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.IOException;
```

Estas linhas acima mostram que estamos utilizando as classes **BufferedReader**, **InputStreamReader** e **IOException** cada qual dentro do pacote **java.io.** Essas APIs ou Interfaces de Programação de Aplicações (Application Programming Interface) contêm centenas de classes pré-definidas que se pode usar nos programas. Essas classes são organizadas dentro do que chamamos de **pacotes**.

Pacotes contêm classes que se relacionam com um determinado propósito. No exemplo, o pacote **java.io** contém as classes que permitem capturar dados de entrada e saída. Essas linhas poderiam ser reescritas da seguinte forma:

```
import java.io.*;
```

que importará todas as classes encontradas no pacote **java.io,** deste modo é possível utilizar todas classes desse pacote no programa.

As próximas linhas:

```
public class GetInputFromKeyboard {
public static void main(String[] args) {
```

já foram discutidas na lição anterior. Isso significa que declaramos uma classe nomeada **GetInputFromKeyboard** e, em seguida, iniciamos o método principal (main).

Na instrução:

BufferedReader dataIn = new BufferedReader(new InputStreamReader(System.in));

declaramos a variável **dataIn** do tipo **BufferedReader**. Não se preocupe com o significado da sintaxe, pois será abordado mais à frente.

A seguir, declaramos a variável **name** do tipo String:

String name = "";

na qual armazenaremos a entrada de dados digitada pelo usuário. Note que foi inicializada como uma String vazia "". É uma boa prática de programação inicializar as variáveis quando declaradas.

Na próxima instrução, solicitamos que o usuário escreva um nome:

System.out.print("Please Enter Your Name:");
*As seguinte linhas definem um bloco **try-catch**:*
try {
name = dataIn.readLine();
} catch (IOException e) {
System.out.println("Error!");
}

que asseguram; caso ocorram exceções, serão tratadas.

Falaremos sobre o tratamento de exceções na última parte deste curso. Por hora, é necessário adicionar essas linhas para utilizar o método readLine() e receber a entrada de dados do usuário.

Em seguida:

name = dataIn.readLine();

capturamos a entrada dos dados digitados pelo usuário e as enviamos para a variável String criada anteriormente. A informação é guardada na variável **name**.

Como última instrução:

System.out.println("Hello " + name + "!");

montamos a mensagem final para cumprimentar o usuário.

Praticando!

Digite (retire no número das linhas, coloque a identação e documentação) o programa abaixo e execute. Veja a forma de diálogo. Observer a forma de conversão de dados *parseInt*.

```
1. import javax.swing.*;
2.
3. public class InputTest
4. {
5. public static void main(String[] args)
6. {
7. // get first input
8. String name = JOptionPane.showInputDialog
9. ("What is your name?");
10.
11. // get second input
12. String input = JOptionPane.showInputDialog
13. ("How old are you?");
14.
15. // convert string to integer value
16. int age = Integer.parseInt(input);
17.
18. // display output on console
19. System.out.println("Hello, " + name +
20. ". Next year, you'll be " + (age + 1));
21.
22. System.exit(0);
23. }
24. }
```

Classe Scanner para capturar dados

Vimos uma maneira para obter dados de entrada através do teclado. O JDK 5.0 lançou uma nova classe chamada **Scanner** que engloba diversos métodos para facilitar este serviço.

Veja a seguir o programa completo utilizando essa classe:

```
import java.util.Scanner;
public class GetInputFromScanner
{
```

```
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
System.out.println("Please Enter Your Name:");
String name = sc.next();
System.out.println("Hello " + name +"!");
}
}
```

Compare-o com o programa visto anteriormente. Percebe-se que fica mais simples conseguir a mesma funcionalidade.

Inicialmente, definimos a chamada ao pacote que contém a classe **Scanner**:

```
import java.util.Scanner;
```

Em seguida, as instruções que define a classe e o método main:

```
public class GetInputFromScanner
{
public static void main(String[] args) {
```

Definimos uma variável, denominada sc, que será criada a partir da classe Scanner e direcionada para a entrada padrão:

```
Scanner sc = new Scanner(System.in);
```

De forma semelhante, mostramos uma mensagem solicitando informação do usuário:

```
System.out.println("Please Enter Your Name:");
```

Utilizamos a variável sc para chamarmos o método que fará o recebimento dos dados digitados:

```
String name = sc.nextLine();
```

A classe **Scanner** possui diversos métodos que podem ser utilizados para realizar esse serviço.

Os principais métodos que podemos utilizar, nesse caso, são:

Método	Finalidade
next()	Aguarda uma entrada em formato String
nextInt()	Aguarda uma entrada em formato Inteiro
nextByte()	Aguarda uma entrada em formato Inteiro
nextLong()	Aguarda uma entrada em formato Longo
nextFloat()	Aguarda uma entrada em formato Número Fracionário
nextDoublet()	Aguarda uma entrada em formato Número Fracionário

Por fim, mostramos o resultado e encerramos o método main e a classe:

```
System.out.println("Hello " + name +"!");
{
{
```

Utilizando a JOptionPane para receber dados

Um outro modo de receber os dados de entrada é utilizar a classe **JOptionPane**, que pertence ao pacote **javax.swing**. A **JOptionPane** possui métodos que conseguem criar caixas de diálogo na qual o usuário pode informar ou visualizar algum dado.

Dado o seguinte código:

```
import javax.swing.JOptionPane;
public class GetInputFromKeyboard {
public static void main( String[] args ){
String name = "";
name = JOptionPane.showInputDialog("Please enter your name");
String msg = "Hello " + name + "!";
JOptionPane.showMessageDialog(null, msg);
{
{
```

esta classe apresentará o seguinte resultado:

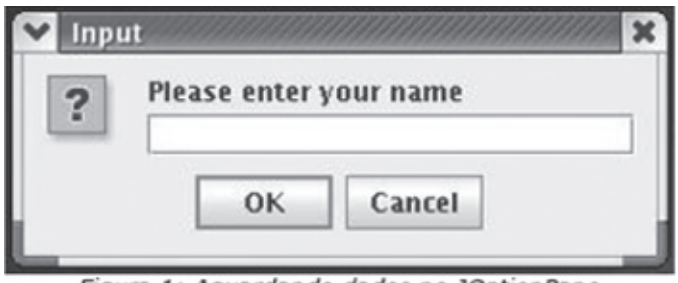
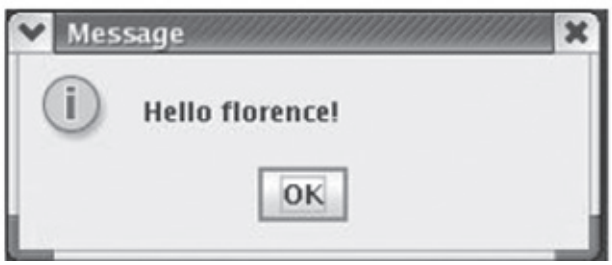


Figura 1: Aguardando dados no JOptionPane



Figura 2: Digitando florence no JOptionPane



A primeira instrução:

import javax.swing.JOptionPane;

mostra que estamos importando a classe JOptionPane do pacote javax.swing.

Poderíamos, de forma semelhante, escrever estas instruções do seguinte modo:

*import javax.swing.**

A instrução seguinte:

name = JOptionPane.showInputDialog("Please enter your name")

cria uma caixa de entrada que exibirá um diálogo com uma mensagem, um campo de texto para receber os dados do usuário e um botão OK, conforme mostrado na acima. O resultado será armazenado

na variável do tipo String name.

Na próxima instrução, criamos uma mensagem de cumprimento, que ficará armazenada na variável **msg**:

String msg = "Hello " + name + "!"

Finalizando a classe, exibiremos uma janela de diálogo que conterá a mensagem e o botão de OK, conforme mostrado na figura acima.

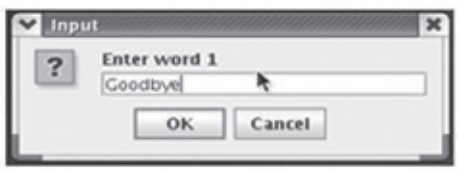
JOptionPane.showMessageDialog(null, msg;

Exercícios Propostos

1. Utilizando a classe **BufferedReader** ou **Scanner**, capture três palavras digitadas pelo usuário e mostre-as como uma única frase na mesma linha. Por exemplo:

Palavra 1: “Meu nome”
Palavra 2: “é”
Palavra 3: “Jesus”
Saída : “Meu nome é Jesus”

2. Utilizando a classe **JOptionPane**, capture palavras em três caixas de diálogos distintas e mostre-as como uma única frase. Por exemplo:









a

linhas do código.

4. Digite (sem nro de linha e identado), compile e execute o programa abaixo. Estude o programa, diga o que faz e coloque os comentários em cada linha.

```
1. import javax.swing.*;
2.
3. public class SimpleFrameTest
4. {
5. public static void main(String[] args)
6. {
7. SimpleFrame frame = new SimpleFrame();
8. frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
9. frame.show();
10. }
11. }
12.
13. class SimpleFrame extends JFrame
14. {
15. public SimpleFrame()
16. {
17. setSize(WIDTH, HEIGHT);
18. }
19.
20. public static final int WIDTH = 300;
21. public static final int HEIGHT = 200;
22. }
```

Estrutura de Controle de Decisão: if-then-else switch

UNIDADE 02

Estruturas de Controle

OBJETIVOS:

- **Desenvolver a capacidade e abstração na especificação ds algoritmos de acordo com os problemas apresentados.**



2 ESTRUTURAS DE CONTROLE

Estruturas de controle de decisão são instruções em linguagem Java que permitem que blocos específicos de código sejam escolhidos para serem executados, redirecionando determinadas partes do fluxo do programa.

Declaração if

Dicas de programação:
1. Expressão lógica é uma declaração que possui um valor lógico. Isso significa que
a execução desta expressão deve resultar em um valor true ou false.
2. Coloque as instruções de forma que elas façam parte do bloco if. Por exemplo:

```
if (expressão_lógica) {
// instrução1;
// instrução2;
}
```

A declaração if especifica que uma instrução ou bloco de instruções seja executado se, e somente se, uma expressão lógica for verdadeira. A declaração if possui a seguinte forma:

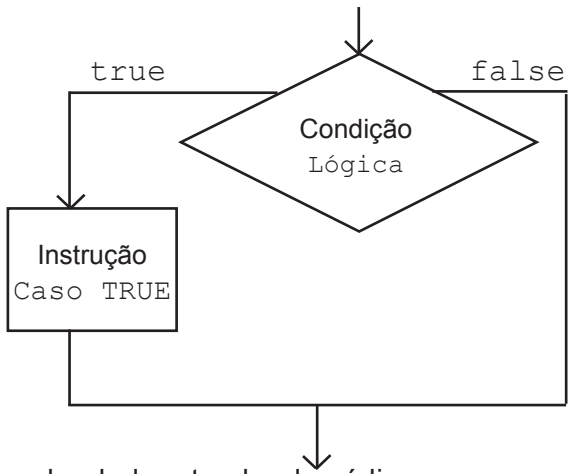
```
if (expressão_lógica)
instrução;
```

ou:

```
if (expressão_lógica) {
```

```
instrução1;
instrução2
...
}
```

onde, **expressão_lógica** representa uma expressão ou variável lógica.



Por exemplo, dado o trecho de código:

Instrução Simples

```
int grade = 68;
if (grade > 60) Stem.out.println  (“Congratulations!");
```

ou:

Instruções em bloco

```
int grade = 68;
if (grade > 60) {
System.out.println("Congratulations!");
System.out.println("You passed!");
}
```

Declaração if-else

A declaração **if-else** é usada quando queremos executar determinado conjunto de instruções se a condição for verdadeira, e outro conjunto se a condição for falsa.

Possui a seguinte forma:

```
if (expressão_lógica)
instrução_caso_verdadeiro;
else
instrução_caso_falso;
```

Também podemos escrevê-la na forma abaixo:

```
if (expressão_lógica) {
instrução_caso_verdadeiro1;
instrução_caso_verdadeiro2;
...
} else {
instrução_caso_falso1;
instrução_caso_falso2;
...
}
```

Dicas de programação:

1. Para evitar confusão, sempre coloque a instrução ou instruções contidas no bloco if ou if-else entre chaves {}.

2. Pode-se ter declarações if-else dentro de declarações if-else, por exemplo:

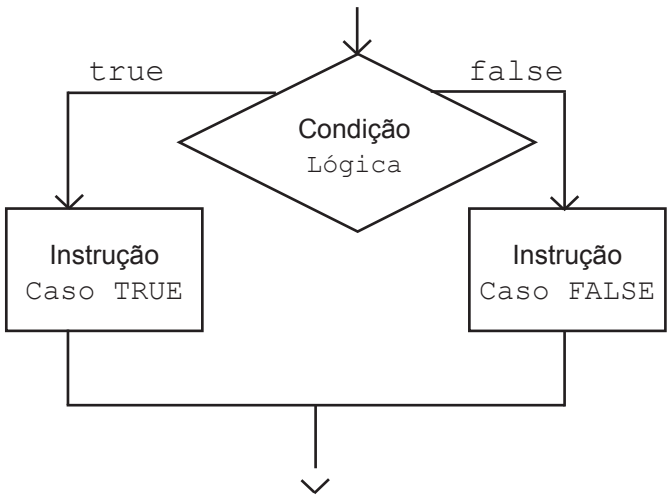
```
if (expressão_lógica) {
if (expressão_lógica) {
...
} else {
...
}
} else {
...
}
```

Por exemplo, dado o trecho de código:

```
int grade = 68;
if (grade > 60)
System.out.println("Congratulations! You passed!");
else
System.out.println("Sorry you failed");
```

ou:

```
int grade = 68;
if (grade > 60) {
System.out.print("Congratulations! ");
System.out.println("You passed!");
} else {
System.out.print("Sorry ");
System.out.println("you failed");
}
```



Declaração if-else-if

A declaração **else** pode conter outra estrutura **if-else.** Esse cascateamento de estruturas permite ter decisões lógicas muito mais complexas.

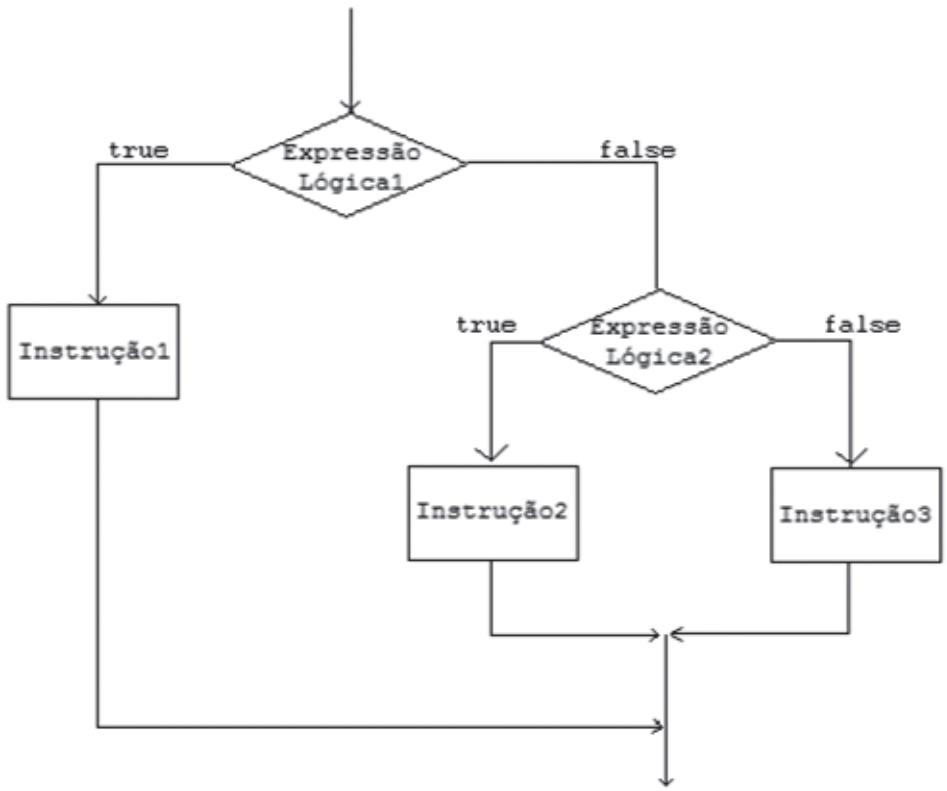
A declaração **if-else-if** possui a seguinte forma:

```
if (expressão_lógica1)
instrução1;
else if(expressão_lógica2)
instrução2;
else
instrução3;
```

Podemos ter várias estruturas **else-if** depois de uma declaração if. A estrutura else é opcional e pode ser omitida. No exemplo mostrado acima, se a **expressão_lógica1** é verdadeira, o programa executa a

instrução1 e salta as outras instruções. Caso contrário, se a expressão_ lógica1 é falsa,o fluxo de controle segue para a análise lógica da **expressão_lógica2**

Se esta for verdadeira, o programa executa a **instrução2** e salta a **instrução3**. Caso contrário, se a **expressão_lógica2** é falsa, então a **instrução3** é executada.



Observe um exemplo da declaração **if-else-if** no seguinte trecho de código:

```
public class Grade {
public static void main( String[] args ) {
double grade = 92.0;
if (grade >= 90) {
System.out.println("Excellent!");
} else if((grade < 90) && (grade >= 80)) {
System.out.println("Good job!");
} else if((grade < 80) && (grade >= 60)) {
System.out.println("Study harder!");
} else {
```

```
System.out.println("Sorry, you failed.");
}
}
}
```

Erros comuns na utilização da declaração if

1. A condição na declaração **if** não avalia um valor lógico. Por exemplo:

```
// ERRADO
int number = 0;
if (number) {
// algumas instruções aqui
}
```

a variável **number** não tem valor lógico.

2. Usar = (sinal de atribuição) em vez de == (sinal de igualdade) para comparação. Por exemplo:

```
// ERRADO
int number = 0;
if (number = 0) {
// algumas instruções aqui
}
```

3. Escrever **elseif** em vez de **else if**.

```
// ERRADO
int number = 0;
if (number == 0) {
// algumas instruções aqui
} elseif (number == 1) {
// algumas instruções aqui
}
```

Declaração switch

Notas:
1. Ao contrário da declaração **if**, múltiplas instruções são executadas sem a necessidade das chaves que determinam o início e término de bloco {}.
2. Quando um case for selecionado, todas as instruções vinculadas ao **case** serão executadas. Além disso, as instruções dos **case** seguintes também serão executadas.
3. Para prevenir que o programa execute instruções dos outros **case** subsequentes, utilizamos a declaração **break** após a última instrução de cada **case**.

Outra maneira de indicar uma condição é através de uma declaração switch. A construção switch permite que uma única variável inteira tenha múltiplas possibilidades de finalização.

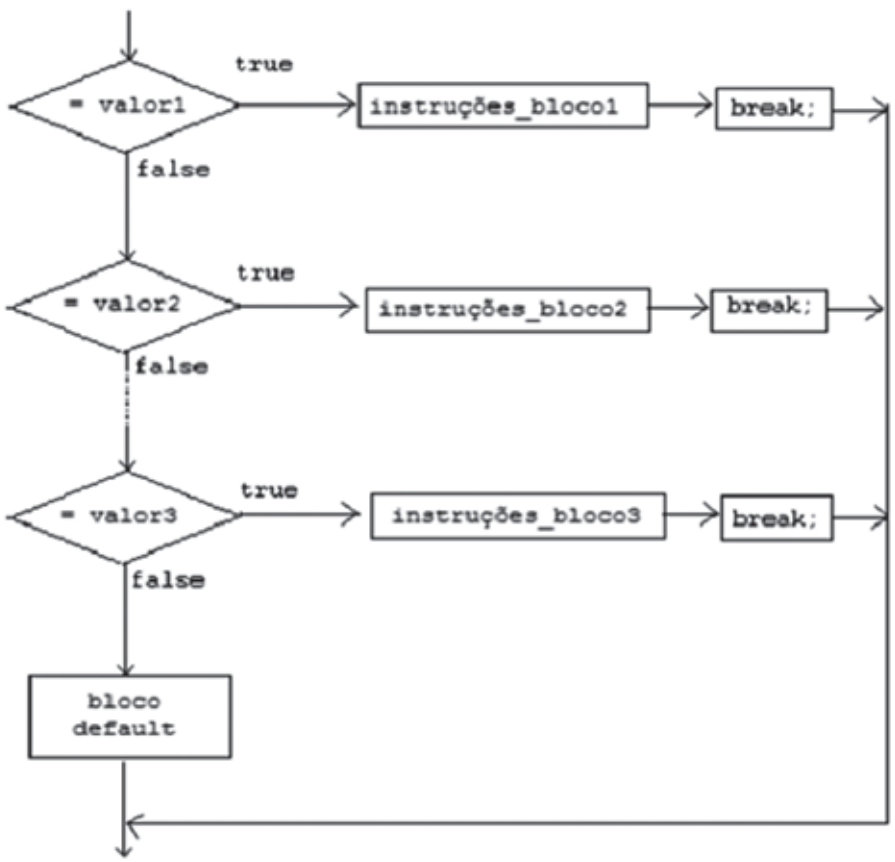
A declaração switch possui a seguinte forma:

```
switch (variável_inteira) {
case valor1:
instrução1;     //
instrução2;     // bloco 1
... //
break;
case valor2:
instrução1;     //
instrução2;     // bloco 2
... //
break;
default:
instrução1      ; //
instrução2;     // bloco n
... //
break;
}
```

Dicas de Programação:
*1. A decisão entre usar uma declaração **if** ou **switch** é subjetiva. O programador pode decidir com base na facilidade de entendimento do código, entre outros fatores.*
*2. Uma declaração **if** pode ser usada para decisões relacionadas a conjuntos, escalas de variáveis ou condições, enquanto que a declaração **switch** pode ser utilizada para situações que envolvam variável do tipo inteiro. Também é necessário que o valor de cada cláusula case seja único.subsequentes; utilizamos a declaração **break** após a última instrução de cada **case** onde, **variável_inteira** é uma variável de tipo byte, short, char ou int; valor1, valor2, e assim por diante, são valores constantes que esta variável pode assumir.*

Quando a declaração switch é encontrada, o fluxo de controle avalia inicialmente a **variável_inteira** e segue para o **case** que possui o valor igual ao da variável. O programa executa todas instruções a partir deste ponto, mesmo as do próximo **case**, até encontrar uma instrução **break**, que interromperá a execução do **switch**.

Se nenhum dos valores **case** for satisfeito, o bloco **default** será executado. Esse é um bloco opcional. O bloco **default** não é obrigatório na declaração **switch**.



Exemplo para switch

```
public class Grade {
public static void main(String[] args) {
        int grade = 92;
switch(grade) {
case 100:
System.out.println("Excellent!");
break;
case 90:
System.out.println("Good job!");
break;
case 80:
System.out.println("Study harder!");
break;
default:
System.out.println("Sorry, you failed.");
}
```

Compile e execute o programa acima e veremos que o resultado será: *Sorry, you failed.*

pois a variável grade possui o valor 92 e nenhuma das opções case atende a essa condição.

Note que, para o caso de intervalos, a declaração if-else-if é mais

Exercícios Propostos



1. Obtenha, do usuário, três notas de exame e calcule a média dessas notas. Reproduza a média dos três exames. Junto com a média, mostre também um :-) no resultado se a média for maior ou igual a 60; caso contrário mostre :-(
Faça duas versões deste programa:

a) Use a classe BufferedReader (ou a classe Scanner) para obter as notas do usuário, e System.out para mostrar o resultado.

b) Use JOptionPane para obter as notas do usuário e para mostrar o resultado.

2. Solicite ao usuário para digitar um número, e mostre-o por extenso. Esse número deverá variar entre 1 e 10. Se o usuário introduzir um número que não está nesse intervalo, mostre: "número inválido".

Faça duas versões deste programa:

a) Use uma declaração if-else-if para resolver este problema

b) Use uma declaração switch para resolver este problema

3. Escreva um programa para calcular o valor da fatura do consumo de energia elétrica. O programa tem como entrada o consumo em Kilo Watt mês. O valor da conta é calculado baseado na faixa de consumo progressivo:

a) de 1a 50 Kw, computa-se R$ 0,50 por Kw consumido

b) de 51a 100 Kw, computa-se R$ 1,00

c) de 101 a 150 Kw, computa-se R$ 2,00

d) acimda de 150 Kw, computa-se R$ 3,00

Por exemplo: Um consumo de 157 Kw/mês gera uma fatura de
(50 x 0,50) + (50 x 1,00) + (50 x 2,00) + (7 x 3,00) = R$ 196,00

Desafio

• O que faz o program abaixo? O que imprime como saída?

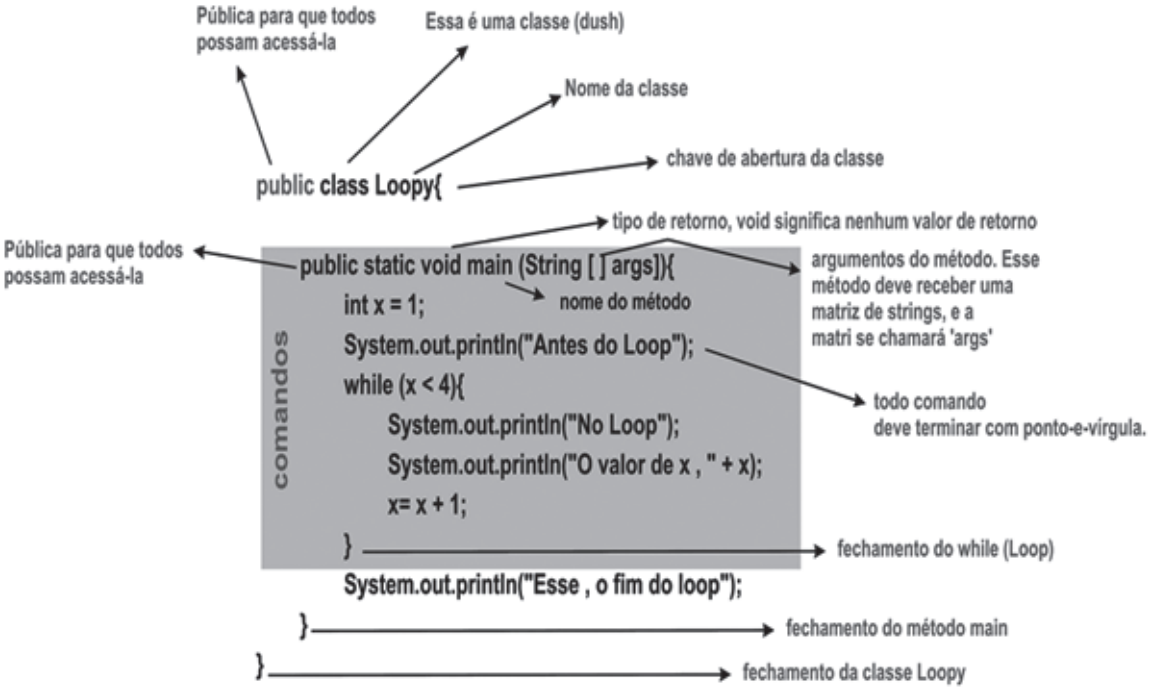
```
public class Mystery {
  public static void main( String args[] )  {
     int y, x = 1, total = 0;
     while ( x <= 10 ) {
        y = x * x;
        System.out.println( y );
        total += y;
        ++x;
      }
     System.out.println( "Total is " + total );
  }
}
```

Estrutura de Controle de Repetição: while, do-while, for

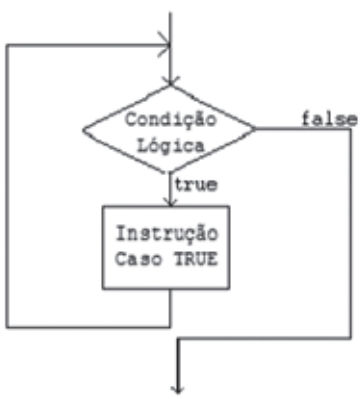
Estruturas de controle de repetição

Estruturas de controle de repetição são comandos em linguagem Java que permitem executar partes específicas do código determinada quantidade de vezes. Existem 3 tipos de estruturas de controle de repetição: **while**, **do-while** e **for**.

A figura abaixo apresenta a anatomia de um program em Java com os elementos mais relevantes da linguagem. Em geral, a maioria da



Declaração while



A declaração while executa repetidas vezes um bloco de instruções enquanto uma determinada condição lógica for verdadeira.

A declaração while possui a seguinte forma:

while (expressão_lógica) {

instrução1;

instrução2;

```
        ...
}
```

As instruções contidas dentro do bloco **while** são executadas repetidas vezes enquanto o valor de **expressão_lógica** for verdadeira.

Por exemplo, dado o trecho de código:

```
int i = 4;
while (i > 0){
        System.out.print(i);
        i--;
}
```

O código acima irá imprimir 4321 na tela. Se a linha contendo a instrução **i--** for removida, teremos uma repetição infinita, ou seja, um código que não termina. Portanto, ao usar laços **while**, ou qualquer outra estrutura de controle de repetição, tenha a certeza de utilizar uma estrutura de repetição que encerre em algum momento.

Praticando!

Digite os exemplos abaixo e observe como funciona a instrução while em diversar formas. Lembre-se da anatomia de um código em Java.

Exemplo 1

```
int x = 0;
while (x<10) {
        System.out.println(x);
        x++;
}
```

Exemplo 2

```
// laço infinito
while (true)
        System.out.println("hello");
```

Exemplo 3

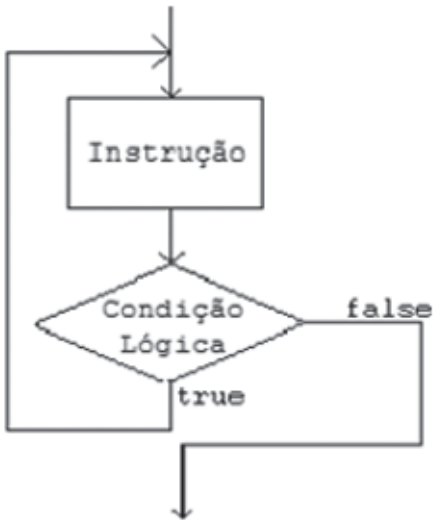
```
// a instrução do laço não será executada
while (false)
        System.out.println("hello");
```

Declaração do-while

A declaração **do-while** é similar ao **while**. As instruções dentro do laço **do-while** serão executadas pelo menos uma vez.

A declaração **do-while** possui a seguinte forma:

```
do {
        instrução1;
        instrução2;
        ...
} while (expressão_lógica);
```



Dicas de programação:

1. Erro comum de programação ao utilizar o laço do-while é esquecer o ponto-e vírgula (;) após a declaração while.

```
do {
...
} while (boolean_expression) // ERRADO -> faltou ;
```

2. Como visto para a declaração while, tenha certeza que a declaração do-while

poderá terminar em algum momento

Inicialmente, as instruções dentro do laço do-while são executadas. Então, a condição na expressão_lógica é avaliada. Se for verdadeira, as instruções dentro do laço do-while serão executadas novamente.

A diferença entre uma declaração while e do-while é que, no laço while, a avaliação da expressão lógica é feita antes de se executarem as instruções nele contidas enquanto que, no laço do-while, primeiro se

executam as instruções e depois realiza-se a avaliação da expressão lógica, ou seja, as instruções dentro em um laço do-while são executadas pelo menos uma vez.

Praticando!

Experimente executar os trechos de código abaixo. Analise o que ocorre.

Exemplo 1

```
int x = 0;
do {
        System.out.println(x);
        x++;
} while (x<10);
```

Este exemplo terá 0123456789 escrito na tela.

Exemplo 2

```
// laço infinito
do {
        System.out.println("hello");
} while(true);
```

Exemplo 3

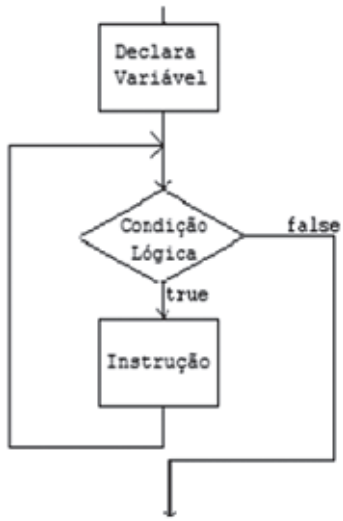
```
// Um laço executado uma vez
do
        System.out.println(“hello”);
while (false);
```

Este exemplo mostrará a palavra hello escrita na tela infinitas vezes.

Declaração for

A declaração for, como nas declarações anteriores, permite a execução do mesmo código uma quantidade determinada de vezes.

A declaração for possui a seguinte forma:




```
for (declaração_inicial; expressão_lógica; salto) {
        instrução1;
        instrução2;
        ...
}
```

onde:

declaração_inicial – inicializa uma variável para o laço

expressão_lógica – compara a variável do laço com um valor limite

salto – atualiza a variável do laço

Um exemplo para a declaração for é:

```
for (int i = 0; i < 10; i++) {
        System.out.print(i);
}
```

Neste exemplo, uma variável **i**, do tipo **int**, é inicializada com o valor zero. A expressão lógica "**i** é menor que 10" é avaliada. Se for verdadeira, então a instrução dentro do laço é executada. Após isso, a expressão **i** terá seu valor adicionado em 1 e, novamente, a condição lógica será avaliada. Este processo continuará até que a condição lógica tenha o valor falso.

Este mesmo exemplo, utilizando a declaração **while**, é mostrado abaixo:

```
int i = 0;
while (i < 10) {
        System.out.print(i);
        i++;
}
```

Praticando!

1. Modifique o programa abaixo com a estrutura “for” para incrementar e decrementar valores de “c”.

```
// Increment.java
public class Increment {
   // main method begins execution of Java application
   public static void main( String args[] )
   int c;
   c = 5;
   System.out.println( c ); // print 5
   System.out.println( c++ ); // print 5 then postincrement
   System.out.println( c ); // print 6
   System.out.println(); // skip a line
   c = 5;
  System.out.println( c ); // print 5
  System.out.println( ++c ); // preincrement then print 6
  System.out.println( c ); // print 6
  } // end method main
} // end class Increment
```

2. Digite o programa abaixo e estude com ele funciona. Remova nos nro de linha e faça a identação do código

```
1 // Average2.java
2 // Class average program with sentinel-controlled repetition.
3
4
// Java core packages
5 import java.text.DecimalFormat;
67
// Java extension packages
8 import javax.swing.JOptionPane;
9
10 public class Average2 {
11
12 // main method begins execution of Java application
13 public static void main( String args[] )
14 {
15 int gradeCounter, // number of grades entered
16 gradeValue, // grade value
17 total; // sum of grades
18 double average; // average of all grades
19 String input; // grade typed by user
20
21 // Initialization phase
```

```
22 total = 0; // clear total
23 gradeCounter = 0; // prepare to loop
24
25 // Processing phase
26 // prompt for input and read grade from user
27 input = JOptionPane.showInputDialog(
28 "Enter Integer Grade, -1 to Quit:" );
29
30 // convert grade from a String to an integer
31 gradeValue = Integer.parseInt( input );
32
33 while ( gradeValue != -1 ) {
34
35 // add gradeValue to total
36 total = total + gradeValue;
37
38 // add 1 to gradeCounter
39 gradeCounter = gradeCounter + 1;
40
41 // prompt for input and read grade from user
42 input = JOptionPane.showInputDialog(
43 "Enter Integer Grade, -1 to Quit:" );
44
45 // convert grade from a String to an integer
46 gradeValue = Integer.parseInt( input );
47 }
48
49 // Termination phase
50 DecimalFormat twoDigits = new DecimalFormat( "0.00" );
51
52 if ( gradeCounter != 0 ) {
53 average = (double) total / gradeCounter;
54
55 // display average of exam grades
56 JOptionPane.showMessageDialog( null,
57 "Class average is " + twoDigits.format( average ),
58 "Class Average", JOptionPane.INFORMATION_MESSAGE );
59 }
60 else
61 JOptionPane.showMessageDialog( null,
62 "No grades were entered", "Class Average",
63 JOptionPane.INFORMATION_MESSAGE );
64
65 System.exit( 0 ); // terminate application
66
67 } // end method main
68
69 } // end class Average2
```

Exercícios Propostos



1. Crie um programa que mostre seu nome cem vezes. Faça três versões deste programa:

a) Use uma declaração while para resolver este problema

b) Use uma declaração do-while para resolver este problema

c) Use uma declaração for para resolver este problema

2. Receba como entrada um número e um expoente. Calcule esse número elevado ao expoente. Faça três versões desse programa:

a) Use uma declaração while

b) Use uma declaração do-while

c) Use uma declaração for

3. O que o programa abaixo faz:

Desafio

Encontre e faça a devida correção dos erros em cada um dos fragmentos de código abaixo.

```
// Cod A
x = 1;
while ( x <= 10 );
x++;
}
```

```
// Cod B
switch ( n ) {
case 1:
System.out.println("The number is 1");
case 2:
System.out.println("The number is 2");
break;
default:
System.out.println("The number is not 1 or 2");
break;
}
```

```
// Cod C imprime valores de
1 a 10
n = 1;
while ( n < 10 )
System.out.println( n++ );
```

```
// Cod D
 for ( y = .1; y != 1.0; y += .1
)
System.out.println( y );
```

Estrutura de Controle de Interrupção: break, continue e return

Declarações de interrupção

Declarações de interrupção permitem que redirecionemos o fluxo de controle do programa. A linguagem Java possui três declarações de interrupção. São elas: **break**, **continue** e **return**.

Declaração break

A declaração **break** possui duas formas: **unlabeled** (não identificada - vimos esta forma com a declaração **switch**) e **labeled** (identificada).

Declaração unlabeled break

A forma **unlabeled** de uma declaração **break** encerra a execução de um **switch** e o fluxo de controle é transferido imediatamente para o final deste. Podemos também utilizar a forma para terminar declarações **for**, **while** ou **do-while.**

Por exemplo:

```
String names[] = {"Beah", "Bianca", "Lance", "Belle",
"Nico", "Yza", "Gem", "Ethan"};
String searchName = "Yza";
boolean foundName = false;
for (int i=0; i < names.length; i++) {
if (names[i].equals(searchName)) {
        foundName = true;
        break;
}
}
if (foundName) {
System.out.println(searchName + " found!");
} else {
System.out.println(searchName + " not found.");
}
```

Neste exemplo, se a String “Yza” for encontrada, a declaração **for** será interrompida e o controle do programa será transferido para a próxima instrução abaixo da declaração **for**.

Declaração labeled break

A forma **labeled** de uma declaração **break** encerra o processamento de um laço que é identificado por um **label** especificado na declaração **break**.

Um label, em linguagem Java, é definido colocando-se um nome seguido de dois-pontos, como por exemplo:

teste: esta linha indica que temos um **label** com o nome teste.

O programa a seguir realiza uma pesquisa de um determinado valor em um array bidimensional. Dois laços são criados para percorrer este array. Quando o valor é encontrado, um **labeled break** termina a execução do laço interno e retorna o controle para o laço mais externo.

```
int[][] numbers = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
int searchNum = 5;
searchLabel: for (int i=0; i<numbers.length; i++) {
        for (int j=0; j<numbers[i].length; j++) {
                if (searchNum == numbers[i][j]) {
                        foundNum = true;
                        break searchLabel;
                }
        } // final do laço j
} // final do laço i
if (foundNum) {
        System.out.println(searchNum + " found!");
} else {
        System.out.println(searchNum + " not found!");
}
```

A declaração **break**, ao terminar a declaração **for**, não transfere o controle do programa ao final de seu laço, controlado pela variável **j**. O controle do programa segue imediatamentepara a declaração **for** marcada com o **label**, neste caso, interrompendo o laço controlado pela variável **i**.

Declaração continue

A declaração **continue** tem duas formas: **unlabeled** e **labeled**. Utilizamos uma declaração continue para saltar a repetição atual de declarações **for**, **while** ou **do-while**.

Declaração unlabeled continue

A forma **unlabeled** salta as instruções restantes de um laço e avalia novamente a expressão lógica que o controla.

O exemplo seguinte conta a quantidade de vezes que a expressão "Beah" aparece no array.

```
String names[] = {"Beah", "Bianca", "Lance", "Beah"};
int count = 0;
for (int i=0; i < names.length; i++) {
        if (!names[i].equals("Beah")) {
                continue; // retorna para a próxima condição
        }
        count++;
}
System.out.println(count + " Beahs in the list");
```

Declaração labeled continue

A forma labeled da declaração continue interrompe a repetição atual de um laço e salta para a repetição exterior, marcada com o label indicado.

```
outerLoop: for (int i=0; i<5; i++) {
for (int j=0; j<5; j++) {
        System.out.println("Inside for(j) loop"); // mensagem1
        if (j == 2)
                continue outerLoop;
}
System.out.println("Inside for(i) loop"); // mensagem2
}
```

Neste exemplo, a mensagem 2 nunca será mostrada, pois a declaração continue outerloop interromperá este laço cada vez que j

atingir o valor 2 do laço interno.

Declaração return

A declaração **return** é utilizada para sair de um método. O fluxo de controle retorna para a declaração que segue a chamada do método original. A declaração de retorno possui dois modos: o que retorna um valor e o que não retorna nada.

Para retornar um valor, escreva o valor (ou uma expressão que calcula este valor) depois da palavra chave **return**. Por exemplo:

return ++count;
ou
return "Hello";

Os dados são processados e o valor é devolvido de acordo com o tipo de dado do método. Quando um método não tem valor de retorno, deve ser declarado como **void**. Use a forma de **return** que não devolve um valor. Por exemplo:

return;

Abordaremos as declarações **return** nas próximas lições, quando falarmos sobre métodos.

Praticando!

Digite o programa abaixo. Veja que brincadeira legal com o uso do *continue*.

```
1  // Fig. 5.14: ContinueLabelTest.java
2  // Using the continue statement with a label
3
4  // Java extension packages
5  import javax.swing.JOptionPane;
6
7  public class ContinueLabelTest {
8
9     // main method begins execution of Java application
10    public static void main( String args[] )
11    {
12       String output = "";
13
14       nextRow:   // target label of continue statement
15
16          // count 5 rows
17          for ( int row = 1; row <= 5; row++ ) {
18             output += "\n";
19
20             // count 10 columns per row
21             for ( int column = 1; column <= 10; column++ ) {
22
23                // if column greater than row, start next row
24                if ( column > row )
25                   continue nextRow; // next iteration of
26                                     // labeled loop
27
28                output += "*  ";
29
30             }  // end inner for structure
31
32          }  // end outer for structure
33
34       JOptionPane.showMessageDialog(
35          null, output,"Testing continue with a label",
36          JOptionPane.INFORMATION_MESSAGE );
37
38       System.exit( 0 );  // terminate application
39
40    }  // end method main
41
42 }  // end class ContinueLabelTest
```

Exercícios Propostos



1. Execute o programa abaixo. Se faz o laço de 1 a 10 vezes, por que imprime somente até 5? Faça o teste passo a passo no algoritmo.

```
// BreakTeste.java
// Laço 10 vezes
import javax.swing.JOptionPane;
public class BreakTest {
public static void main( String args[] )  {
   String output = "";
   int count;
     for ( count = 1; count <= 10; count++ ) {
           if ( count == 5 )
           break;
           output += count + " ";
      }
     output += "\nLaço = " + count;
 JOptionPane.showMessageDialog( null, output );
 System.exit( 0 );
 }
}
```

2. Execute o programa abaixo. Se faz o laço até de 1 a 9 vezes, o que será impresso? Faça o teste passo a passo no algoritmo.

```
// ContinueTest.java
import javax.swing.JOptionPane;
public class ContinueTest {
  public static void main( String args[] ) {
     String output = "";
     for ( int count = 1; count <= 10; count++ ) {
     if ( count == 5 )
         continue;
     output += count + " ";
     }
     output += "\nUsed continue to skip printing 5";
    JOptionPane.showMessageDialog( null, output );
    System.exit( 0 );
  }
}
```



Resumindo

Esta unidade apresentou as estruturas de controles básicas como desvios condicionais simples (IF/then) e compostos (IF/tem/else); e repetições condicionais com teste no início (while/for) e no final (do/while) das estruturas da programação Java. Essas estruturas são importantes na especificação de algoritmos, pois de acordo com as entradas, o fluxo de execução do algoritmo pode ser alterado.

UNIDADE 03

Arranjos e Argumentos

OBJETIVOS:

- **Conhecer as estruturas de dados básicas constantes nos programas e saber empregál-as no momento da especificação do modo que o código fique mais claro e otimizável.**

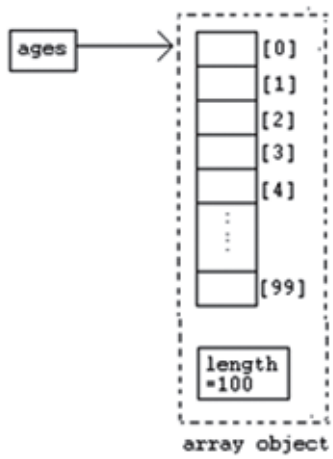
3 ARRANJOS E ARGUMENTOS

Array em Java

Introdução a array

Em lições anteriores, discutimos como declarar diferentes variáveis usando os tipos de dados primitivos. Na declaração de variáveis, freqüentemente utilizamos um identificador ou um nome e um tipo de dados. Para se utilizar uma variável, deve-se chamá-la pelo nome que a identifica.

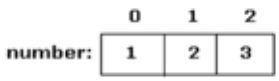
Por exemplo, temos três variáveis do tipo int com diferentes identificadores para cada variável:




```
int number1;
int number2;
int number3;

number1 = 1;
number2 = 2;
number3 = 3;
```

Como se vê, inicializar e utilizar variáveis pode tornar-se uma tarefa tediosa, especialmente se elas forem utilizadas para o mesmo objetivo. Em Java, e em outras linguagens de programação, pode-se utilizar uma variável para armazenar e manipular uma lista de dados com maior eficiência. Esse tipo de variável é chamado de array.



Um array armazena múltiplos itens de um mesmo tipo de dado em um bloco contínuo de memória, dividindo-o em certa quantidade de posições. Imagine um array como uma variável esticada – que tem um nome que a identifica e que pode conter mais de um valor para esta mesma variável.

Declarando array

Array precisa ser declarado como qualquer variável. Ao declarar um array, defina o tipo de dados deste, seguido por colchetes [] e pelo nome que o identifica.

Por exemplo:

```
int [] ages;
```

ou colocando os colchetes depois do identificador. Por exemplo:

```
int ages[];
```

Depois da declaração, precisamos criar o array e especificar seu tamanho. Esse processo é chamado de construção (a palavra, em orientação a objetos, para a criação de objetos). Para se construir um objeto, precisamos utilizar um construtor. Por exemplo:

```
// declaração
int ages[];
// construindo
ages = new int[100];
```

ou, pode ser escrito como:

```
// declarar e construir
int ages[] = new int[100];
```

No exemplo, a declaração diz ao compilador Java que o identificador ages será usado como um nome de um array contendo inteiros, usado para criar, ou construir, um novo array contendo 100 elementos.

Em vez de utilizar uma nova linha de instrução para construir um

array, também é possível automaticamente declarar, construir e adicionar um valor uma única vez.

Exemplos:

```
// criando um array de valores lógicos em uma variável
// results. Este array contém 4 elementos que são
// inicializados com os valores {true, false, true, false}
boolean results[] ={ true, false, true, false };
// criando um array de 4 variáveis double inicializados
// com os valores {100, 90, 80, 75};
double []grades = {100, 90, 80, 75};
// criando um array de Strings com identificador days e
// também já inicializado. Este array contém 7 elementos
String days[] = {"Mon","Tue","Wed","Thu","Fri","Sat","Sun"};
```

Uma vez que tenha sido inicializado, o tamanho de um array não pode ser modificado, pois é armazenado em um bloco contínuo de memória.

Acessando um elemento do Array

Para acessar um elemento do array, ou parte de um array, utiliza-se um número inteiro chamado de índice. Um índice é atribuído para cada membro de um array, permitindo ao programa e ao programador acessar os valores individualmente quando necessário. Os números dos índices são sempre inteiros. Eles começam com zero e progridem sequencialmente por todas as posições até o fim do array. Lembre-se que os elementos dentro do array possuem índice de 0 a tamanhoDoArray-1.

Dicas de programação:

1. Normalmente, é melhor inicializar, ou instanciar, um array logo após declará-lo.

Por exemplo, a instrução:

```
int []arr = new int[100];
```

é preferível, ao invés de:

```
int [] arr;
arr = new int[100];
```

2. Os elementos de um array de n elementos têm índices de 0 a n-1. Note que não existe o elemento arr[n]. A tentativa de acesso a esse elemento causará uma exceção do tipo ArrayIndexOutOfBoundsException, pois o índice deve ser até n-1.

3. Não é possível modificar o tamanho de um array.

Por exemplo, dado o array ages que declaramos anteriormente, temos:

```
// atribuir 10 ao primeiro elemento do array
ages[0] = 10;

// imprimir o último elemento do array
System.out.print(ages[99]);
```

Lembre-se que o array, uma vez declarado e construído, terá o valor de cada membro inicializado automaticamente, conforme a seguinte tabela:

Tipo Primitivo	**Iniciado com**
boolean	false
byte, short e int	0
char	'\u0000'
long	0L
float	0.0F
double	0.0

Entretanto, tipos de dados por referência, como as Strings, não serão inicializados caracteres em branco ou com uma string vazia "", serão inicializados com o valor **null**. Deste modo, o ideal é preencher os elementos do arrays de forma explícita antes de utilizá-los. A manipulação de objetos nulos pode causar a desagradável surpresa de uma exceção do tipo **NullPointerException**, por exemplo, ao tentar executar algum método da classe String, conforme o exemplo a seguir:

Dicas de programação:
1. Quando criar laços com for para o processamento de um array, utilize o campo
length como argumento da expressão lógica. Isso irá permitir ao laço ajustar-se, automaticamente para tamanhos de diferentes arrays.
2. Declare o tamanho dos arrays, utilizando variáveis do tipo constante para facilitar
alterações posteriores. Por exemplo:

```
final int ARRAY_SIZE = 1000; // declarando uma constante
...
int[] ages = new int[ARRAY_SIZE];
```

```
public class ArraySample {
        public static void main(String[] args){
                String [] nulls = new String[2];
                System.out.print(nulls[0]);
                        // Linha correta, mostra null
                System.out.print(nulls[1].trim());
                        // Causa erro
        }
}
```

O código abaixo utiliza uma declaração **for** para mostrar todos os elementos de um array.

```
public class ArraySample {
        public static void main(String[] args){
                int[] ages = new int[100];
                for (int i = 0; i < 100; i++) {
                        System.out.print(ages[i]);
                }
        }
}
```

Tamanho de array

Para se obter o número de elementos de um array, pode-se utilizar o atributo **length**. O atributo **length** de um array retorna seu tamanho, ou seja, a quantidade de elementos. É utilizado como no código abaixo:

nomeArray.length

Por exemplo, dado o código anterior, podemos reescrevê-lo como:

```
public class ArraySample {
        public static void main (String[] args) {
                int[] ages = new int[100];
                for (int i = 0; i < ages.length; i++) {
                        System.out.print(ages[i]);
                }
        }
}
```

Arrays multidimensionais

Arrays multidimensionais são implementados como arrays dentro de arrays. São declarados ao atribuir um novo conjunto de colchetes depois do nome do array. Por exemplo:

```
// array inteiro de 512 x 128 elementos
int [][] twoD = new int[512][128];

// array de caracteres de 8 x 16 x 24
char [][][] threeD = new char[8][16][24];

// array de String de 4 linhas x 2 colunas
String [][] dogs = {{"terry", "brown"},
        {"Kristin", "white"},
        {"toby", "gray"},
        {"fido", "black"}};
```

Acessar um elemento em um array multidimensional é semelhante a acessar elementos em um array de uma dimensão. Por exemplo, para acessar o primeiro elemento da primeira linha do array **dogs**, escreve-se:

```
System.out.print(dogs[0][0]);
```

Isso mostrará a String "terry" na saída padrão. Caso queira mostrar todos os elementos deste array, escreve-se:

```
for (int i = 0; i < dogs.length; i++) {
        for (int j = 0; j < dogs[i].length; j++) {
                System.out.print(dogs[i][j] + " ");
        }
}
```



Exercícios Propostos

1. Criar um array de Strings inicializado com os nomes dos sete dias da semana. Por exemplo:

String days[] = {"Monday", "Tuesday", "Wednesday", "Thursday",

"Friday", "Saturday", "Sunday"};

Usando uma declaração while, imprima todo o conteúdo do array. Faça o mesmo para as declarações do-while e for.

2. Usando as classes BufferedReader, Scanner ou JOptionPane, solicite 10 números ao usuário. Utilize um array para armazenar o valor desses números. Mostre o número de maior valor.

3. Dado o seguinte array multidimensional, que contém as entradas da agenda telefônica:

String entry = {{"Florence", "735-1234", "Manila"},
{"Joyce", "983-3333", "Quezon City"},
{"Becca", "456-3322", "Manila"}};

mostre-as, conforme o formato abaixo:

Name : Florence
Tel. # : 735-1234
Address: Manila

Name : Joyce
Tel. # : 983-3333
Address: Quezon City

Name : Becca
Tel. # : 456-3322
Address: Manila

Desafio:

1. O programa abaixo ler os argumentos na linha de comando e coloca no array de args. Altere esse programa para mostrar as opções disponíveis, caso o usuário entre com argumentos inválidos.

```
public class Comand {
   public static void main(String[] args) {
      if (args[0].equals("-h"))
         System.out.print("Help,");
      else if (args[0].equals("-g"))
               System.out.print("Go to do,");
      for (int i = 1; i < args.length; i++)
          System.out.print(" " + args[i]);
      System.out.println("!");
  }
}
```

2. Escreva um programa para ler um conjuto de elementos, armazenar em arrays, classificar em ordem crescente e imprimir os elementos do array.

Argumentos em Java

Argumentos de linha de comando

Uma aplicação em Java aceita qualquer quantidade de argumentos passados pela linha de comando. Argumentos de linha de comando permitem ao usuário modificar a operação de uma aplicação a partir de sua execução. O usuário insere os argumentos na linha de comando no momento da execução da aplicação. Deve-se lembrar que os argumentos de linha de comando são especificados depois do nome da classe a ser executada.

Por exemplo, suponha a existência de uma aplicação Java, chamada **Sort**, que ordena cinco números que serão recebidos. Essa aplicação seria executada da seguinte maneira:

java Sort 5 4 3 2 1

Lembre-se que os argumentos são separados por espaços.

Em linguagem Java, quando uma aplicação é executada, o sistema repassa os argumentos da linha de comando para a o método main da aplicação através de um array de String. Cada elemento deste array conterá um dos argumentos de linha de comando passados. Lembre-se da declaração do método **main**:

Dicas de programação:
1. Antes de usar os argumentos de linha de comando, observe a quantidade de argumentos passados para a aplicação. Deste modo, nenhuma exceção será disparada.

```
public static void main(String[] args) {
}
```

Os argumento que são passados para o programa são salvos em um array de String com o identificador args. No exemplo anterior, os argumentos de linha de comando passados para a aplicação Sort estarão em um array que conterá cinco strings: "5", "4", "3", "2" e "1". É possível conhecer o número de argumentos passados pela linha de comando,

utilizando-se o atributo length do array.

Por exemplo:

```
int numberOfArgs = args.length;
```

Se o programa precisa manipular argumento de linha de comando numérico, então, deve-se converter o argumento do tipo String, que representa um número, assim como "34", para um número. Aqui está a parte do código que converte um argumento de linha de comando para inteiro:

```
int firstArg = 0;
if (args.length > 0) {
firstArg = Integer.parseInt(args[0]);
}
```

parseInt dispara uma exceção do tipo **NumberFormatException** se o conteúdo do elemento arg[0] não for um número.

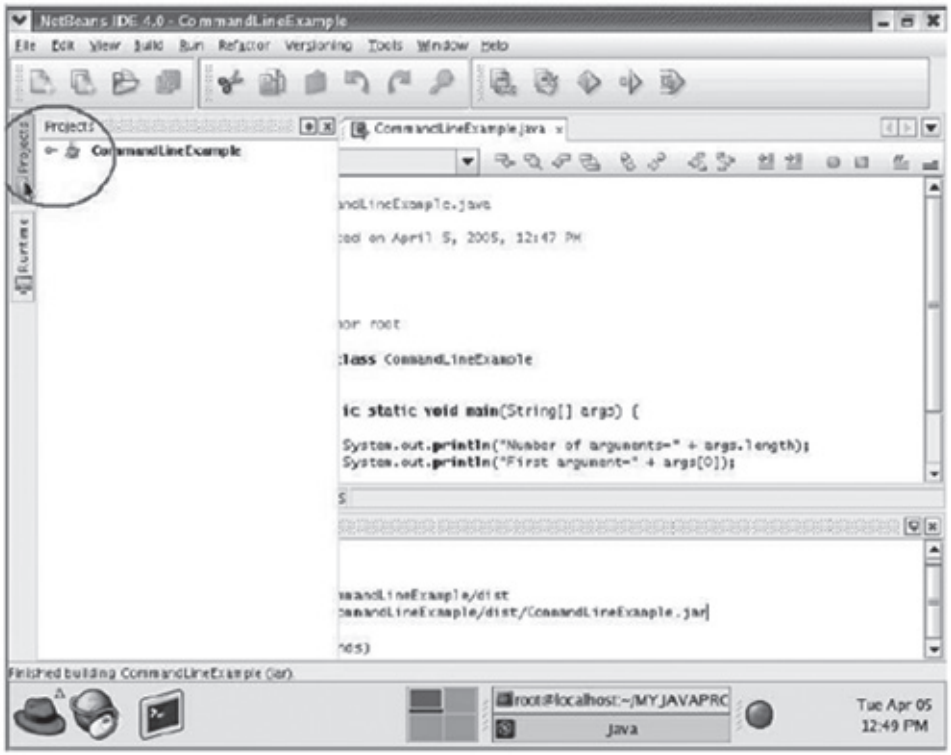
Argumentos de linha de comando no NetBeans

Para ilustrar a passagem de alguns argumentos para um projeto no NetBeans, vamos criar um projeto em Java que mostrará na tela o número de argumentos e o primeiro argumento passado.

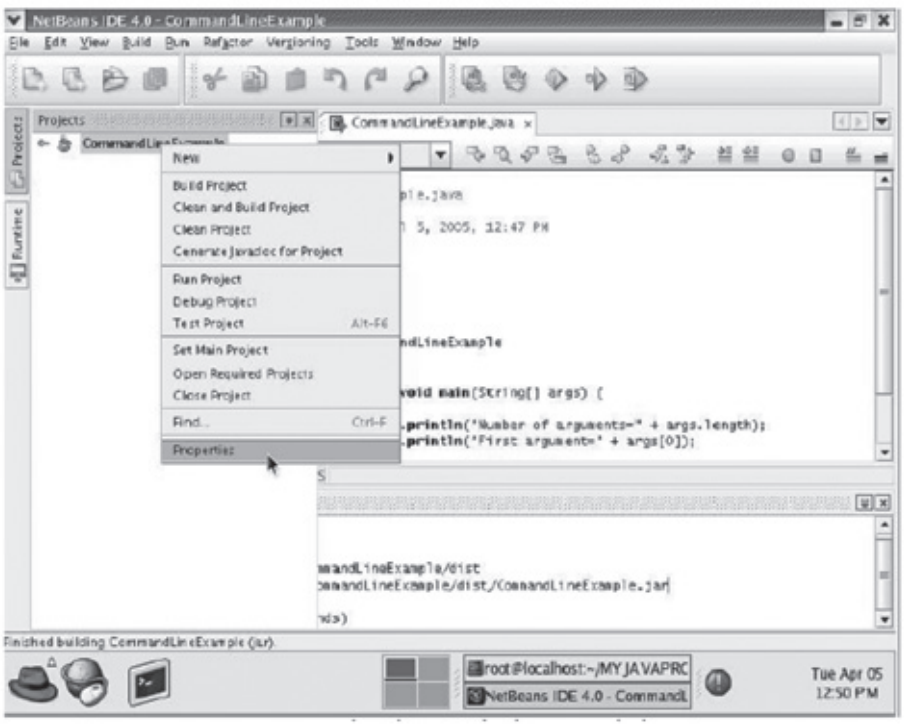
```
public class CommandLineExample {
        public static void main( String[] args ) {
                System.out.println(
                        "Number of arguments=" + args.length);
                System.out.println(
                        "First Argument="+ args[0]);
        }
}
```

Praticando!

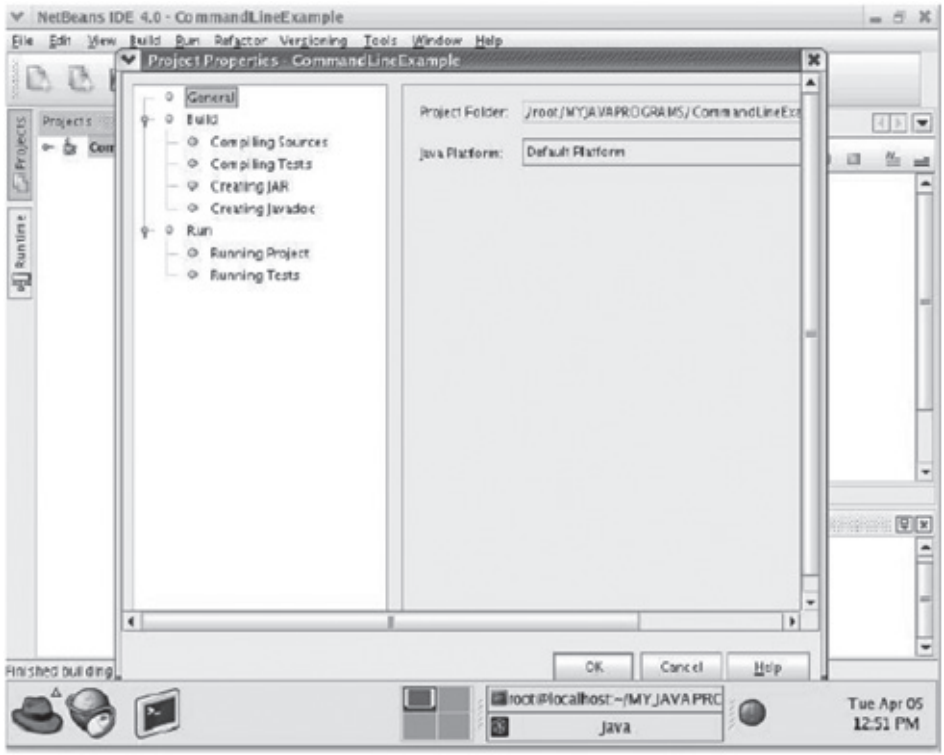
Abra o NetBeans, crie um novo projeto e dê o nome de **CommandLineExample**. Copie o código mostrado anteriormente e o compile. Em seguida, siga estas etapas para passar argumentos para o programa, utilizando o NetBeans.



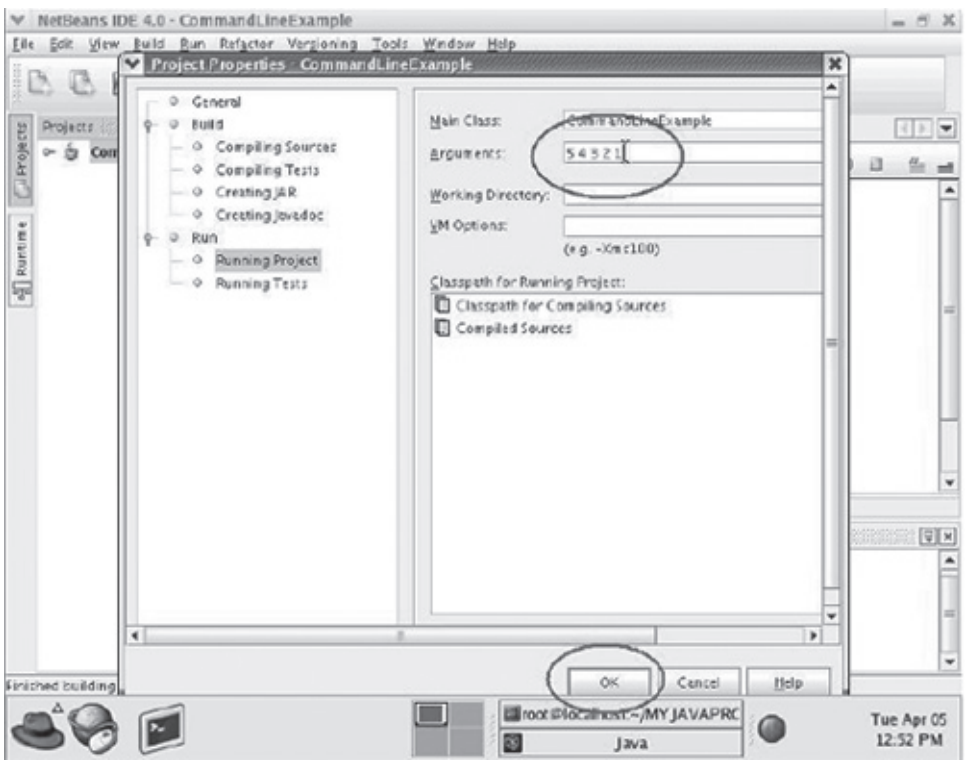
Dê um clique com o botão direito do mouse no ícone **CommandLineExample**, conforme destacado na **figura acima**. Um menu aparecerá, conforme a figura abaixo. Selecione a opção "Properties".



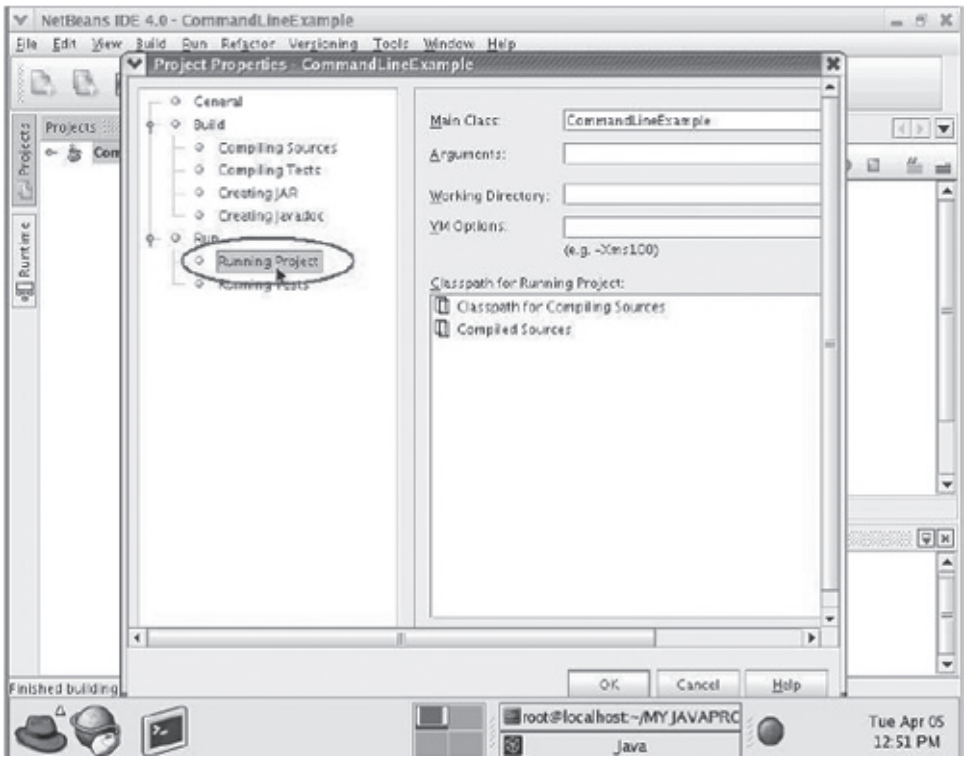
A janela **"Project Properties"** irá aparecer, conforme a figura abaixo:



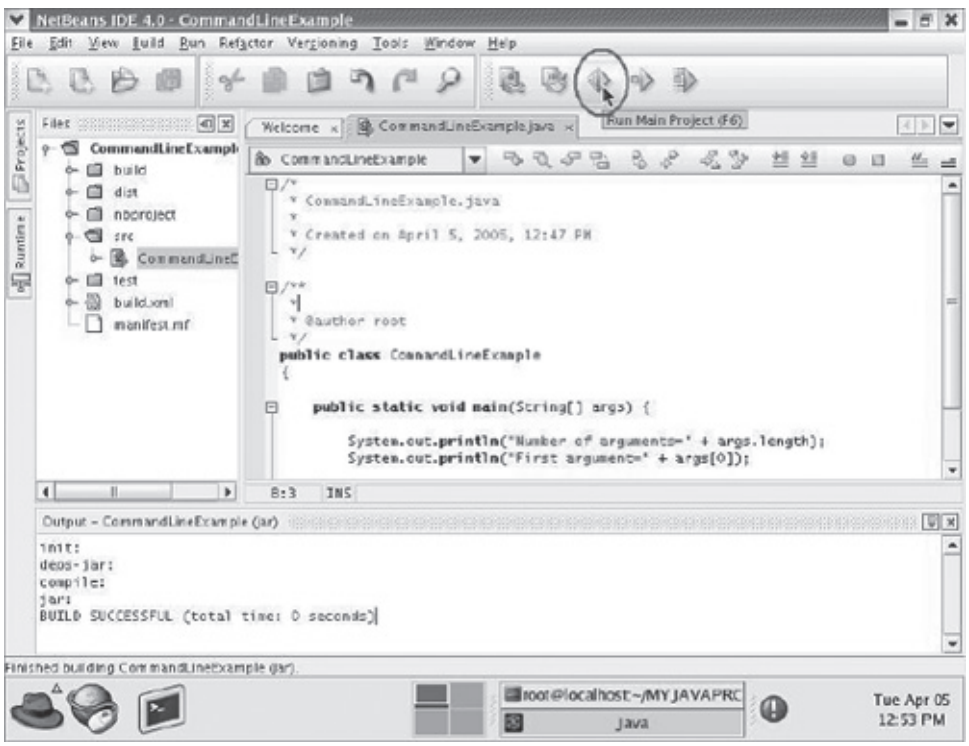
Acesse a opção Run □ Running Project.



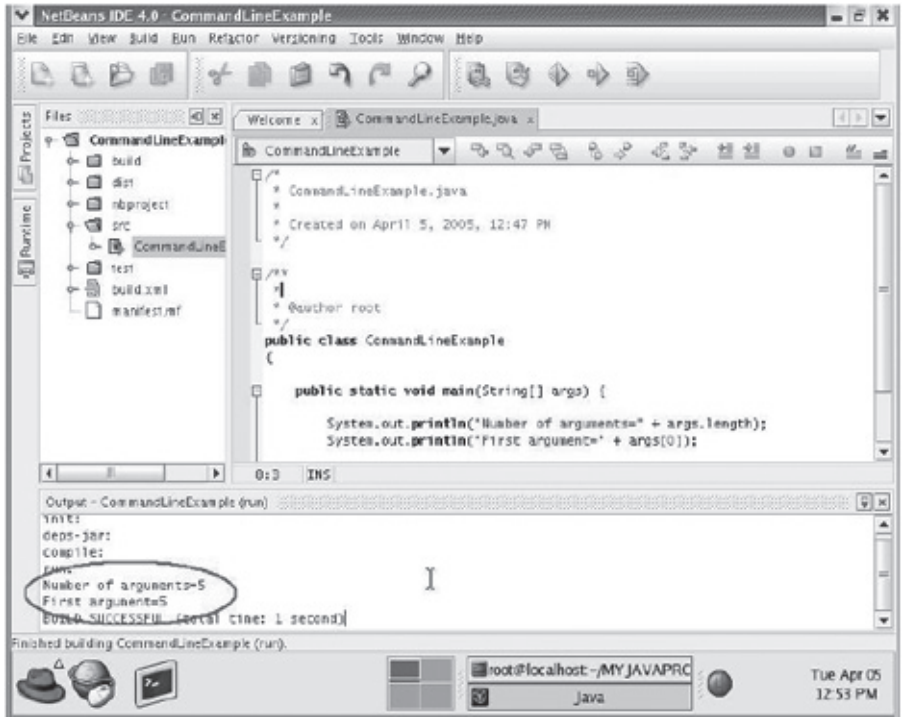
Na caixa de texto dos argumentos, digite os argumentos que se quer passar para o programa. Nesse caso, digitamos os argumentos 5 4 3 2 1. Pressione o botão OK.



Execute o projeto.



Como pode-se ver, a saída do projeto é a quantidade de argumentos, que é 5, e o primeiro argumento passado, que também é 5.



Exercícios Propostos

1. Utilizando os dados passados pelo usuário através dos argumentos de linha de comando, exiba os argumentos recebidos. Por exemplo, se o usuário digitar:

java Hello world that is all

o programa deverá mostrar na tela:

world
that
is
all

2. Obtenha dois números, passados pelo usuário usando argumentos de linha de comando, e mostre o resultado da soma, subtração, multiplicação e divisão destes números. Por exemplo, se o usuário digitar:

java ArithmeticOperation 20 4,

o programa deverá mostrar na tela:

sum = 24
subtraction = 16
multiplication = 80
division = 5

Desafio

• Altere o programa abaixo para receber os parâmetros, através da linha de comando.

```
import javax.swing.*;
public class InputArgs {
    public static void main(String[] args) {
       String name = JOptionPane.showInputDialog
       ("What is your name?");
      String input = JOptionPane.showInputDialog
      ("How old are you?");
       int age = Integer.parseInt(input);
       System.out.println("Hello, " + name +
       ". Next year, you'll be " + (age + 1));
       System.exit(0);
   }
}
```



Resumindo

Nesta unidade discutimos sobre a forma de se ter acesso a conjunto de dados agrupados em um programa Java, através do estudo de arranjos homogêneos e heterogêneos elementares, destacando-se exemplos com vetores, matrizes. Também foram apresentados argumentos como forma de transferências de dados para os aplicativos.

UNIDADE 04

Paradigma de Programação Orientada a Objetos

OBJETIVOS:

- **Conhecer fundamentalmente os recursos de programação orientada por objetos como uma ferramenta poderosa que pode gerar códigos sofisticados e complexos.**
- **Implementar os algoritmos de interesse do aprendiz, usando a linguagem Java, para solidificar os conhecimentos adquiridos nas sub-unidades anteriores.**

4 PARADIGMA DE PROGRAMAÇÃO ORIENTADA A OBJETOS

Classes, Objetos e Métodos

Introdução à Programação Orientada a Objeto

Programação Orientada a Objetos (POO) refere-se ao conceito de **objetos** como elemento básico das classes. O mundo físico é constituído por objetos tais como carro, leão, pessoa, dentre outros. Esses objetos são caracterizados pelas suas **propriedades** (ou **atributos**) e seus **comportamentos**.

Por exemplo, um objeto "carro" tem as propriedades, tipo de câmbio, fabricante e cor. O seu comportamento pode ser 'virar', 'frear' e 'acelerar'. Igualmente, podemos definir diferentes propriedades e comportamentos para um leão. Veja exemplos na **tabela abaixo**:

Objeto	Propriedades	Comportamentos
carro	tipo de câmbio fabricante cor	virar frear acelerar
leão	peso cor apetite (faminto ou saciado) temperamento (dócil ou selvagem)	rugir dormir caçar

Com tais descrições, os objetos do mundo físico podem ser facilmente modelados como objetos de software usando as **propriedades como atributos** e os **comportamentos como métodos**. Esses atributos e métodos podem ser usados em softwares de jogos ou interativos para simular objetos do mundo real! Por exemplo, poderia ser um objeto de

'carro' numa competição de corrida ou um objeto de 'leão' num aplicativo educacional de zoologia para crianças.

Classes e Objetos

Diferenças entre classes e objetos

No mundo do computador, um **objeto** é um componente de software cuja estrutura é similar a um objeto no mundo real. Cada objeto é composto por um conjunto de **atributos** (propriedades) que são as variáveis que descrevem as características essenciais do objeto e consiste também, num conjunto de métodos (comportamentos) que descrevem como esse objeto se comporta. Assim, um objeto é uma coleção de atributos e métodos relacionados. Os atributos e métodos de um objeto Java são formalmente conhecidos como **atributos e métodos de objeto** para distinguir dos atributos e métodos de classes, que serão discutidos mais adiante.

A **classe** é a estrutura fundamental na Programação Orientada a Objetos. Ela pode ser pensada como um gabarito, um protótipo ou, ainda, uma **planta** para a construção de um objeto. Ela consiste em dois tipos de elementos que são chamados **atributos** (ou propriedades) e **métodos**. Atributos especificam os tipos de dados definidos pela classe, enquanto que os métodos especificam as operações. Um objeto é uma **instância** de uma classe.

Para diferenciar entre classes e objetos, vamos examinar um exemplo. O que temos aqui é uma classe Carro que pode ser usada pra definir diversos objetos do tipo carro. Na tabela mostrada abaixo, Carro A e Carro B são objetos da classe Carro. A classe tem os campos número da placa, cor, fabricante e velocidade que são preenchidos com os valores correspondentes do carro A e B. O carro também tem alguns métodos: acelerar, virar e frear.

Classe Carro		**Objeto Carro A**	**Objeto Carro B**
Atributos de Objeto	Número da placa	ABC 111	XYZ 123
	Cor	Azul	Vermelha
	Fabricante	Mitsubishi	Toyota
	Velocidade	50 km/h	100 km/h
Modelos de Objeto	Método Acelerar		
	Método Girar		
	Método Frear		

Quando construídos, cada objeto adquire um conjunto novo de estado. Entretanto, as implementações dos métodos são compartilhadas entre todos os objetos da mesma classe.

As classes fornecem o benefício da **Reutilização de Classes** (ou seja, utilizar a mesma classe em vários projetos). Os programadores de software podem reutilizar as classes várias vezes para criar os objetos.

Encapsulamento

Encapsulamento é um princípio que propõe ocultar determinados elementos de uma classe das demais classes. Ao colocar uma proteção ao redor dos atributos e criar métodos para prover o acesso a estes, desta forma estaremos prevenindo contra os efeitos colaterais indesejados que podem afetá-los ao ter essas propriedades modificadas de forma inesperada.

Podemos prevenir o acesso aos dados dos nossos objetos declarando que temos controle desse acesso. Aprenderemos mais sobre como Java implementa o encapsulamento quando discutirmos mais detalhadamente sobre as classes.

Atributos e métodos de classe

Além dos atributos de objeto, também é possível definir **atributos de classe**, que são atributos que pertencem à classe como um todo. Isso significa que possuem o mesmo valor para todos os objetos daquela classe e, também, são chamados de **atributos estáticos**. Para melhor descrever os atributos de classe, vamos voltar ao exemplo da classe **Carro**.

Suponha que a classe **Carro** tenha um atributo de classe chamado **Contador**. Ao mudarmos o valor de **Contador** para 2, todos os objetos da classe **Carro** terão o valor 2 para seus atributos Contador.

Classe Carro		**Objeto Carro A**	**Objeto Carro B**
Atributos de Objeto	Número da placa	ABC 111	XYZ 123
	Cor	Azul	Vermelha
	Fabricante	Mitsubishi	Toyota
	Velocidade	50 km/h	100 km/h
Atributos de Classe	Contador = 2		

Modelos de Objeto	Método Acelerar
	Método Girar
	Método Frear

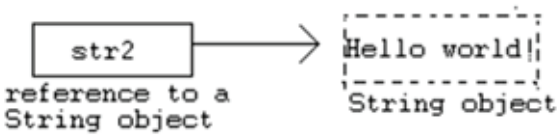
Instância de Classe

Para criar um objeto ou uma instância da classe, utilizamos o operador **new**. Por exemplo, para criar uma instância da classe String, escrevemos o seguinte código:

String str2 = new String("Hello world!");

ou, o equivalente:

String str2 = "Hello world!";



O operador **new** aloca a memória para o objeto e retorna uma **referência** para essa alocação.

Ao criar um objeto, invoca-se, na realidade, o **construtor** da classe. O **construtor** é um método onde todas as inicializações do objeto são declaradas e possui o mesmo nome da classe.

Métodos

O que são métodos e porque usar métodos?

Nos exemplos apresentados anteriormente, temos apenas um método, o método main(). Em Java, nós podemos definir vários métodos e podemos chamá-los a partir de outros métodos.

Um **método** é um trecho de código distinto que pode ser chamado por qualquer outro método para realizar alguma função específica.

Métodos possuem as seguintes características:

• Podem ou não retornar um valor;

• Podem aceitar ou não argumentos;

• Após o método encerrar sua execução, o fluxo de controle é

retornado a quem o chamou.

O que é necessário para se criar métodos? Por que não colocamos todas as instruções dentro de um grande método? O foco dessas questões é chamado de **decomposição**. Conhecido o problema, nós o separamos em partes menores, que torna menos crítico o trabalho de escrever grandes classes.

Chamando métodos de objeto e enviando argumentos

Para ilustrar como chamar os métodos, utilizaremos como exemplo a classe **String**. Pode-se usar a documentação da API Java para conhecer todos os atributos e métodos disponíveis na classe **String**. Posteriormente, iremos criar nossos próprios métodos.

Para chamar um **método** a partir de um objeto, escrevemos o seguinte:

nomeDoObjeto.nomeDoMétodo([argumentos])

Vamos pegar dois métodos encontrados na classe String como exemplo:

Declaração do método	Definição
`public char charAt(int index)`	Retorna o caractere especificado no índice. Um índice vai de 0 até length() - 1. O primeiro caractere da seqüência está no índice 0, o seguinte, no índice 1, e assim sucessivamente por todo o array.
`public boolean equalsIgnoreCase (String anotherString)`	Compara o conteúdo de duas Strings, ignorando maiúsculas e minúsculas. Duas strings são consideradas iguais quando elas têm o mesmo tamanho e os caracteres das duas strings são iguais, sem considerar caixa alta ou baixa.

Usando os métodos:

```
String str1 = "Hello";
char x = str1.charAt(0); // retornará o caracter H
// e o armazenará no atributo x
String str2 = "hello";
// aqui será retornado o valor booleano true
boolean result = str1.equalsIgnoreCase(str2);
```

Envio de argumentos para métodos

Em exemplos anteriores, enviamos atributos para os métodos. Entretanto, não fizemos nenhuma distinção entre os diferentes tipos de atributos que podem ser enviados como argumento para os métodos. Há duas formas para se enviar argumentos para um método, o primeiro é envio por valor e o segundo é envio por referência.

Envio por valor

Quando ocorre um **envio por valor**, a chamada do método faz uma cópia do valor do atributo e o reenvia como argumento. O método chamado não modifica o valor original do argumento mesmo que estes valores sejam modificados durante operações de cálculo implementadas pelo método. Por exemplo:

```
public class TestPassByValue {
     public static void main( String[] args ){
      int i = 10;
      //exibe o valor de i
      System.out.println( i );

      //chama o método test
      //envia i para o método test
      test( i );

      //exibe o valor de i não modificado
      System.out.println( i );
     }

     public static void test( int j ){
      //muda o valor do argumento j
      j = 33;
     }
}
```

enviado como argumento. O valor de **i** é copiado para o atributo do método **j**. Já que **j** é o atributo modificado no método **test**, não afetará o valor do atributo **i**, o que significa uma cópia diferente do atributo.

Como padrão, todo tipo primitivo, quando enviado para um método, utiliza a forma de **envio por valor.**

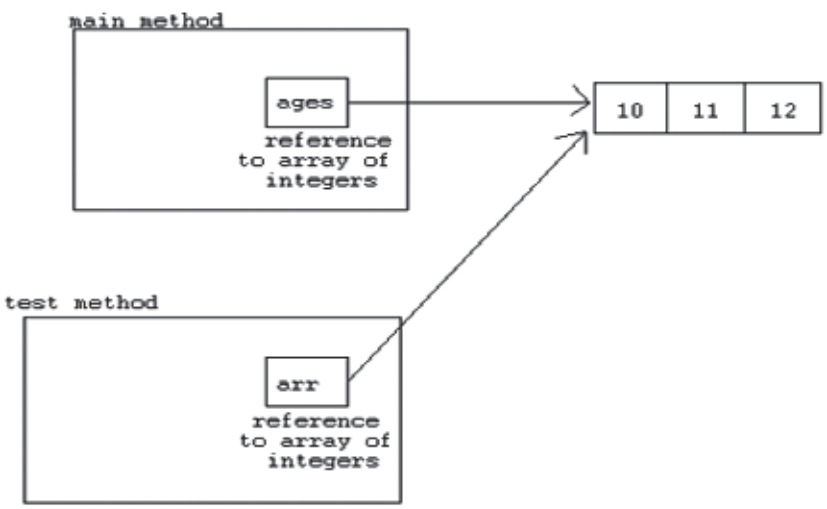
Envio por referência

Quando ocorre um **envio por referência**, a referência de um objeto é enviada para o método chamado. Isso significa que o método faz uma cópia da referência do objeto enviado.

Entretanto, diferentemente do que ocorre no envio por valor, o método pode modificar o objeto para o qual a referência está apontando. Mesmo que diferentes referências sejam usadas nos métodos, a localização do dado para o qual ela aponta é a mesma. Por exemplo:

```
class TestPassByReference {
    public static void main(String[] args) {
        // criar um array de inteiros
        int []ages      = {10, 11, 12};
        // exibir os valores do array
        for (int i=0; i < ages.length; i++) {
            System.out.println( ages[i] );
        }
        // chamar o método test e enviar a
        // referência para o array
        test( ages );

        // exibir os valores do array
        for (int i=0; i < ages.length; i++) {
            System.out.println(ages[i]);
        }
    }
    public static void test( int[] arr ){
        // mudar os valores do array
        for (int i=0; i < arr.length; i++) {
            arr[i] = i + 50;
        }
    }
}
```



Dicas de programação:

1. Um erro comum sobre **envio por referência** *acontece quando criamos um método para fazer trocas (***swap***) usando referência. Note que Java manipula objetos 'por referência', entretanto envia-se a referência para um método 'por valor'. Como consequência, não se escreve um método padrão para fazer troca de valores (***swap***) entre objetos.*

Chamando métodos estáticos

Métodos estáticos são métodos que podem ser invocados sem que um objeto tenha sido instanciado pela classe (sem utilizar a palavra-chave new). Métodos estáticos pertencem a classe como um todo e não ao objeto especifico da classe. Métodos estáticos são diferenciados dos métodos de objeto pela declaração da palavra-chave static na definição do método.

Para chamar um método estático, digite:

NomeClasse.nomeMétodoEstático(argumentos)

Alguns métodos estáticos, que já foram usados em nossos exemplos são:

```
// Converter a String 10 em um atributo do tipo inteiro
int i = Integer.parseInt("10")

// Retornar uma String representando um inteiro sem o sinal da
// base 16
String hexEquivalent = Integer.toHexString(10)
```

Escopo de um atributo

Além do atributo ter um nome e um tipo, ele também possui um **escopo**. O escopo determina onde o atributo é acessível dentro da classe. O escopo também determina o tempo de vida do atributo ou quanto tempo o atributo irá existir na memória. O escopo é determinado pelo local onde o atributo é declarado na classe.

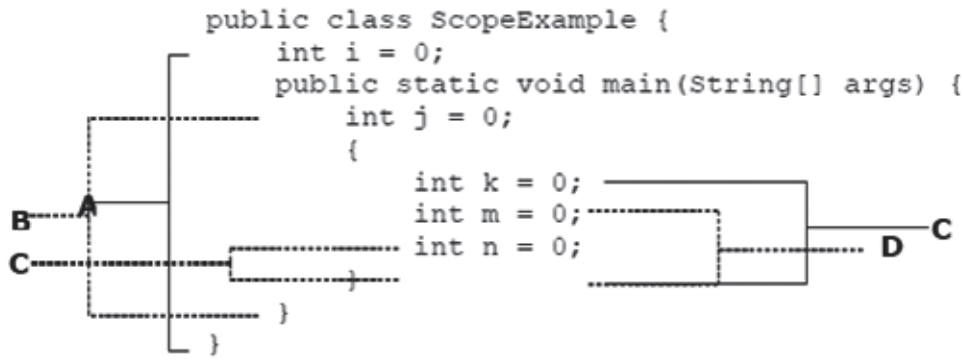
Para simplificar, vamos pensar no escopo como sendo algo existente entre as chaves {...}. A chave à direita é chamada de chave de saída do bloco (**outer**) e a chave à esquerda é chamada chave de entrada do bloco (**inner**).

Ao declarar atributos fora de um bloco, eles serão visíveis (usáveis), inclusive pelas linhas da classe dentro do bloco. Entretanto, ao declarar os atributos dentro do bloco, não será possível utilizá-los fora do bloco.

O escopo de um atributo é dito local quando é declarado dentro do bloco. Seu escopo inicia com a sua declaração e vai até a chave de saída

do bloco.

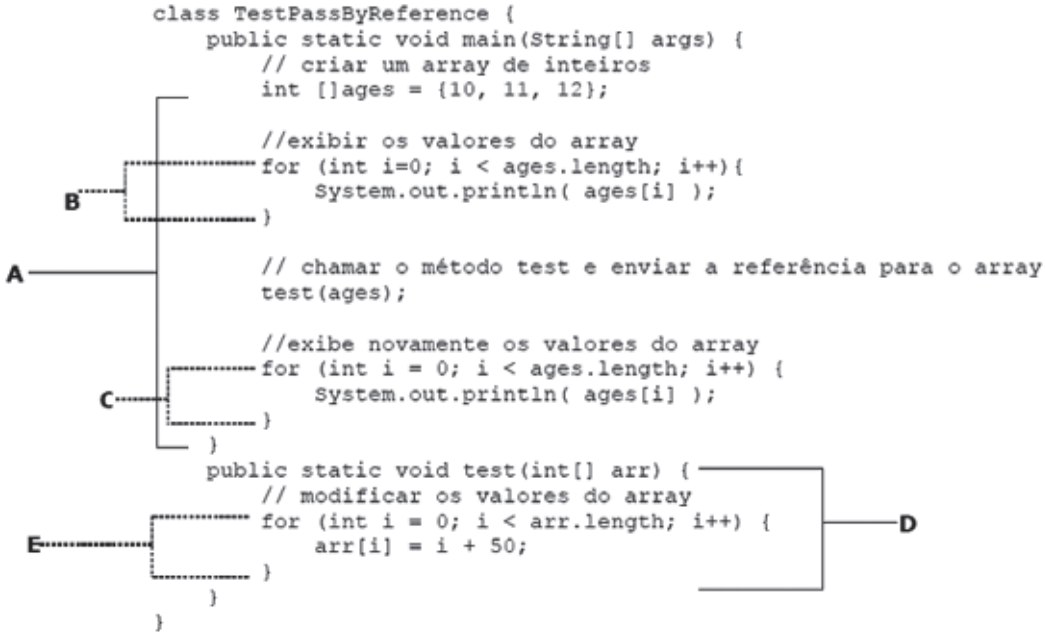
Por exemplo, dado o seguinte fragmento de código:



O código acima representa cinco escopos indicado pelas letras. Dados os atributos i, j, k, m e n, e os cinco escopos A, B, C, D e E, temos os seguintes escopos para cada atributo:

O escopo do atributo i é A.
O escopo do atributo j é B.
O escopo do atributo k é C.
O escopo do atributo m é D.
O escopo do atributo n é E.

Dado dois métodos main e test, teremos o seguinte exemplo:



Para o método main, os escopos dos atributos são:

ages[] - escopo A
i em B - escopo B
i em C – escopo C

E, no método test, os escopos dos atributos são:

arr[] - escopo D
i em E - escopo E

Quando atributos são declarados, o identificador deve ser único no escopo. Isso significa que se você tiver a seguinte declaração:

```
{
int test = 10;
int test = 20;
}
```

Dicas de programação:
1. Evite ter atributos declarados com o mesmo nome dentro de um método para não causar confusão.

O compilador irá gerar um erro pois deve-se ter um nome único para o atributos dentro do bloco. Entretanto, é possível ter atributos definidos com o mesmo nome, se não estiverem declarados no mesmo bloco. Por exemplo:

```
public class TestBlock {
        int test = 10;
        public void test() {
                System.out.print(test);
                int test = 20;
                System.out.print(test);
        }
        public static void main(String[] args) {
                TestBlock testBlock = new TestBlock();
                testBlock.test();
        }
}
```

Quando a primeira instrução **System.out.print** for invocada, exibirá o valor **10** contido na primeira declaração do atributo **test**. Na segunda instrução **System.out.print,** o valor **20** é exibido, pois é o valor do atributos **test** neste escopo.

Casting, conversão e comparação de objetos

Nesta seção, vamos aprender como realizar um casting. Casting, ou typecasting, é o processo de conversão de um certo tipo de dado para outro. Também aprenderemos como converter tipos de dados primitivos para objetos e vice-versa. E, finalmente, aprenderemos como comparar objetos.

Casting de tipos primitivos

Casting entre tipos primitivos permite converter o valor de um dado de um determinado tipo para outro tipo de dado primitivo. O casting entre primitivos é comum para os tipos numéricos.

Há um tipo de dado primitivo que não aceita o casting, o tipo de dado **boolean**.

Como demonstração de casting de tipos, considere que seja necessário armazenar um valor do tipo **int** em um atributo do tipo **double**. Por exemplo:

```
int numInt = 10;
double numDouble = numInt; // cast implícito
```

uma vez que o atributo de destino é double, pode-se armazenar um valor cujo tamanho seja menor ou igual aquele que está sendo atribuído. O tipo é convertido implicitamente.

Quando convertemos um atributo cujo tipo possui um tamanho maior para um de tamanho menor, necessariamente devemos fazer um casting explícito. Esse possui a seguinte forma:

(tipoDado) valor, onde:

tipoDado é o nome do tipo de dado para o qual se quer converter o valor;

valor é um valor que se quer converter.

Por exemplo:

```
double valDouble = 10.12;
int valInt = (int)valDouble; //converte valDouble para o tipo int
double x = 10.2;
int y = 2;
int result = (int)(x/y); //converte o resultado da operação para int
```

Outro exemplo é quando desejamos fazer um casting de um valor do tipo **int** para **char**. Um caractere pode ser usado como int porque para cada caractere existe um correspondente numérico que representa sua posição no conjunto de caracteres. O casting **(char)65** irá produzir a saída 'A'. O código numérico associado à letra maiúscula A é 65, segundo o conjunto de caracteres ASCII. Por exemplo:

```
char valChar = 'A';
System.out.print((int)valChar); //casting explícito produzirá 65
```

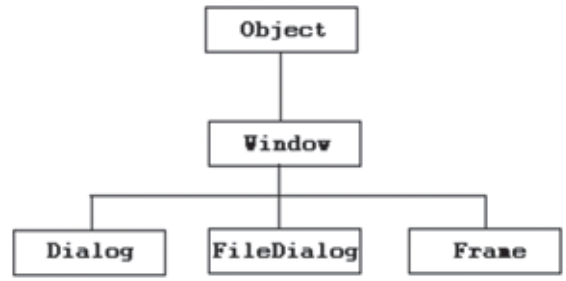
Casting de Objetos

Para objetos, a operação de casting também pode ser utilizada para fazer a conversão para outras classes, com a seguinte restrição: a classe de origem e a classe de destino devem ser da mesma família, relacionadas por herança; uma classe deve ser subclasse da outra. Veremos mais em lições posteriores sobre herança.

Analogamente à conversão de valores primitivos para um tipo maior, alguns objetos não necessitam ser convertidos explicitamente. Em consequência de uma subclasse conter todas as informações da sua superclasse, pode-se usar um objeto da subclasse em qualquer lugar onde a superclasse é esperada.

Por exemplo, um método que recebe dois argumentos, um deles do tipo **Object** e outro do tipo **Window**. Pode-se enviar um objeto de qualquer classe como argumento Object porque todas as classes Java são subclasses de **Object**. Para o argumento **Window**, é possível enviar apenas suas subclasses, tais como **Dialog**, **FileDialog**, **Frame** (ou quaisquer de subclasses de suas subclasses, indefinidamente). Isso vale para qualquer parte da classe, não apenas dentro da chamada do método. Para um objeto definido como uma classe Window, é possível atribuir objetos dessa classe ou qualquer uma de suas subclasses para

esse objeto sem o casting.



O contrá erclasse pode ser usada quando uma subclasse é esperada. Entretanto, nesse caso, o casting é necessário porque as subclasses contêm mais métodos que suas superclasses, e isso acarreta em perda de precisão. Os objetos das superclasses podem não dispor de todo o comportamento necessário para agir como um objeto da subclasse. Por exemplo, se uma operação faz a chamada a um método de um objeto da classe **Integer**, usando um objeto da classe **Number**, ele não terá muitos dos métodos que foram especificados na classe **Integer**. Erros ocorrerão se você tentar chamar métodos que não existem no objeto de destino.

Para usar objetos da superclasse onde uma subclasse é esperada, é necessário fazer o ***casting* explícito.** Nenhuma informação será perdida no casting, entretanto, ganhará todos os atributos e métodos que a subclasse define. Para fazer o casting de um objeto para outro, utiliza-se a mesma operação utilizada com os tipos primitivos.

Para fazer o *casting*:

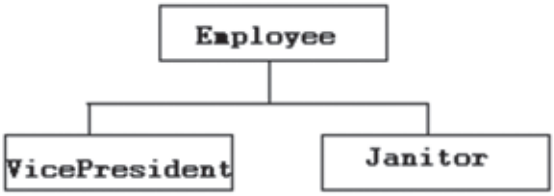
(nomeClasse)objeto

onde:

nomeClasse é o nome da classe destino;

objeto é a referência para o objeto origem que se quer converter.

Uma vez que casting cria uma referência para o antigo objeto de nomeClasse, ele continua a existir depois do casting.



O exemplo a seguir realiza um casting de um objeto da classe VicePresident para um objeto da classe Employee. VicePresident é uma subclasse de Employee com mais informações, supondo que tenha sido definido que a classe VicePresident tem mais privilégios que um Employee.

```
Employee emp = new Employee();
VicePresident veep = new VicePresident();
emp = veep; // casting não é necessário de baixo para cima
veep = (VicePresident)emp; // necessita do casting explícito
```

Convertendo tipos primitivos para objetos e vice-versa

Dicas de programação:
1. A classe Void representa vazio em Java. Desse modo, não existem motivos para essa classe ser usada na conversão de valores primitivos e objetos. Ela é um tratador para a palavra-chave void, que é utilizada na assinatura de métodos indicando que estes não retornam valor.

Não se pode fazer, sob qualquer circunstância, um casting de um objeto para um tipo de dado primitivo, ou vice-versa. Tipos primitivos e objetos são muito diferentes em Java e não se pode fazer o casting automaticamente entre os dois ou intercambiar o seu uso.

Como alternativa, o pacote java.lang oferece classes que fazem a correspondência para cada tipo primitivo: Float, Boolean, Byte, dentre outros. Muitas dessas classes têm o mesmo nome dos tipos de dados primitivos, exceto pelo fato que o nome dessas classe começam com letra maiúscula (Short ao invés de short, Double ao invés de double, etc). Há duas classes que possuem nomes que diferem do correspondente tipo de dado primitivo: Character é usado para o tipo char e Integer usado para o tipo int. Estas classes são chamadas de Wrapper Class.

Versões anteriores a 5.0 de Java tratam os tipos de dados e suas Wrapper Class de forma muito diferente, e uma classe não será compilada com sucesso se for utilizada uma quando deveria usar a outra. Por exemplo:

```
Integer ten = 10;
Integer two = 2;
System.out.println(ten + two).
```

Usando as classes que correspondem a cada um dos tipos primitivos, é possível criar objetos que armazenam esse mesmo valor. Por exemplo:

```
// A declaração seguinte cria um objeto de uma classe Integer
// com o valor do tipo int 7801 (primitivo -> objeto)
Integer dataCount = new Integer(7801);

// A declaração seguinte converte um objeto Integer para um
// tipo de dado primitivo int. O resultado é o valor 7801
int newCount = dataCount.intValue();

// Uma conversão comum de se fazer é de String para um tipo
// numérico (objeto -> primitivo)
String pennsylvania = "65000";
int penn = Integer.parseInt(pennsylvania);
```

Comparando Objetos

Em lições prévias, aprendemos sobre operadores para comparar valores — igualdade, negação, menor que, etc. Muitos desses operadores trabalham apenas com dados de tipo primitivo, não com objetos. Ao tentar utilizar outros tipos de dados, o compilador produzirá erros.

Uma exceção para essa regra são os operadores de igualdade: == (igual) e != (diferente). Quando aplicados a objetos, esses operadores não fazem o que se poderia supor. Ao invés de verificar se um objeto tem o mesmo valor de outro objeto, eles determinam se os dois objetos comparados pelo operador têm a mesma referência. Por exemplo:

```
String valor1 = new String;
Integer dataCount = new Integer(7801);
```

Para comparar objetos de uma classe e ter resultados apropriados, deve-se implementar e chamar métodos especiais na sua classe. Um bom exemplo disso é a classe String.

É possível ter dois objetos String diferentes que contenham o mesmo valor. Caso se empregue o operador == para comparar objetos, estes serão considerados diferentes. Mesmo que seus conteúdos sejam iguais, eles não são o mesmo objeto.

Para ver se dois objetos String têm o mesmo conteúdo, um método chamado equals() é utilizado. O método compara cada caractere presente no conteúdo das Strings e retorna true se ambas strings tiverem o mesmo valor.

O código seguinte ilustra essa comparação,

```
class EqualsTest {
        public static void main(String[] args) {
        String str1, str2;
        str1 = "Free the bound periodicals.";
        str2 = str1;

        System.out.println("String1: " + str1);
        System.out.println("String2: " + str2);
        System.out.println("Same object? " + (str1 == str2));

        str2 = new String(str1);

        System.out.println("String1: " + str1);
        System.out.println("String2: " + str2);
        System.out.println("Same object? " + (str1 == str2));
        System.out.println("Same value? " + str1.equals(str2));
        }
}
```

Dicas de programação:
1. Por que não se pode ter a mesma literal quando mudamos str2, a não ser quando utilizamos o new? Literais String são otimizadas em Java; se uma String é criada utilizando uma literal e cria-se outra String com os mesmos caracteres, Java tem inteligência suficiente para retornar apenas a posição em memória do primeiro objeto String criado. Ou seja, ambas strings se referem ao mesmo objeto como se fosse um apelido. Para obrigar ao Java criar um novo objeto String, deve-se utilizar o operador new.

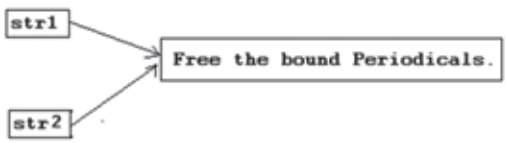
A saída dessa classe é a seguinte:

```
String1: Free the bound periodicals.
String2: Free the bound periodicals.
Same object? true
```

String1: Free the bound periodicals.
String2: Free the bound periodicals.
Same object? false
Same value? True

Vamos entender o processo envolvido.

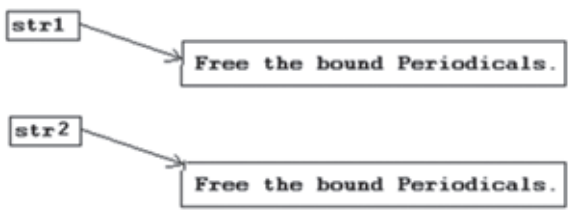
```
String str1, str2;
str1 = "Free the bound periodicals.";
str2 = str1;
```



A primeira parte dessa classe declara dois atributos (str1 e str2) e atribui a literal "Free the bound periodicals " a str1, e depois atribui esse valor a str2. Como visto anteriormente, str1 e str2 agora apontam para o mesmo objeto, e o teste de igualdade prova isso.

Em seguida, temos a seguinte instrução:

```
str2 = new String(str1);
```



Na segunda parte da classe, cria-se um novo objeto String com o mesmo valor de str1 e faz-se a atribuição de str2 para esse novo objeto String. Agora temos dois diferentes tipos de objetos na str1 e str2, ambos com o mesmo conteúdo. Testando para ver se eles são o mesmo objeto usando o operador de igualdade obtemos a resposta esperada: **false** — eles não são o mesmo objeto na memória. Utilizando o método **equals()**, recebemos a resposta esperada: **true** — eles tem o mesmo conteúdo.

Determinando a classe de um objeto

Existem duas maneiras de se descobrir a qual classe determinado objeto pertence:

1. Para obter o nome da classe:

Utiliza-se o método **getClass()** que retorna a classe do objeto (onde Class é a classe em si). Esta, por sua vez, possui o método chamado **getName()** que retorna o nome da classe. Por exemplo:

```
String name = key.getClass().getName();
```

2. Para testar se um objeto qualquer foi instanciado de uma determinada classe:

Utiliza-se a palavra-chave instanceof. Esta palavra-chave possui dois operadores: a referência para o objeto à esquerda e o nome da classe à direita. A expressão retorna um lógico dependendo se o objeto é uma instância da classe declarada ou qualquer uma de suas subclasses.

Por exemplo:

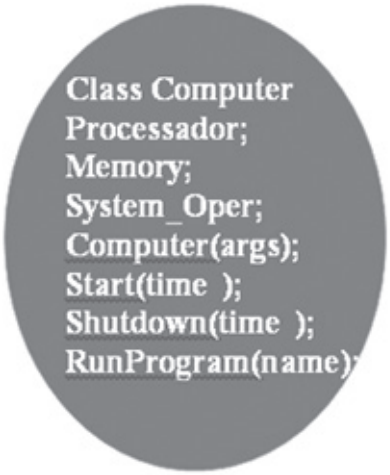
```
String ex1 = "Texas";
System.out.println(ex1 instanceof String); // retorna true
String ex2;
System.out.println(ex2 instanceof String); // retorna false
```

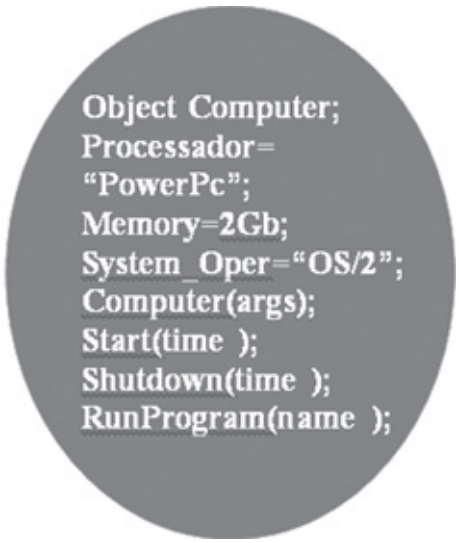


Exercícios Propostos

1. Conceitue os termos, com suas próprias palavras: a) Classe, b) Objeto, c) Instanciação, e) Atributo de objeto, f) Método de objeto, g) Atributo de classe ou atributos estáticas, h) Construtor, i) Método de classe ou métodos estáticos.

2. Identifique, na figua abaixo, os elementos: a) Classe, b) Objeto, c) Instância, e) Método, f) Atributo, g) Construtor, h) Mensagem, i) Argumento.





3. **Pipoy** é um novato na linguagem de programação Java. Ele apenas ouviu que existem Java APIs (Application Programming Interface) prontas para serem usadas em suas classes e ele está ansioso para fazer uns testes com elas. O problema é que **Pipoy** não tem uma cópia da documentação Java e, também, não tem acesso à Internet, desse modo, não há como ele ver as APIs java.

Sua tarefa é ajudar **Pipoy** a procurar as APIs. Você deve informar as classes às quais os métodos pertencem e como o método deve ser declarado com um exemplo de uso desse método.

Por exemplo, se Pipoy quer saber qual o método que converte uma String para int, sua resposta deve ser:

Classe: Integer
Declaração do Método: public static int parseInt(String value)
Exemplo de Uso:

```
String strValue = "100";
int value = Integer.parseInt( strValue );
```

Tenha certeza de que o fragmento de código que você escreveu em seu exemplo de uso compila e que produza o resultado correto. Enfim, não deixe **Pipoy** confuso. (**Dica: Todos os métodos estão no package java.lang**). Caso haja mais de um método para atender à tarefa, utilize apenas um.

Agora vamos iniciar a busca! Aqui estão alguns métodos que **Pipoy** necessita:

a) Procure, pelo método que verifica, se uma String termina com um determinado sufixo. Por exemplo, se a String dada é "Hello", o método deve retornar true se o sufixo informado é "lo", e false se o sufixo for "alp".

b) Procure, pelo método que determina, a representação do caractere para um dígito e base específica. Por exemplo, se o dígito informado é 15 e a base é 16, o método retornará o caractere 'F', uma vez que 'F' é a representação hexadecimal para o número 15 em base 10.

c) Procure, por um método que retorna, a parte inteira de um valor double. Por exemplo, se a entrada for 3.13, o método deve retornar o valor 3.

d) Procure por um método que determina se um certo caractere é um dígito. Por exemplo, se a entrada for '3', retornará o valor true.

e) Procure, por um método que interrompe, a execução da Java Virtual Machine corrente.

4. A emprea MagSoft contrata desenvolvedores para trabalhar em projetos de software. Apresente um modelo de implementação em objetos que possa representar o sistema de controle de projetos da empresa. Determine as classes, atributos e métodos.

Desafio:

• O código seguinte cria uma lista de objetos empregados, associados a uma identificação. Perceba que, nas classes empregado e empresa, está declarado o método main(). Compile e execute as duas classes. Justique a execução. Agora altere o código para não permitir execução da classe empregado e na criação de cada empregado incrementar o identificador automaticamente.

```
// classe Empresa
public class Empresa
   public static void main(String[] args) {
   // fill the staff array with three Employee objects
   Employee[] staff = new Employee[3];
   staff[0] = new Employee("Tom", 40000);
   staff[1] = new Employee("Dick", 60000);
   staff[2] = new Employee("Harry", 65000);
   // print out information about all Employee objects
   for (int i = 0; i < staff.length; i++)
       Employee e = staff[i];
     e.setId();
     System.out.println("name=" + e.getName()
      + ",id=" + e.getId()
      + ",salary=" + e.getSalary());
     }
   int n = Employee.getNextId(); // calls static method
   System.out.println("Next available id=" + n);
   }
 }
// classe Empregado
class Employee {
public Employee(String n, double s) {
   name = n;
   salary = s;
    id = 0;
 }
 public String getName() {
    return name;
 }
 public double getSalary() {
    return salary;
 }
 public int getId() {
     return id;
 }
 public void setId() {
     id = nextId; // set id to next available id
     nextId++;
 }
public static int getNextId() {
    return nextId; // returns static field
 }
 public static void main(String[] args) // unit test {
      Employee e = new Employee("Harry", 50000);
      System.out.println(e.getName() + " " + e.getSalary());
 }
 private String name;
 private double salary;
 private int id;
```

Definição de Classe em Java

Definindo classes

Antes de escrever sua classe, primeiro pense onde e como sua classe será usada. Pense em um nome apropriado para a classe e liste todas as informações ou propriedades que deseje que ela tenha. Liste também os métodos que serão usados para a classe.

Para definir uma classe, escrevemos:

<modificador> class <nome> {*
*<declaraçãoDoAtributo>**
*<declaraçãoDoConstrutor>**
*<declaraçãoDoMétodo>**
}

onde:

Dicas de programação:
1. Lembre-se de que, para a declaração da classe, o único modificador de acesso válido é o public. De uso exclusivo para a classe que possuir o mesmo nome do arquivo externo.

<modificador> *é um modificador de acesso, que pode ser usado em combinação com outros*

<nome> *nome da sua classe*

<declaraçãoDoAtributo> *atributos definidos para a classe*

<declaraçãoDoConstrutor> *método construtor*

<declaraçãoDoMétodo> *métodos da classe*

Dicas de programação:
1. Pense em nomes apropriados para a sua classe. Não a chame simplesmente de classe XYZ ou qualquer outro nome aleatório.
2. Os nomes de classes devem ser iniciadas por letra maiúscula.
3. O nome do arquivo de sua classe obrigatoriamente possui o mesmo nome da sua classe pública.

Nesta lição, criaremos uma classe que conterá o registro de um estudante. Como já identificamos o objetivo da nossa classe, agora podemos nomeá-la. Um nome apropriado para nossa classe seria StudentRecord.

Para definir nossa classe, escrevemos:

public class StudentRecord {
// adicionaremos mais código aqui
}

onde:

public modificador de acesso e significa que qualquer classe pode acessar esta
class palavra-chave usada para criar uma classe
StudentRecord identificador único que identifica a classe

Declarando atributos

Para declarar um certo atributo para a nossa classe, escrevemos:

<modificador> <tipo> <nome> [= <valorInicial>];*

onde:

modificador tipo de modificador do atributo
tipo tipo do atributo
nome pode ser qualquer identificador válido
valorInicial valor inicial para o atributo

Relacionaremos a lista de atributos que um registro de **estudante** pode conter. Para cada informação, listaremos os tipos de dados apropriados que serão utilizados. Por exemplo, não seria ideal usar um tipo **int** para o **nome do estudante** ou **String** para a **nota do estudante**.

Abaixo, por exemplo, temos algumas informações que podemos adicionar ao registro do estudante:

nome - String
endereço - String
idade - int
nota de matemática - double
nota de inglês - double
nota de ciências - double

Futuramente, é possível adicionar mais informações. Para este exemplo, utilizaremos somente estas.

Atributos de objeto

Dicas de programação:
1. Declare todas os atributos de objeto na parte superior da declaração da classe.
2. Declare cada atributo em uma linha.
3. Atributos de objeto, assim como qualquer outro atributo devem iniciar com letra **minúscula**.
4. Use o tipo de dado apropriado para cada atributo declarado.
5. Declare atributos de objetos como **private** de modo que somente os métodos da classe possam acessá-los diretamente.

Agora que temos uma lista de todos os atributos que queremos adicionar à nossa classe, vamos adicioná-los ao nosso código. Uma vez que queremos que estes atributos sejam únicos para cada objeto (ou para cada estudante), devemos declará-los como atributos de objeto.

Por exemplo:

```
public class StudentRecord {
        private String name;
        private String address;
        private int age;
        private double mathGrade;
        private double scienceGrade;
}
```

onde:

private significa que os atributos são acessíveis apenas de dentro da classe. Outros objetos não podem acessar diretamente estes atributos.

Atributos de classe ou atributos estáticos

Além dos atributos de objeto, podemos também declarar atributos de classe ou atributos que pertençam à classe como um todo. O valor desses atributos é o mesmo para todos os objetos da mesma classe. Suponha que queiramos saber o número total de registros criados para a classe. Podemos declarar um atributo estático que armazenará este valor. Vamos chamá-lo de **studentCount**.

Para declarar um atributo estático:

```
public class StudentRecord {
// atributos de objeto declarados anteriormente
private static int studentCount;
}
```

usamos a palavra-chave static para indicar que é um atributo estático.

Então, nosso código completo deve estar assim:

```
public class StudentRecord {
private String name;
private String address;
private int age;
private double mathGrade;
private double englishGrade;
private double scienceGrade;
private static int studentCount;
}
```

Declarando métodos

Antes de discutirmos quais métodos a nossa classe deverá conter, vejamos a sintaxe geral usada para a declaração de métodos.

Para declararmos métodos, escrevemos:

```
<modificador>* <tipoRetorno> <nome>(<argumento>*) {
        <instruções>*
}
```

onde:

<modificador> pode ser utilizado qualquer modificador de acesso
<tipoRetorno> pode ser qualquer tipo de dado (incluindo void)
<nome> pode ser qualquer identificador válido
<argumento> argumentos recebidos pelo método separados por vírgulas. São definidos por:

<tipoArgumento> <nomeArgumento>

Métodos assessores

Para que se possa implementar o princípio do encapsulamento, isto é, não permitir que quaisquer objetos acessem os nossos dados de qualquer modo, declaramos campos, ou atributos, da nossa classe como particulares.

Entretanto, há momentos em que queremos que outros objetos acessem esses dados particulares. Para que possamos fazer isso, criamos métodos assessores.

Métodos assessores são usados para ler valores de atributos de objeto ou de classe. O método assessor recebe o nome de get<NomeDoAtributo>. Ele retorna um valor.

Para o nosso exemplo, queremos um método que possa ler o nome, endereço, nota de inglês, nota de matemática e nota de ciências do estudante.

Vamos dar uma olhada na implementação deste:

```
public class StudentRecord {
        private String name;
        :
        :
        public String getName() {
        return name;
        }
}
```

onde:

public significa que o método pode ser chamado por objetos externos à classe

String é o tipo do retorno do método. Isto significa que o método deve retornar um valor de tipo **String**.

getName o nome do método () significa que o nosso método não tem nenhum argumento

A instrução:

return name;

no método, significa que retornará o conteúdo do atributo **name** ao método que o chamou. Note que o tipo do retorno do método deve ser do mesmo tipo do atributo utilizado na declaração **return**. O seguinte erro de compilação ocorrerá caso o método e o atributo de retorno não tenham o mesmo tipo de dados:

```
StudentRecord.java:14: incompatible types
found : int
required: java.lang.String
                return name;
                       ^
1 error
```

Outro exemplo de um método assessor é o método getAverage:

```
public class StudentRecord {
        private String name;
        :
        :
        public double getAverage(){
                double result = 0;
                result =
        (mathGrade+englishGrade+scienceGrade)/3;
        return result;
        }
}
```

O método **getAverage** calcula a média das 3 notas e retorna o resultado.

Métodos modificadores

Para que outros objetos possam modificar os nossos dados, disponibilizamos métodos que possam gravar ou modificar os valores dos atributos de objeto ou de classe. Chamamos a esses métodos modificadores. Esse método é escrito como set<NomeDoAtributoDeObjeto>.

Vamos dar uma olhada na implementação de um método modificador:

```
public class StudentRecord {
        private String name;
```

```
        :
        :
        public void setName(String temp) {
                name = temp;
        }
}
```

onde:

public significa que o método pode ser chamado por objetos externos à classe

void significa que o método não retorna valor

setName o nome do método

(String temp) argumento que será utilizado dentro do nosso método

A instrução:

name = temp;

atribui o conteúdo de temp para name e, portanto, altera os dados dentro do atributo de objeto name. Métodos modificadores não retornam valores, entretanto, eles devem receber um argumento com o mesmo tipo do atributo no qual estão tratando.

Múltiplos comandos return

É possível ter vários comandos return para um método desde que eles não pertençam ao mesmo bloco. É possível utilizar constantes para retornar valores, em vez de atributos.

Por exemplo, considere o método:

```
public String getNumberInWords(int num) {
        String defaultNum = "zero";
        if (num == 1) {
                return "one"; // retorna uma constante
        } else if( num == 2) {
                return "two"; // retorna uma constante
        }
                // retorna um atributo
        return defaultNum;
}
```

Métodos estáticos

Para o atributo estático **studentCount**, podemos criar um método estático para obter o seu conteúdo.

```
public class StudentRecord {
        private static int studentCount;
        public static int getStudentCount(){
                return studentCount;
        }
}
```

Dicas de programação:
1. Nomes de métodos devem iniciar com letra MINÚSCULA.
2. Nomes de métodos devem conter verbos
3. Sempre faça documentação antes da declaração do método. Use o estilo **javadoc** para isso.

onde:

public significa que o método pode ser chamado por objetos externos à classe.

static significa que o método é estático e deve ser chamado digitando-se [NomeClasse].[nomeMétodo].

int é o tipo do retorno do método. Significa que o método deve retornar um valor de tipo int.

getStudentCount nome do método.

() significa que o método não tem nenhum argumento.

Por enquanto, getStudentCount retornará sempre o valor zero já que ainda não fizemos nada na nossa classe para atribuir o seu valor. Modificaremos o valor de studentCount mais tarde, quando discutirmos construtores.

Exemplo de código fonte para a classe StudentRecord

Aqui está o código para a nossa classe **StudentRecord**:

```
public class StudentRecord {
        private String name;
        private String address;
        private int age;
        private double mathGrade;
```

```
        private double englishGrade;
        private double scienceGrade;
        private static int studentCount;
        /**
         * Retorna o nome do estudante
        */
        public String getName(){
                return name;
        }
        /**
         * Muda o nome do estudante
        */
        public void setName( String temp ){
                name = temp;
        }
        // outros métodos modificadores aqui ....
        /**
         * Calcula a média das classes de inglês, matemática
         * e ciências
        */
        public double getAverage(){
                double result = 0;
                result =
        (mathGrade+englishGrade+scienceGrade)/3;
                return result;
        }
        /**
         * Retorna o número de ocorrências em StudentRecords
        */
        public static int getStudentCount(){
                return studentCount;
        }
}
```

Aqui está um exemplo do código de uma classe que utiliza a nossa classe **StudentRecord**.

```
public class StudentRecordExample {
public static void main( String[] args ){
        // criar três objetos para StudentRecord
        StudentRecord annaRecord = new StudentRecord();
        StudentRecord beahRecord = new StudentRecord();
        StudentRecord crisRecord = new StudentRecord();
        // enviar o nome dos estudantes
        annaRecord.setName("Anna");
        beahRecord.setName("Beah");
```

```
        crisRecord.setName("Cris");
        // mostrar o nome de anna
        System.out.println(annaRecord.getName());
        //mostrar o número de estudantes
        System.out.println("Count=" +
                StudentRecord.getStudentCount());
        }
}
```

A saída dessa classe é:

Anna

Count = 0

This

O objeto **this** é usado para acessar atributos de objeto ou métodos da classe. Para entender isso melhor, tomemos o método **setAge** como exemplo.

Suponha que tenhamos o seguinte método para **setAge**:

```
public void setAge(int age){
        age = age; // Não é uma boa prática
}
```

O nome do argumento nesta declaração é **age**, que tem o mesmo nome do atributo de objeto **age**. Já que o argumento **age** é a declaração mais próxima do método, o valor do argumento **age** será usado. Na instrução:

age = age;

estamos simplesmente associando o valor do argumento age para si mesmo.

Isso não é o que queremos que aconteça no nosso código. A fim de corrigir esse erro, usamos o objeto this. Para utilizar o objeto this, digitamos:

this.<nomeDoAtributo>

O ideal é reescrever o nosso método do seguinte modo:

```
public void setAge(int age){
```

```
        this.age = age;
}
```

Esse método irá atribuir o valor do argumento age para a atributo de objeto age do objeto StudentRecord.

Overloading de Métodos

Nas nossas classes, podemos necessitar de criar métodos que tenham os mesmos nomes, mas que funcionem de maneira diferente, dependendo dos argumentos que informamos. Essa capacidade é chamada de overloading de métodos.

Overloading de métodos permite que um método com o mesmo nome, entretanto com diferentes argumentos, possa ter implementações diferentes e retornar valores de diferentes tipos. Em vez de inventar novos nomes todas as vezes, o overloading de métodos pode ser utilizado quando a mesma operação tem implementações diferentes.

Por exemplo, na nossa classe **StudentRecord**, queremos ter um método que mostre as informações sobre o estudante. Entretanto, queremos que o método print mostre dados diferentes, dependendo dos argumentos que lhe informamos. Por exemplo, quando não enviamos qualquer argumento, queremos que o método **print** mostre o nome, endereço e idade do estudante.

Quando passamos 3 valores **double**, queremos que o método mostre o nome e as notas do estudante.

Temos os seguintes métodos dentro da nossa classe **StudentRecord**:

```
public void print(){
        System.out.println("Name:" + name);
        System.out.println("Address:" + address);
        System.out.println("Age:" + age);
}
public void print(double eGrade, double mGrade, double sGrade)
        System.out.println("Name:" + name);
        System.out.println("Math Grade:" + mGrade);
        System.out.println("English Grade:" + eGrade);
        System.out.println("Science Grade:" + sGrade);
}
```

Quando tentamos chamar esses métodos no método **main**, criado

para a classe **StudantRecordExample**:

```
public static void main(String[] args) {
        StudentRecord annaRecord = new StudentRecord();
        annaRecord.setName("Anna");
        annaRecord.setAddress("Philippines");
        annaRecord.setAge(15);
        annaRecord.setMathGrade(80);
        annaRecord.setEnglishGrade(95.5);
        annaRecord.setScienceGrade(100);
        // overloading de métodos
        annaRecord.print();
        annaRecord.print(
                annaRecord.getEnglishGrade(),
                annaRecord.getMathGrade(),
                annaRecord.getScienceGrade());
        }
```

teremos a saída para a primeira chamada ao método **print**:

```
Name:Anna
Address:Philippines
Age:15
```

e, em seguida, a saída para a segunda chamada ao método **print**:

```
Name:Anna
Math Grade:80.0
English Grade:95.5
Science Grade:100.0
```

Lembre-se sempre que métodos **overload** possuem as seguintes propriedades:

1. o mesmo nome
2. argumentos diferentes
3. tipo do retorno igual ou diferente

Declarando construtores

Discutimos anteriormente o conceito de construtores. Construtores são importantes na criação de um objeto. É um método onde são

colocadas todas as inicializações.

A seguir, temos as propriedades de um construtor:

1. Possuem o **mesmo nome da classe**

2. Construtor é um método, entretanto, somente as seguintes informações podem ser colocadas no cabeçalho do construtor:

   ○ Escopo ou identificador de acessibilidade (como **public**)

   ○ Nome do construtor

   ○ Argumentos, caso necessário

3. Não retornam valor

4. São executados automaticamente na utilização do operador **new** durante a instanciação da classe. Para declarar um construtor, escrevemos:

```
[modificador] <nomeClasse> (<argumento>*) {
<instrução>*
}
```

Construtor padrão (default)

Toda classe tem o seu construtor padrão. O **construtor padrão** é um construtor público e sem argumentos. Se não for definido um construtor para a classe, então, implicitamente, é assumido um construtor padrão.

Por exemplo, na nossa classe **StudentRecord**, o construtor padrão é definido do seguinte modo:

```
public StudentRecord() {
}
```

Overloading de construtores

Como mencionamos, construtores também podem sofrer overloading, por exemplo, temos aqui quatro construtores:

```
public StudentRecord() {
        // qualquer código de inicialização aqui
}
public StudentRecord(String temp){
        this.name = temp;
}
public StudentRecord(String name, String address) {
        this.name = name;
```

```
        this.address = address;
    }
    public StudentRecord(double mGrade, double eGrade,
    double sGrade) {
        mathGrade = mGrade;
        englishGrade = eGrade;
        scienceGrade = sGrade;
    }
```

Usando Construtores

Para utilizar esses construtores, temos as seguintes instruções:

```
public static void main(String[] args) {
        // criar três objetos para o registro do estudante
        StudentRecord annaRecord = new
                StudentRecord("Anna");
        StudentRecord beahRecord =
                new StudentRecord("Beah", "Philippines");
        StudentRecord crisRecord =
                new StudentRecord(80,90,100);
        // algum código aqui
}
```

Antes de continuarmos, vamos retornar o atributo estático **studentCount** que declaramos agora a pouco. O objetivo de studentCount é contar o número de objetos que são instanciados com a classe **StudentRecord**. Então, o que desejamos é incrementar o valor de studentCount toda vez que um objeto da classe **StudentRecord** é instanciado. Um bom local para modificar e incrementar o valor de **studentCount** é nos construtores, pois são sempre chamados toda vez que um objeto é instanciado. Por exemplo:

```
public StudentRecord() {
        studentCount++; // adicionar um estudante
}
public StudentRecord(String name) {
        studentCount++; // adicionar um estudante
        this.name = name;
}
public StudentRecord(String name, String address) {
        studentCount++; // adicionar um estudante
        this.name = name;
        this.address = address;
```

```
}
public StudentRecord(double mGrade, double eGrade,
double sGrade) {
        studentCount++; // adicionar um estudante
        mathGrade = mGrade;
        englishGrade = eGrade;
        scienceGrade = sGrade;
}
```

Utilizando o this()

Chamadas a construtores podem ser cruzadas, o que significa ser possível chamar um construtor de dentro de outro construtor. Usamos a chamada **this()** para isso. Por exemplo, dado o seguinte código,

```
public StudentRecord() {
        this("some string");
}
public StudentRecord(String temp) {
        this.name = temp;
}
public static void main( String[] args ) {
        StudentRecord annaRecord = new StudentRecord();
}
```

Dado o código acima, quando se executa a instrução do método **main**, será chamado o **primeiro** construtor. A instrução inicial deste construtor resultará na chamada ao **segundo** construtor.

Há algum detalhes que devem ser lembrados na utilização da chamada ao construtor por **this()**:

1. A chamada ao construtor **deve sempre ocorrer na primeira linha de instrução.**
2. **Utilizado para a chamada de um construtor.**

A chamada ao **this()** pode ser seguida por outras instruções.

Como boa prática de programação, é ideal nunca construir métodos que repitam as instruções. Buscamos a utilização de **overloading** com o objetivo de evitarmos essa repetição. Desse modo, reescreveremos os construtores da classe **StudentRecord** para:

```
public StudentRecord() {
        studentCount++; // adicionar um estudante
```

```
}
public StudentRecord(String name) {
        this();
        this.name = name;
}
public StudentRecord(String name, String address) {
        this(name);
        this.address = address;
}
public StudentRecord(double mGrade, double eGrade,
double sGrade) {
        this();
        mathGrade = mGrade;
        englishGrade = eGrade;
        scienceGrade = sGrade;
}
```

Pacotes

São utilizados para agrupar classes e interfaces relacionadas em uma única unidade (discutiremos interfaces mais tarde). Esta é uma característica poderosa que oferece um mecanismo para gerenciamento de um grande grupo de classes e interfaces e evita possíveis conflitos de nome.

Importando pacotes

Para utilizar classes externas ao pacote da classe, é necessário importar os pacotes dessas classes. Por padrão, todos as suas classes Java importam o pacote java.lang. É por isso que é possível utilizar classes como String e Integer dentro da sua classe, mesmo não tendo importado nenhum pacote explicitamente.

A sintaxe para importar pacotes é como segue:

import <nomeDoPacote>.<nomeDaClasse>;

Por exemplo, para a classe Color dentro do pacote awt, é necessário a seguinte instrução:

import java.awt.Color;

ou:

import java.awt.;*

A primeira linha de instrução importa especificamente a classe **Color** enquanto que a seguinte importa todas as classes do pacote **java.awt**.

Outra maneira de importar classes de outros pacotes é através da referência explícita ao pacote. Isto é feito utilizando-se o nome completo do pacote para declaração do objeto na classe:

java.awt.Color color;

Criando pacotes

Para criar os nossos pacotes, escrevemos:

package <nomeDoPacote>;

Suponha que desejamos criar um pacote em que colocaremos a nossa classe **StudentRecord** juntamente com outras classes relacionadas. Chamaremos o nosso pacote de **schoolClasses**.

A primeira coisa que temos a fazer é criar uma pasta chamada **schoolClasses**. Em seguida, copiar para essa pasta todas as classes que pertençam a esse pacote. Adicione a seguinte instrução no arquivo da classe; esta linha deve ser colocada antes da definição da classe. Por exemplo:

```
package schoolClasses;
        public class StudentRecord {
                // instruções da classe
}
```

Pacotes podem ser aninhados. Nesse caso, o interpretador espera que a estrutura de diretórios contendo as classes combinem com a hierarquia dos pacotes.

Definindo a variável de ambiente CLASSPATH

Suponha que coloquemos o pacote **schoolClasses** sob o diretório

C:\. Precisamos que a **classpath** aponte para este diretório de tal forma que, quando executemos a classe, a JVM seja capaz de enxergar o local onde está armazenada.

Antes de discutirmos como ajustar a variável **classpath**, vamos ver um exemplo sobre o que aconteceria se esta não fosse ajustada. Suponha que sigamos os passos para compilar e executar a classe **StudentRecord** que escrevemos:

```
C:\schoolClasses>javac StudentRecord.java
C:\schoolClasses>java StudentRecord
Exception in thread "main" java.lang.NoClassDefFoundError:
StudentRecord
(wrong name: schoolClasses/StudentRecord)
at java.lang.ClassLoader.defineClass1(Native Method)
at java.lang.ClassLoader.defineClass(Unknown Source)
at java.security.SecureClassLoader.defineClass(Unknown Source)
at java.net.URLClassLoader.defineClass(Unknown Source)
at java.net.URLClassLoader.access$100(Unknown Source)
at java.net.URLClassLoader$1.run(Unknown Source)
at java.security.AccessController.doPrivileged(Native Method)
at java.net.URLClassLoader.findClass(Unknown Source)
at java.lang.ClassLoader.loadClass(Unknown Source)
at sun.misc.Launcher$AppClassLoader.loadClass(Unknown Source)
at java.lang.ClassLoader.loadClass(Unknown Source)
at java.lang.ClassLoader.loadClassInternal(Unknown Source)
```

Surge o erro **NoClassDefFoundError,** que significa que o Java desconhece onde procurar por essa classe. A razão disso é que a sua classe **StudentRecord** pertence a um pacote denominado **schoolClasses**. Se desejamos executar essa classe, teremos que dizer ao Java o seu nome completo **schoolClasses.StudentRecord**. Também teremos que dizer à JVM onde procurar pelos nossos pacotes, que, nesse caso, é no C:\. Para fazer isso, devemos definir a variável **classpath**.

Para definir a variável **classpath** no Windows, digitamos o seguinte na linha de comando:

```
C:\schoolClasses>set classpath=C:\
```

onde C:\ é o diretório em que colocamos os pacotes. Após definir a variável classpath, poderemos executar a nossa classe em qualquer pasta, digitando:

```
C:\schoolClasses>java schoolClasses.StudentRecord
```

Para sistemas baseados no Unix, suponha que as nossas classes estejam no diretório usr/local/myClasses, escrevemos:

export classpath=/usr/local/myClasses

Observe que é possível definir a variável classpath em qualquer lugar. É possível definir mais de um local de pesquisa; basta separá-los por ponto-evírgula (no Windows) e dois-pontos (nos sistemas baseados em Unix). Por exemplo:

set classpath=C:\myClasses;D:\;E:\MyPrograms\Java

e para sistemas baseados no Unix:

export classpath=/usr/local/java:/usr/myClasses

Modificadores de acesso

Quando estamos criando as nossas classes e definindo as suas propriedades e métodos, queremos implementar algum tipo de restrição para se acessar esses dados. Por exemplo, ao necessitar que um certo atributo seja modificado apenas pelos métodos dentro da classe, é possível esconder isso dos outros objetos que estejam usando a sua classe. Para implementar isso, no Java, temos os **modificadores de acesso.**

Existem quatro diferentes tipos de modificadores de acesso: **public**, **private**, **protected** e **default**. Os três primeiros modificadores são escritos explicitamente no código para indicar o acesso. Para o tipo **default**, não se utiliza nenhuma palavra-chave.

Acesso padrão

Especifica que os elementos da classe são acessíveis somente aos métodos internos da classe e às suas subclasses. Não há palavra chave para o modificador **default**; sendo aplicado na ausência de um modificador de acesso. Por exemplo:

```
public class StudentRecord {
        // acesso padrão ao atributo
        int name;
```

```
        // acesso padrão para o método
        String getName(){
                return name;
        }
}
```

O atributo de objeto **name** e o método **getName()** podem ser acessados somente por métodos internos à classe e por subclasses de **StudentRecord**. Falaremos sobre subclasses em próximas lições.

Acesso público

Especifica que os elementos da classe são acessíveis tanto internamente quanto externamente à classe. Qualquer objeto que interage com a classe pode ter acesso aos elementos públicos da classe. Por exemplo:

```
public class StudentRecord {
        // acesso público o atributo
        public int name;
        // acesso público para o método
        public String getName(){
                return name;
        }
}
```

O atributo de objeto **name** e o método **getName()** podem ser acessados a partir de outros objetos.

Acesso protegido

Especifica que somente classes no mesmo pacote podem ter acesso aos atributos e métodos da classe. Por exemplo:

```
public class StudentRecord {
        //acesso protegido ao atributo
        protected int name;
        //acesso protegido para o método
        protected String getName(){
                return name;
        }
}
```

O atributo de objeto **name** e o método **getName()** podem ser acessados por outros objetos, desde que o objetos pertençam ao mesmo pacote da classe **StudentRecord**.

Acesso particular

Especifica que os elementos da classe são acessíveis somente na classe que o definiu. Por exemplo:

Dicas de programação:
Normalmente, os atributos de objeto de uma classe devem ser declarados particulares e a classe pode fornecer métodos assessores e modificadores para esses atributos.

```
public class StudentRecord {
        // acesso particular ao atributo
        private int name;
        // acesso particular para o método
        private String getName(){
                return name;
        }
}
```

O atributo de objeto name e o método getName() podem ser acessados somente por métodos internos à classe.



Exercícios Propostos

1. Sua tarefa é criar uma classe que contenha um Registro de Agenda. A tabela 1 descreve as informações que um Registro de Agenda deve conter:

Atributos/Propriedades	Descrição
Nome	Nome da pessoa
Endereço	Endereço da pessoa
Número de Telefone	Número de telefone da pessoa
email	Endereço eletrônico da pessoa

Crie os seguintes métodos:

a. Forneça todos os métodos assessores e modificadores necessários para

todos os atributos.

b. Construtores.

2. Crie uma classe Agenda que possa conter entradas de objetos tipo Registro de Agenda (utilize a classe criada no primeiro exercício). Devem ser oferecidos os seguintes métodos para a agenda:

a. Adicionar registro

b. Excluir registro

c. Visualizar registros

d. Modificar um registro

Desafio

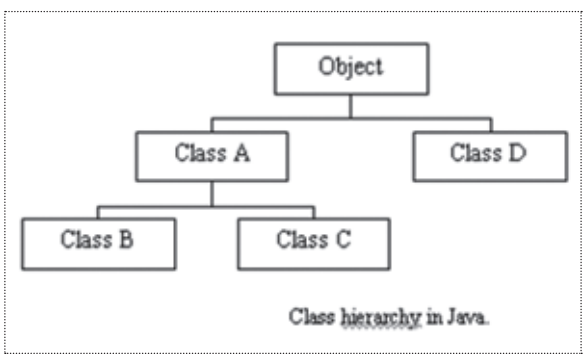
• A empresa MagSoft contrata desenvolvedores para trabalhar em projetos de software. Implemente uma solução em Java para o controle do empregados e projetos, usando pacote Java.

Herança, Polimorfismo e Interfaces

Herança

Todas as classes, incluindo as que compõem a API Java, são subclasses da classe Object. Um exemplo de hierarquia de classes é mostrado com a figura, abaixo.

A partir de uma determinada classe, qualquer classe acima desta, na hierarquia de classes, é conhecida como uma **superclasse** (ou classe Pai). Enquanto que qualquer classe abaixo, na hierarquia de classes, é conhecia como uma **subclasse** (ou classe Filho).



Class hierarchy in Java.

Herança é um dos principais princípios em orientação a objeto. Um comportamento (método)

é definido e codificado uma única vez em uma única classe, e esse comportamento é herdado por todas suas subclasses. Uma subclasse precisa apenas implementar as diferenças em relação a sua classe pai, ou seja, adaptar-se ao meio em que vive.

Definindo superclasses e subclasses

Para herdar uma classe, usamos a palavra-chave **extends**. Ilustraremos criando uma classe pai como exemplo. Suponha que tenhamos uma classe pai chamada **Person**.

```
public class Person {
        protected String name;
        protected String address;
        /**
        * Construtor Padrão
        */
        public Person(){
                System.out.println("Inside Person:Constructor");
                name = "";
                address = "";
        }
        /**
        * Construtor com 2 parâmetros
        */
        public Person( String name, String address ){
                this.name = name;
                this.address = address;
        }
        /**
        * Métodos modificadores e acessores
        */
        public String getName(){
                return name;
        }
        public String getAddress(){
                return address;
        }
        public void setName( String name ){
                this.name = name;
        }
        public void setAddress( String add ){
                this.address = add;
        }
}
```

Os atributos **name** e **address** são declarados como **protected**.

A razão de termos feito isso é que queremos que esses atributos sejam acessíveis às subclasses dessa classe. Se a declararmos com o modificador private, as subclasses não estarão aptas a usá-los. Todas as propriedades de uma superclasse que são declaradas como **public**, **protected** e **default** podem ser acessadas por suas subclasses.

Vamos criar outra classe chamada **Student**. E, como um estudante também é uma pessoa, concluímos que iremos estender a classe **Person**, então, poderemos herdar todas as propriedades existentes na classe **Person**. Para isso, escrevemos:

```
public class Student extends Person {
        public Student(){
                System.out.println("Inside Student:Constructor");
        //Algum código aqui
        }
        // Algum código aqui
}
```

O fluxo de controle é mostrado na figura.

Quando a classe **Student** for instanciada, o construtor padrão da superclasse **Person** é invocado implicitamente para fazer as inicializações necessárias. Após isso, as instruções dentro do construtor da subclasse são executadas. Para ilustrar, considere o seguinte código:

```
public static void main( String[] args ){
        Student anna = new Student();
}
```

No código, criamos um objeto da classe **Student**. O resultado da execução deste programa é:

```
Inside Person:Constructor
Inside Student:Constructor
```

Super

Uma subclasse pode, explicitamente, chamar um construtor de sua superclasse imediata. Isso é feito utilizando uma chamada ao objeto **super**. Uma chamada ao **super** no construtor de uma subclasse

irá resultar na execução de um construtor específico da superclasse, baseado nos argumentos passados.

Por exemplo, dada a seguinte instrução para a classe **Student**:

```
public Student(){
        super( "SomeName", "SomeAddress" );
        System.out.println("Inside Student:Constructor");
}
```

Este código chama o segundo construtor de sua superclasse imediata (a classe **Person**) e a executa. Outro código de exemplo é mostrado abaixo:

```
public Student(){
        super();
        System.out.println("Inside Student:Constructor");
}
```

Este código chama o construtor padrão de sua superclasse imediata (a classe **Person**) e o executa.

Devemos relembrar, quando usamos uma chamada ao objeto super:

1. A instrução **super()** deve ser a primeira instrução em um construtor.

2. As instruções **this()** e **super()** não podem ocorrer simultaneamente no mesmo construtor.

O objeto **super** é uma referência aos membros da superclasse (assim como o objeto this é da sua própria classe). Por exemplo:

```
public Student() {
super.name = "person name"; // Nome da classe pai
this.name = "student name"; // Nome da classe atual
{
```

Override de Métodos

Se, por alguma razão, uma classe derivada necessita que a implementação de algum método seja diferente da superclasse, o polimorfismo por override pode vir a ser muito útil. Uma subclasse pode modificar um método definido em sua superclasse fornecendo uma nova

implementação para aquele método.

Supondo que tenhamos a seguinte implementação para o método getName da superclasse Person:

```
public class Person {
...
public String getName(){
        System.out.println("Parent: getName");
        return name;
{
...
{
```

Para realizar um **polimorfismo por override** no método **getName** da subclasse Student, escrevemos:

```
public class Student extends Person {
...
public String getName(){
        System.out.println("Student: getName");
        return name;
{
...
{
```

Então, quando invocarmos o método **getName** de um objeto da classe **Student**, o método chamado será o de **Student**, e a saída será:

```
Student: getName
```

É possível chamar o método **getName** da superclasse, basta, para isso:

```
public class Student extends Person {
...
public String getName() {
        super.getName();
        System.out.println("Student: getName");
        return name;
{
...
{
```

Inserimos uma chamada ao objeto super, e a saída será:
Parent: getName Student: getName

Método final e classe final

Podemos declarar classes que não permitem a herança. Essas classes são chamadas classes finais. Para definir que uma classe seja **final**, adicionamos a palavra-chave final na declaração da classe (na posição do modificador). Por exemplo, desejamos que a classe **Person** não possa ser herdada por nenhuma outra classe, escrevemos:

```
public final class Person {
        // Código da classe aqui
}
```

Muitas classes na API Java são declaradas **final** para certificar que seu comportamento não seja herdado e, possivelmente , modificado. Exemplos, são as classes Integer, Double e Math.

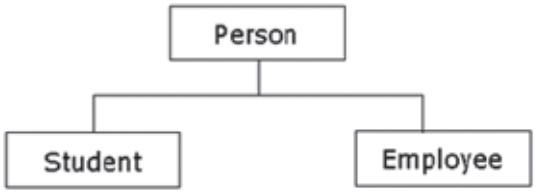
Também é possível criar métodos que não possam ser modificados pelos filhos, impedindo o polimorfismo por override. Esses métodos são o que chamamos de métodos finais. Para declarar um método final, adicionamos a palavra-chave final na declaração do método (na posição do modificador). Por exemplo, se queremos que o método getName da classe **Person** não possa ser modificado, escrevemos:

```
public final String getName(){
        return name;
}
```

Caso o programador tente herdar uma classe final, ocorrerá um erro de compilação. O mesmo acontecerá ao se tentar fazer um override de um método final.

Polimorfismo

Considerando a classe pai **Person** e a subclasse **Student** do exemplo anterior, adicionaremos outra subclasse a **Person**, que se chamará **Employee**. Abaixo está a hierarquia de classes que ilustra o cenário:



Podemos criar uma referência do tipo da superclasse para a subclasse. Por exemplo:

```
public static main( String[] args ) {
Person ref;
Student studentObject = new Student();
Employee employeeObject = new Employee();
ref = studentObject; //Person ref: ponteiro para um Student
// algum código aqui
```

Supondo que tenhamos um método **getName** em nossa superclasse **Person**, iremos realizar uma modificação desse método nas subclasses **Student** e **Employee**:

```
public class Person {
public String getName(){
        System.out.println("Person Name:" + name);
        return name;
        }
}
        public class Student extends Person {
        public String getName(){
                System.out.println("Student Name:" + name);
                return name;
        }
}
        public class Employee extends Person {
        public String getName(){
                System.out.println("Employee Name:" + name);
                return name;
        }
}
```

Voltando ao método **main**, quando tentamos chamar o método **getName** da referência ref do tipo **Person**, o método **getName** do objeto

Student será chamado. Agora, se atribuirmos **ref** ao objeto **Employee**, o método **getName** de **Employee** será chamado.

```
public static main(String[] args) {
Person ref;
Student studentObject = new Student();
Employee employeeObject = new Employee();
ref = studentObject; //ponteiro de referência para um Student
String temp = ref.getName(); //getName de Student é chamado
System.out.println(temp);
ref = employeeObject; // ponteiro de referência Person para um
// objeto Employee
String temp = ref.getName(); // getName de Employee
// classe é chamada
System.out.println(temp);
```

A capacidade de uma referência mudar de comportamento de acordo com o objeto a que se refere é chamada de polimorfismo. O **polimorfismo** permite que múltiplos objetos de diferentes subclasses sejam tratados como objetos de uma única superclasse, e que automaticamente sejam selecionados os métodos adequados a serem aplicados a um objeto em particular, baseado na subclasse a que ele pertença. Outro exemplo que demonstra o polimorfismo é realizado ao passar uma referência a métodos.

Supondo que exista um método estático **printInformation** que recebe como parâmetro um objeto do tipo **Person**, pode-se passar uma referência do tipo **Employee** e do tipo **Student**, porque são subclasses do tipo **Person**.

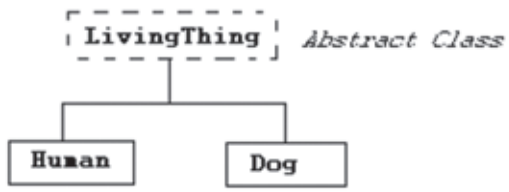
```
public static main(String[] args) {
        Student studentObject = new Student();
        Employee employeeObject = new Employee();
        printInformation(studentObject);
        printInformation(employeeObject);
}
public static printInformation(Person p){
        ...
}
```

Classes abstratas

Para criar métodos em classes, devemos, necessariamente, saber

qual o seu comportamento. Entretanto, em muitos casos não sabemos como estes métodos se comportarão na classe que estamos criando, e, por mera questão de padronização, desejamos que as classes que herdem dessa classe possuam, obrigatoriamente, esses métodos.

Por exemplo, queremos criar uma superclasse chamada **LivingThing**. Esta classe tem certos métodos como **breath**, **sleep** e **walk**. Entretanto, existem tantos métodos nesta superclasse que não podemos generalizar esse comportamento. Tome por exemplo, o método walk (andar). Nem todos os seres vivos andam da mesma maneira. Tomando os humanos como exemplo, os humanos andam sobre duas pernas, enquanto que outros seres vivos como os cães andam sobre quatro. Entretanto, existem muitas características que os seres vivos têm em comum, isso é o que nós queremos ao criar uma superclasse geral.



Para realizarmos isso, teremos que criar uma superclasse que possua alguns métodos com implementações e outros não. Este tipo de classe é chamada de **classe abstrata.**

Uma classe abstrata é uma classe que não pode gerar um objeto. Frequentemente aparece no topo de uma hierarquia de classes no modelo de programação orientada a objetos.

Os métodos nas **classes abstratas** que não têm implementação são chamados de **métodos abstratos.** Para criar um método abstrato, apenas escreva a assinatura do método sem o corpo e use a palavra-chave **abstract**. Por exemplo:

```
public abstract void someMethod();
```

Agora, vamos criar um exemplo de classe abstrata:

```
public abstract class LivingThing {
public void breath(){
        System.out.println("Living Thing breathing...");
}
public void eat(){
        System.out.println("Living Thing eating...");
```

```
}
/**
* método abstrato walk
* Queremos que esse método seja criado pela
* subclasse de LivingThing
*/
public abstract void walk();
}
```

Quando uma classe estende a classe abstrata LivingThing, ela é obrigada a implementar o método abstrato walk. Por exemplo:

```
public class Human extends LivingThing {
        public void walk(){
                System.out.println("Human walks...");
        }
}
```

Dicas de programação:
Use classes abstratas para definir muitos tipos de comportamentos no topo de uma hierarquia de classes de programação orientada a objetos. Use suas subclasses para prover detalhes de implementação da classe abstrata.

Se a classe Human não implementar o método **walk**, será mostrada a seguinte mensagem de erro de compilação:

```
Human.java:1: Human is not abstract and does not override
abstract
method walk() in LivingThing
public class Human extends LivingThing
       ^
1 error
```

Interfaces

Uma interface é um tipo especial de classe que contém unicamente métodos abstratos ou atributos finais. Interfaces, por natureza, são abstratas.

Interfaces definem um padrão e o caminho público para especificação do comportamento de classes. Permitem que classes, independente de sua localização na estrutura hierárquica, implementem

comportamentos comuns.

Porque utilizar interfaces?

Utilizamos interfaces quando queremos classes não relacionadas que implementem métodos similares. Através de interfaces, podemos obter semelhanças entre classes não relacionadas sem forçar um relacionamento artificial entre elas.

Tomemos como exemplo a classe **Line**, contém métodos que obtém o tamanho da linha e compara o objeto **Line** com objetos de mesma classe. Considere também que tenhamos outra classe, **MyInteger**, que contém métodos que comparam um objeto **MyInteger** com objetos da mesma classe. Podemos ver que ambas classes têm os mesmos métodos similares que os comparam com outros objetos do mesmo tipo, entretanto eles não são relacionados. Para se ter certeza de que essas classes implementem os mesmos métodos com as mesmas assinaturas, utilizamos as interfaces. Podemos criar uma interface Relation que terá declarada algumas assinaturas de métodos de comparação. A interface Relation pode ser implementada da seguinte forma:

```
public interface Relation {
        public boolean isGreater(Object a, Object b);
        public boolean isLess(Object a, Object b);
        public boolean isEqual(Object a, Object b);
}
```

Outra razão para se utilizar interfaces na programação de objetos é revelar uma interface de programação de objeto sem revelar essas classes. Como veremos mais adiante, podemos utilizar uma interface como tipo de dados.

Finalmente, precisamos utilizar interfaces como mecanismo alternativo para herança múltipla, permitindo que as classes tenham mais de uma superclasse. A herança múltipla não está implementada em Java.

Interface vs. classe abstrata

A principal diferença entre uma **interface** e uma **classe abstrata** é que a classe abstrata pode possuir métodos implementados (reais) ou não implementados (abstratos). Na interface, todos os métodos são

obrigatoriamente abstratos e públicos, tanto que para essa classe, a palavrachave **abstract** ou **public** é opcional.

Interface vs. classe

Uma característica comum entre uma interface e uma classe é que ambas são tipos. Isso significa que uma interface pode ser usada no lugar onde uma classe é esperada. Por exemplo, dadas a classe Person e a interface PersonInterface, as seguintes declarações são válidas:

```
PersonInterface pi = new Person();
Person pc = new Person();
```

Entretanto, não se pode criar uma instância de uma interface sem implementá-la. Um exemplo disso é:

```
PersonInterface pi = new PersonInterface(); //ERRO DE
                                            //COMPILAÇÃO !!!
```

Outra característica comum é que ambas, interfaces e classes, podem definir métodos, embora uma interface não possa tê-los implementados. Já uma classe pode.

Criando interfaces

Para criarmos uma interface, utilizamos:

```
[public] [abstract] interface <NomeDaInterface> {
< [public] [final] <tipoAtributo> <atributo> = <valorInicial>; >*
< [public] [abstract] <retorno>
<nomeMetodo>(<parametro>*); >*
}
```

Como exemplo, criaremos uma interface que defina o relacionamento entre dois objetos de acordo com a "ordem natural" dos objetos:

```
interface Relation {
        boolean isGreater(Object a, Object b);
```

```
        boolean isLess(Object a, Object b);
        boolean isEqual( Object a, Object b);
}
```

Para implementar esta **interface**, usaremos a palavra chave "implements". Por exemplo:

```
/**
* Esta classe define um segmento de linha
*/
public class Line implements Relation {
private double x1;
private double x2;
private double y1;
private double y2;
public Line(double x1, double x2, double y1, double y2) {
        this.x1 = x1;
        this.x2 = x2;
        this.y2 = y2;
        this.y1 = y1;
}
public double getLength(){
        double length = Math.sqrt((x2-x1)*(x2-x1)+(y2-y1)*(y2-y1));
        return length;
}
public boolean isGreater( Object a, Object b){
        double aLen = ((Line)a).getLength();
        double bLen = ((Line)b).getLength();
        return (aLen > bLen);
}
public boolean isLess( Object a, Object b){
        double aLen = ((Line)a).getLength();
        double bLen = ((Line)b).getLength();
        return (aLen < bLen);
}
```

Dicas de programação:
Use interface para criar uma definição padrão de métodos em classes diferentes. Uma vez que o conjunto de definições de métodos é criado, pode ser escrito um método simples para manipular todas as classes que implementam a interface.

```
public boolean isEqual( Object a, Object b){
        double aLen = ((Line)a).getLength();
        double bLen = ((Line)b).getLength();
```

```
        return (aLen == bLen);
    }
```

Quando a classe implementa uma interface, deve-se implementar **todos** os métodos desta, caso contrário será mostrado o erro:

```
Line.java:4: Line is not abstract and does not override abstract
method isGreater(java.lang.Object,java.lang.Object) in Relation
public class Line implements Relation
                ^
1 error
```

Relacionamento de uma Interface para uma classe

Como vimos nas seções anteriores, a classe pode implementar uma interface e para isso, prover o código de implementação para todos os métodos definidos na interface.

Outro detalhe a se notar na relação entre uma interface e uma classe é que a classe não pode apenas estender uma única superclasse, mas pode implementar diversas interfaces. Um exemplo de uma classe que implementa diversas interfaces:

```
public class Person
implements PersonInterface, LivingThing, WhateverInterface {
        //algumas linhas de código
```

Outro exemplo de uma classe que estende outra superclasse e implementa interfaces:

```
public class ComputerScienceStudent extends Student
implements PersonInterface, LivingThing {
        // algumas linhas de código
```

Uma interface não é parte uma hierarquia de classes. Classes não relacionadas podem implementar a mesma interface.

Herança entre interfaces

Interfaces não são partes de uma hierarquia de classes. Entretanto, interfaces podem ter relacionamentos entre si. Por exemplo, suponha que tenhamos duas interfaces, StudentInterface e PersonInterface. Se StudentInterface estende PersonInterface, esta herda todos os métodos declarados em PersonInteface.

```
public interface PersonInterface {

}
public interface StudentInterface extends PersonInterface {

}
```

Exercícios Propostos



1. Neste exercício, queremos criar um registro mais especializado de **Student** que contém informações adicionais sobre um estudante de Ciência da Computação. Sua tarefa é estender a classe **StudentRecord** que foi implementada nas lições anteriores e acrescentar atributos e métodos que são necessários para um registro de um estudante de Ciência da Computação. Utilize **override** para modificar alguns métodos da super.
2. Crie uma classe abstrata chamada **Shape** com os métodos abstratos **getArea()** e **getName().** Escreva duas de suas subclasses Circle e Square. E acrescente métodos adicionais a estas subclasses. classe **StudentRecord**, caso seja necessário.

Tratamento Básico de Exceções

O que são exceções (Exception)?

Uma exceção é um evento que interrompe o fluxo normal de processamento de uma classe. Esse evento é um erro de algum tipoe e isso causa o término anormal da classe.

Estes são alguns dos exemplos de exceções que podem ter ocorridos em exercícios anteriores:

● **ArrayIndexOutOfBoundsException** ocorre ao acessar um elemento inexistente de um array.

● **NumberFormatException** ocorre ao enviar um parâmetro não-numérico para o método **Integer.parseInt()**.

Tratando exceções

Para tratar exceções em Java, utilizamos a declaração **try-catch-finally.** O que devemos fazer para proteger as instruções passíveis de gerar uma exceção, é inserí-las dentro deste bloco.

A forma geral de um **try-catch-finally** é:

```
try{
// escreva as instruções passíveis de gerar uma exceção
// neste bloco
} catch (<exceptionType1> <varName1>){
// escreva a ação que o seu programa fará caso ocorra
// uma exceção de um determinado
} . . .
} catch (<exceptionTypen> <varNamen>){
// escreva a ação que o seu programa fará caso ocorra
// uma exceção de um determinado tipo
} finally {
// escreva a ação que o seu programa executará caso ocorra
// ou não um erro ou exceção
}
```

Exceções geradas durante a execução do bloco **try** podem ser detectadas e tratadas num bloco **catch**. O código no bloco **finally** é sempre executado, ocorrendo ou não a exceção.

A seguir são mostrados os principais aspectos da sintaxe da construção de um **try-catchfinally**:

• A notação de bloco é obrigatória.

• Para cada bloco **try**, pode haver um ou mais blocos **catch**, mas somente um bloco **finally**.

• Um bloco try deve ser seguido de PELO MENOS um bloco **catch**

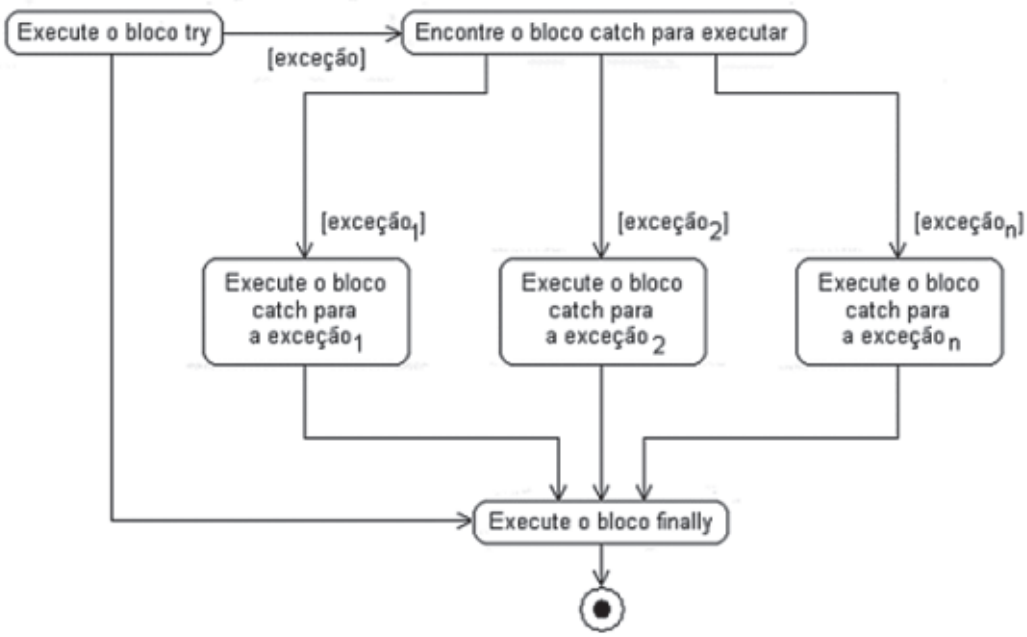
OU um bloco **finally**, ou ambos.

• Cada bloco **catch** define o tratamento de uma exceção.

• O cabeçalho do bloco **catch** recebe somente um argumento, que é a exceção (Exception) que este bloco pretende tratar.

• A exceção deve ser da classe **Throwable** ou de uma de suas subclasses.

Para um melhor entendimento, observe a figura 1 que demonstra o fluxo seguido pelo trycatch-finally:



Tomemos, por exemplo, uma classe que imprime o segundo argumento passado através da linha de comandos. Supondo que não há verificação no código para o número de argumentos.

```
public class ExceptionExample {
        public static void main( String[] args ) {
                System.out.println(args[1]);
                System.out.println("Finish");
        }
}
```

Ao executar esta classe sem informar nenhum argumento e, ao tentar acessar diretamente, conforme o exemplo descrito, o segundo argumento args[1], uma exceção é obtida que interromperá a execução normal do programa, e a seguinte mensagem será mostrada:

Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 1
at ExceptionExample.main(ExceptionExample.java:5)

Para prevenir que isso ocorra, podemos colocar o código dentro de um bloco **try-catch**. O bloco **finally** é opcional. Neste exemplo, não utilizaremos o bloco **finally**.

```
public class ExceptionExample
{
public static void main( String[] args ){
        try {
                System.out.println( args[1] );
        } catch (ArrayIndexOutOfBoundsException exp) {
                System.out.println("Exception caught!");
        }
        System.out.println("Finish");
}
}
```

Assim, quando tentarmos rodar o programa novamente sem a informação dos argumentos, a saída trataria a exceção e o fluxo do programa não seria interrompido, mostrando o resultado:

```
Exception caught!
Finish
```



Resumindo

Nesta unidade, apresentamos fundamentos do paradigma de programação orientada a objetos, caracterizando com exemplos uma linguagem orientada a objetos com Java; e os elementos inerentes como classes, objetos e métodos. Mostramos, com exemplos, características que podem ser exploradas como herança polimorfismo e interface e, ainda, como é feito o tratamento de exerções em linguagem OO como Java, apresentando exemplos ilustrativos e exercícios propostos a serem vencidos pelos aprendizes.

Exercícios Propostos



1. Capturando exceções 1

Dada a seguinte classe:

```
public class TestException {
public static void main(String[] args) {
        for (int i=0; true; i++) {
                System.out.println("args["+i+"]="+ args[i]);
        }
        System.out.println("Quiting...");
}
```

Compile e rode a classe **TestException**. E como saída será:

```
java TestExceptions one two three
args[0]=one
args[1]=two
args[2]=three
Exception in thread "main"
        java.lang.ArrayIndexOutOfBoundsException: 3
                at TestExceptions.main(1.java:4)
```

Modifique a classe **TestException** para tratar esta exceção. A saída depois do tratamento da exceção deverá ser:

```
java TestExceptions one two three
args[0]=one
args[1]=two
args[2]=three
Exception caught: java.lang.ArrayIndexOutOfBoundsException: 3
Quiting...
```

2. Capturando exceções 2

Há uma boa chance de que algumas classes escritas anteriormentes tenham disparados exceções. Como as exceções não foram tratadas, simplesmente interromperam a execução.

Retorne a esses programas e implemente o tratamento de exceções.

A01 – EXERCÍCIOS ADICIONAIS

1. Agenda Telefônica

Escrever um programa que cria uma agenda telefônica na qual seja possível acrescentar, excluir, visualizar e pesquisar os registros. O usuário deve ter a possibilidade de visualizar todos registros por ordem alfabética ou por ordem crescente de números de telefone. Na pesquisa por registros, o usuário deve ter a opção de pesquisar por nome ou por número de telefone. Na pesquisa pelo nome, o usuário deve ter uma opção em que possa selecionar se a pesquisa será efetuada com base no primeiro ou último nome.

MENU PRINCIPAL
1 – Adicionar registro na agenda telefônica
2 – Excluir registro da agenda telefônica
3 – Visualizar todos os registros
a – ordem alfabética
b – ordem numérica crescente de número de telefone
4 – Pesquisa de registros
a – por nome
1 – pelo primeiro nome
2 – pelo último nome
b – por número de telefone
5 – Sair

Esse é um exemplo da aplicação rodando:
Adicionar registro na agenda telefônica
Digite o Nome:
Digite o número do Telefone:
(* Se o registro já existir, avise o operador da existência do registro)
Visualizar todos os registros
Mostra todos os registros em ordem alfabética
Mostra todos os registros em ordem crescente de número de telefone
Pesquisa registros
Pesquisa agenda telefônica por nome
Pesquisa agenda telefônica pelo primeiro nome
Pesquisa agenda telefônica pelo último nome
Pesquisa agenda telefônica por número do telefone

Sair
Fechar agenda telefônica

2. Caça-Minas

Este jogo é uma versão simplificada do popular jogo de computador Caça-minas (minesweeper). Inicialmente, é questionado se o usuário quer jogar numa grade de 5x5 ou numa grade de 10x10. Você tem 2 arrays bidimensionais que contém informações sobre a grade selecionada. Um registro desse array pode conter 0 ou 1. O valor 1 significa que existe uma bomba nessa localização e o valor 0 se não existir.

Por exemplo, dado o seguinte array:

```
int bombList5by5[][]={{0, 0, 1, 0, 0},
{0, 0, 0, 0, 0},
{0, 1, 0, 0, 0},
{0, 0, 0, 1, 1},
{0, 1, 1, 0, 0}};
```

Dada a lista de bombas, temos 6 bombas nessa lista. As bombas estão localizadas nas células (linha,coluna), (0,2), (2,1), (3,3), (3,4), (4,1) e (4,2).

Se o usuário escolhe uma célula que contenha uma bomba, o jogo acaba e todas as bombas são mostradas. Se o usuário escolhe uma célula que não contenha uma bomba, um número é mostrado naquela posição indicando a quantidade de células vizinhas que contém bombas. O jogo deverá terminar quando todas as células que não contenham bombas tiverem sido marcadas (jogador vence) ou quando o usuário seleciona uma bomba (jogador perde).

Segue um exemplo de tela do jogo quando selecionada uma grade 5x5 que tenha o mesmo conteúdo do array bombList5by5 acima.

```
Benvindo ao Caça-Minas!
Escolha o tamanho da grade(Digite 1 para 5x5, Digite 2 para 10x10): 1
[ ] [ ] [ ] [ ] [ ]
[ ] [ ] [ ] [ ] [ ]
[ ] [ ] [ ] [ ] [ ]
[ ] [ ] [ ] [ ] [ ]
[ ] [ ] [ ] [ ] [ ]
Digite linha e coluna da célula que você quer abrir[linha  coluna]:
```

1 1

[ ] [ ] [ ] [ ] [ ]
[ ] [2] [ ] [ ] [ ]
[ ] [ ] [ ] [ ] [ ]
[ ] [ ] [ ] [ ] [ ]
[ ] [ ] [ ] [ ] [ ]

Digite linha e coluna da célula que você quer abrir[linha coluna]:

3 2

[ ] [ ] [ ] [ ] [ ]
[ ] [2] [ ] [ ] [ ]
[ ] [ ] [ ] [ ] [ ]
[ ] [ ] [4] [ ] [ ]
[ ] [ ] [ ] [ ] [ ]

Digite linha e coluna da célula que você quer abrir[linha coluna]:

0 2

[ ] [ ] [X] [ ] [ ]
[ ] [2] [ ] [ ] [ ]
[ ] [ ] [ ] [ ] [ ]
[ ] [ ] [4] [ ] [ ]
[ ] [ ] [ ] [ ] [ ]

Ooppps! Você pisou numa bomba. Sinto muito, o jogo acabou e você perdeu!

3. Conversão Numérica

Criar uma calculadora científica que converta os números digitados para as quatro representações numéricas: decimal, binário, octal e hexadecimal. O projeto deve gerar o seguinte menu na tela.

MENU PRINCIPAL:
Por favor, selecione o tipo de conversão:
1 – Binário para Decimal
2 – Decimal para Octal
3 – Octal para Hexadecimal
4 – Hexadecimal para Binário
5 – Sair

A seguinte tela deve ser mostrada quando uma das opções do menu for escolhida.

Seleção 1: Digite um número binário: 11000
11000 base 2 = 24 base 10 (volta para o menu principal)
Seleção 2: Digite um número Decimal: 24
24 base 10 = 30 base 8 (volta para o menu principal)
Seleção 3: Digite um número Octal: 30
30 base 8 = 18 base 16 (volta para o menu principal)
Seleção 4: Digite um número Hexadecimal: 18
18 base 16 = 11000 base 2
Seleção 1: Digite um número Binário: 110A
Número binário inválido!
Digite um número binário: 1
1 base 2 = 1 base 10 (volta para o menu principal)
Usuário selecionou 5
Tchau!



Referências

ASCENCIO, Ana Fernanda Gomes e CAMPOS, Edilene Aparecida Veneruchi. **Fundamentos da Programação de Computadores - Algoritmos, Pascal, C/C++ E Java**. Pearson, 2007.

BOENTE, Alfredo. **Aprendendo a programar em Java 2**: Orientado a Objetos. Brasport. Rio de Janeiro, 2003.

BORATTI, Isaias Camilo. **Programação Orientada a Objetos em Java**. Visual books, 2007.

CADENHEAD, Rogers. **Aprenda em 21 Dias Java 2.** 4ª Ed. Campus, 2005.

DEITEL, H. M. e DEITEL, P. J. **Java:** como programar. 3ª Ed. Porto Alegre: Bookman, 2001.

_____. **Java:** como programar. 6ª Ed. Porto Alegre: Pearson, 2005.

GONÇALVES, Edson.**Ciência modern**, 2007.

PRESSMAN, R. **Software Engineering:** A Pratctitioner´s Approach. 5ª Ed. São Paulo, Mc Graw-Hill, 2000.

RUBINSTEIN, Roberto. **Certificação Java 5 SERSON**, Brasport, 2006.

SOMMERVILLE, Ian. **Engenharia de Software**. 8º Ed. Pearson, 2004.



Sobre autor

Magno Santos

Doutorando em Biotecnologia pelo programa RENORBIO (2006). Mestre em Ciências da Computação pela Universidade Federal de Minas Gerais (1997). Graduado em Física pela Universidade Federal do Piauí (1991). Experiência em Computação e informática, atuando principalmente nas áreas de redes de computadores, modelagem de sistemas e bioinformática.

Currículo Lattes: *http://lattes.cnpq.br/5095626292200565*



www.uapi.ufpi.br